

数字后端设计流程

--后端设计全局观
数字后端做什么
从RTL到GDS

爱芯人课堂/QQ群187291154



内容

数字IC后端流程

关于后端学习

关于爱芯人课堂

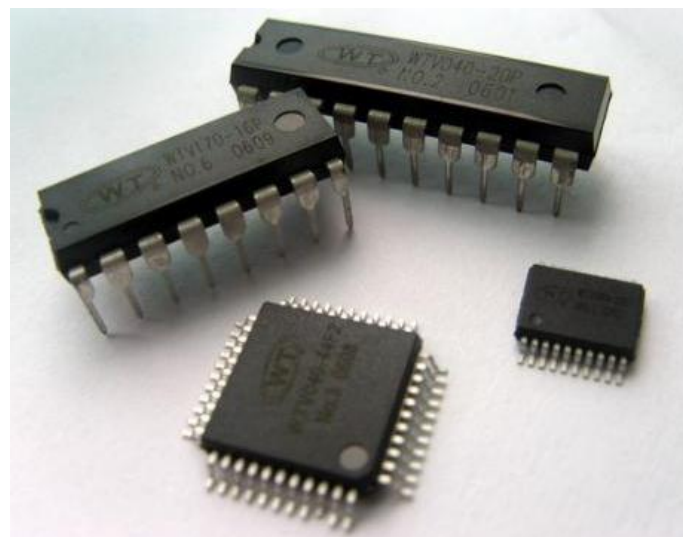
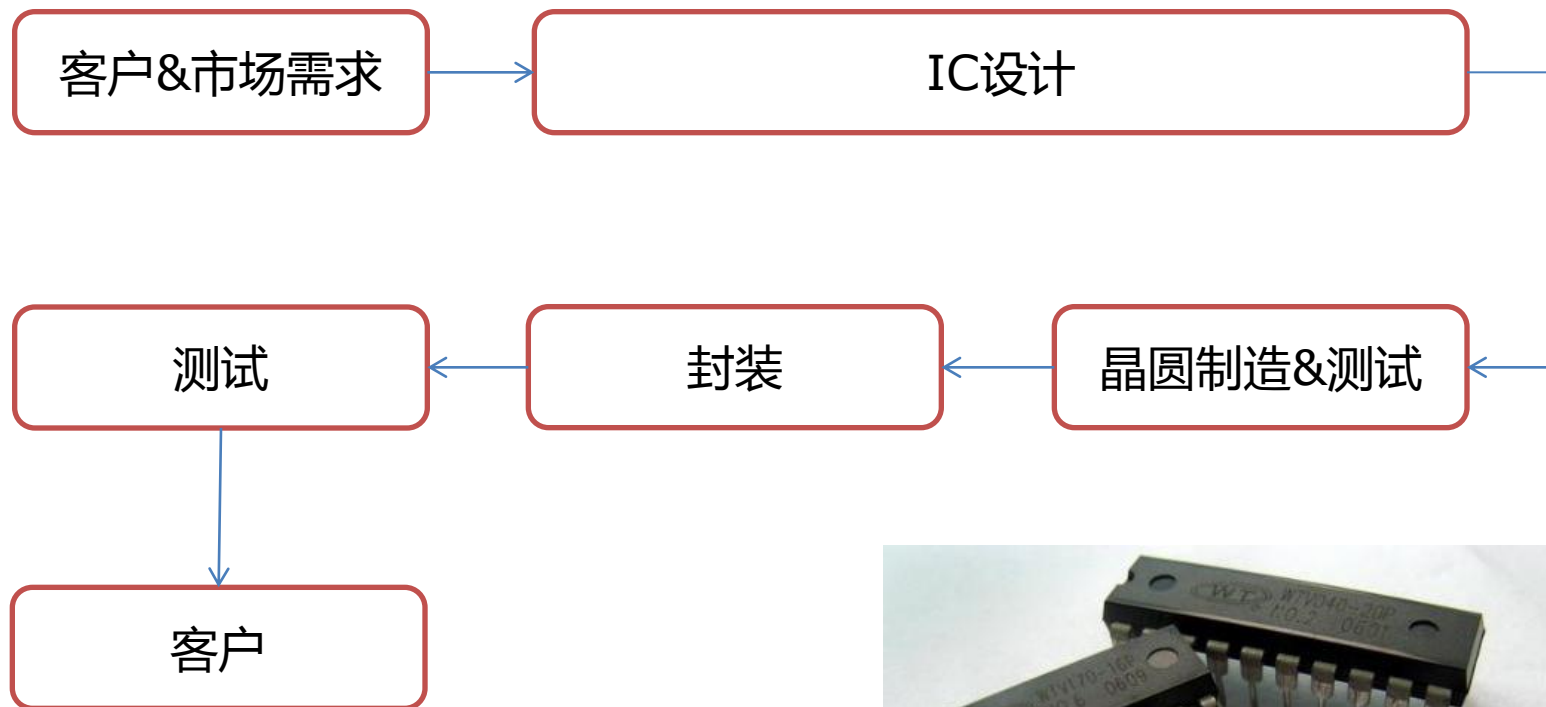
内容

数字IC后端流程

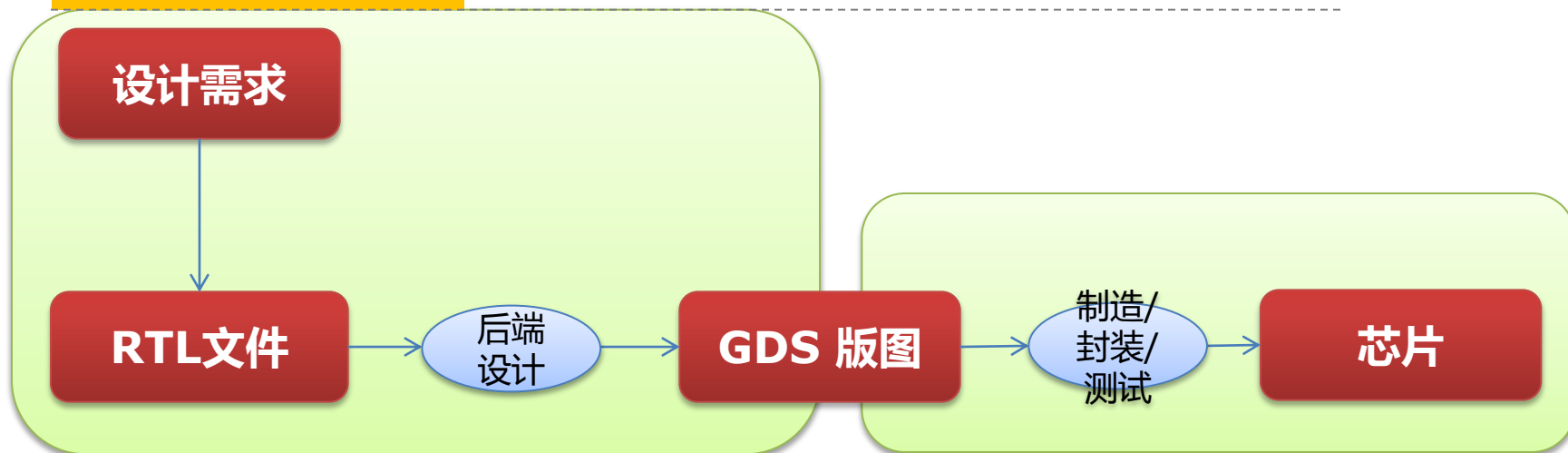
关于后端学习

关于爱芯人课堂

芯是怎么产生的？



从需求到芯片



由IC设计公司完成

由晶圆制造、封装测试等工厂完成

Notes :

1. 后端设计 : Backend Design , 前端设计 : Frontend Design
2. 晶圆制造 : Wafer Processing/Fabrication/Manufacturing , 晶圆制造厂 (如TSMC , SMIC) 也通常被称为fab,而没有制造厂的IC设计公司也通常被称为fabless公司 ;
3. 封装 : Package , 常见的工艺有Wire Bonding , Flip-Chip等 ;
4. 测试 : Testing/Wafer Test/ATE Test ;

RTL文件

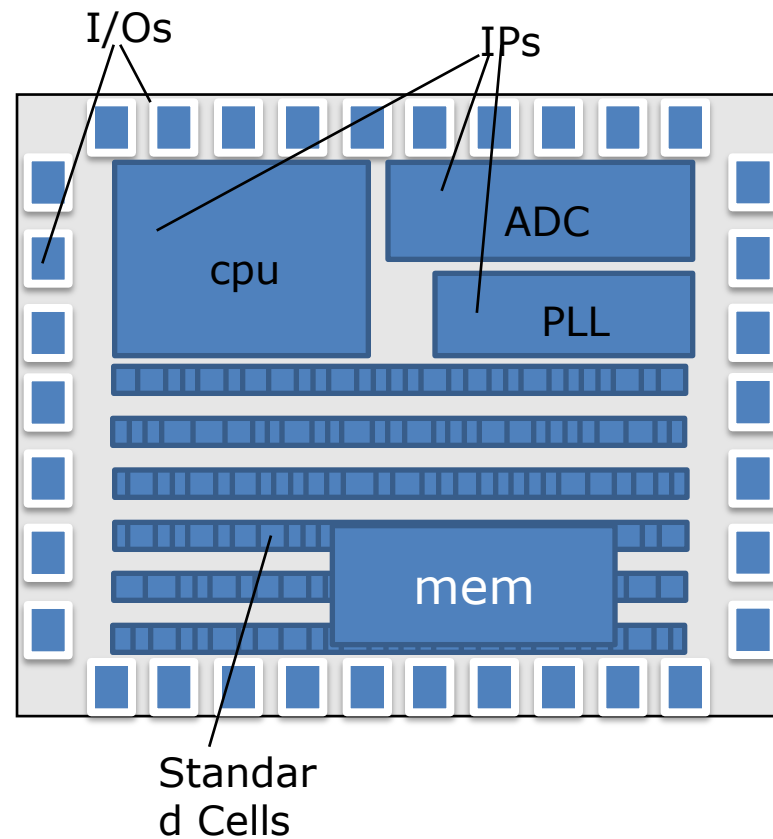
后端
设计

GDS 版图

制造/
封装/
测试

芯片

GDS: Geometry Data Standard



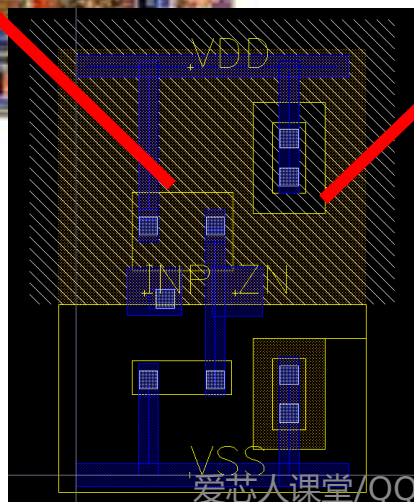
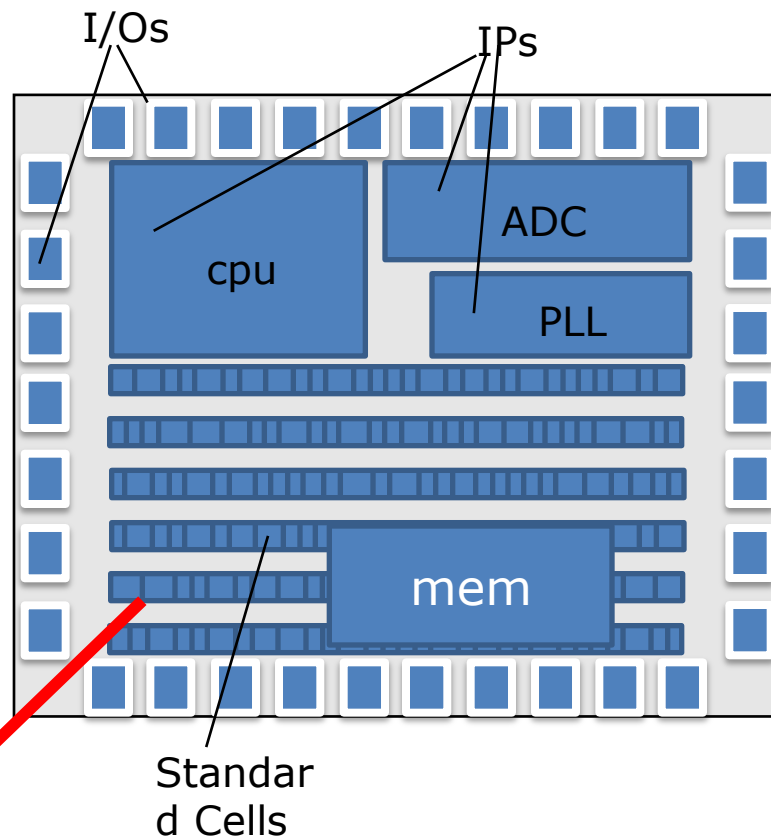
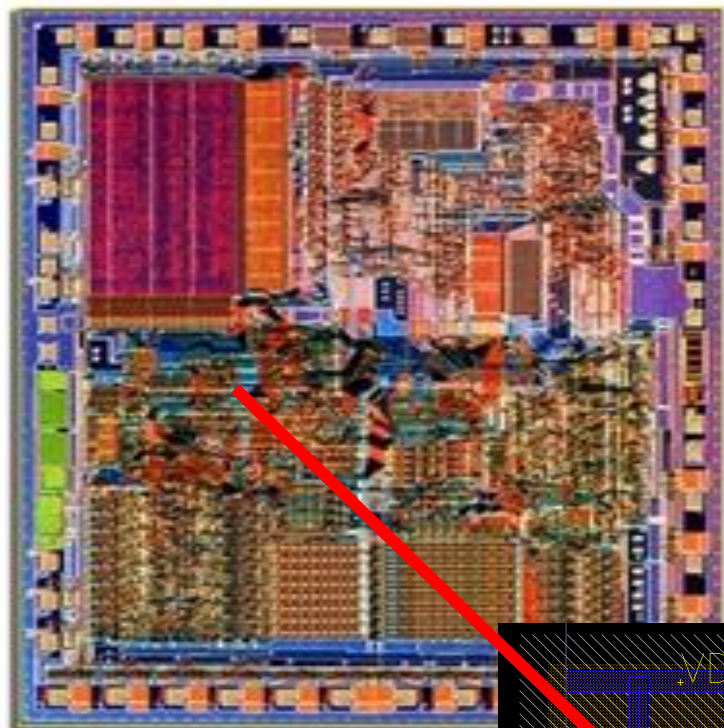
RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片



RTL文件

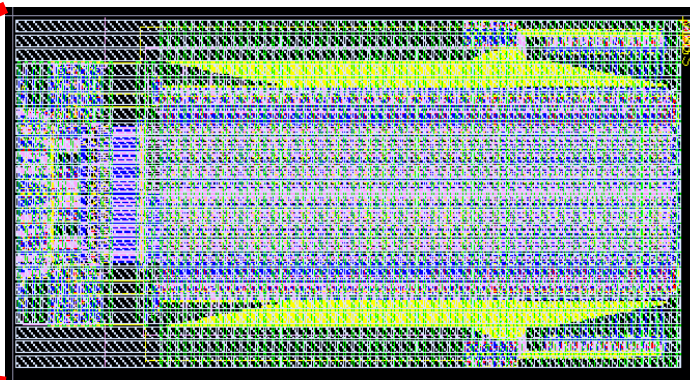
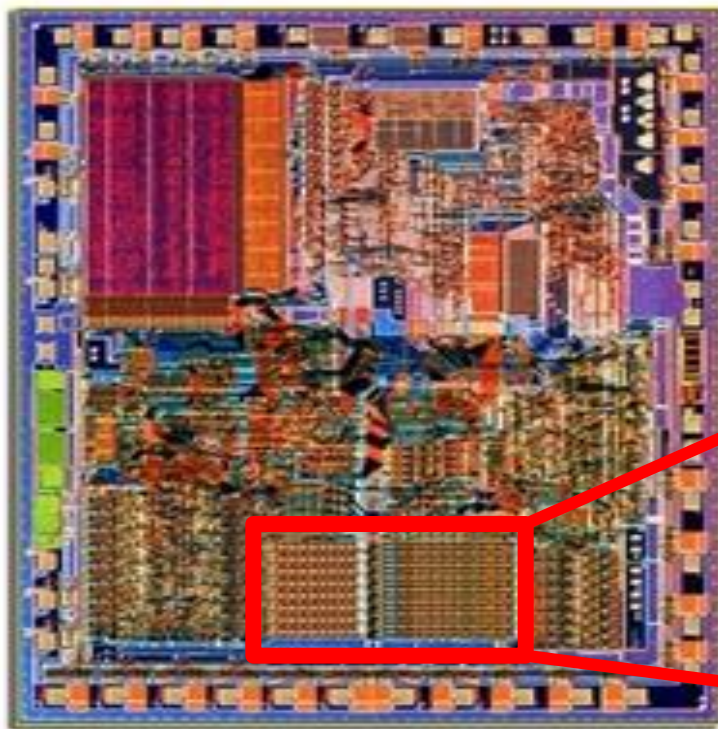
后端
设计

GDS 版图

制造/
封装/
测试

芯片

GDS: Geometry Data Standard



RTL文件

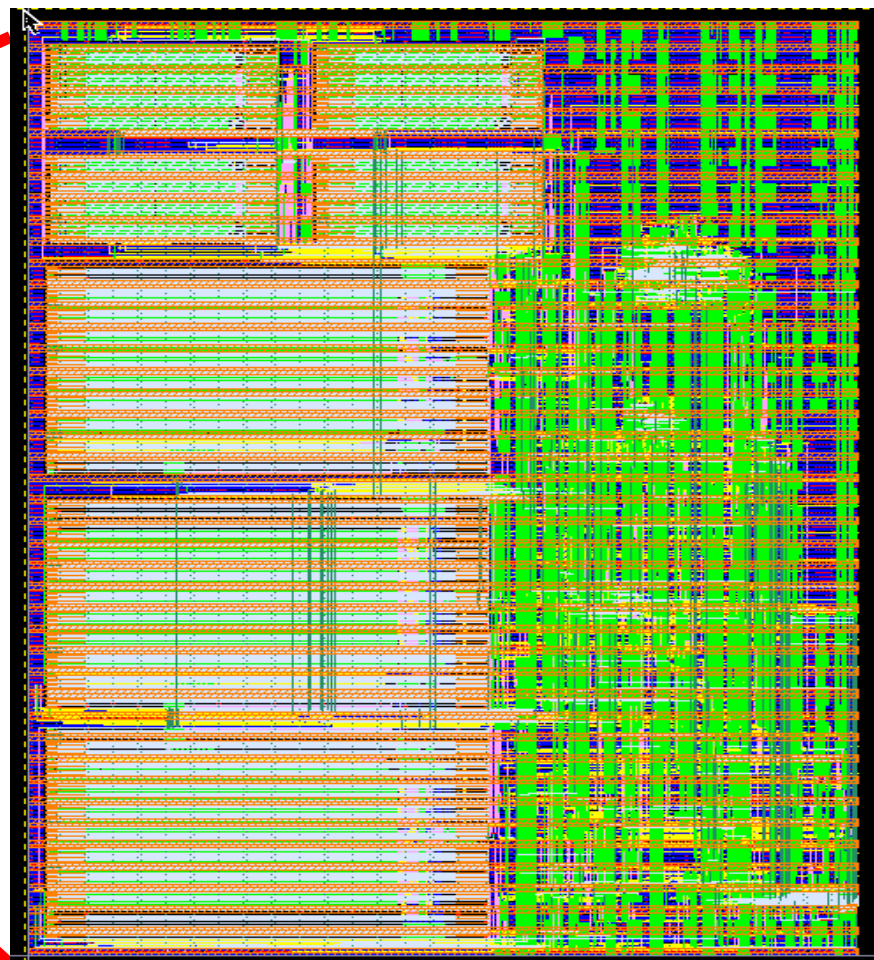
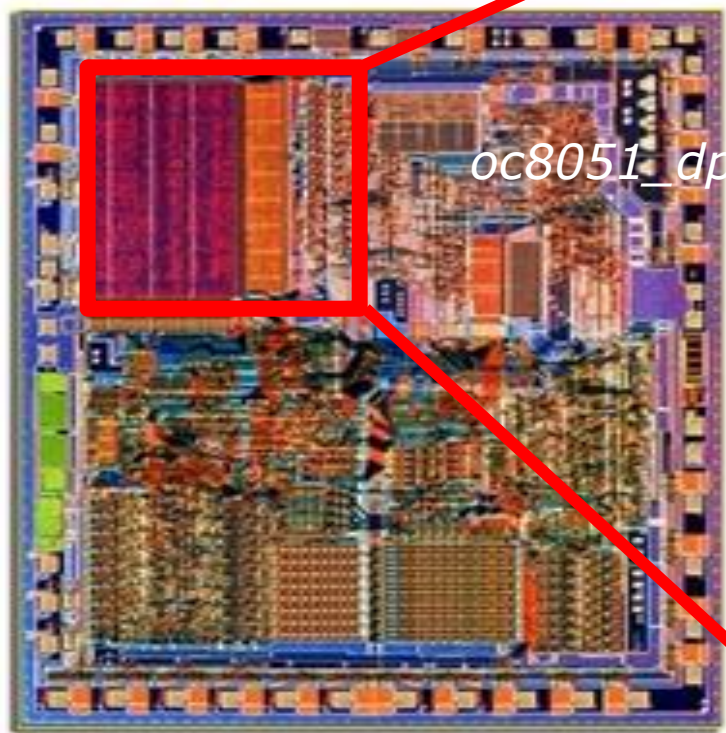
后端
设计

GDS 版图

制造/
封装/
测试

芯片

GDS: Geometry Data Standard



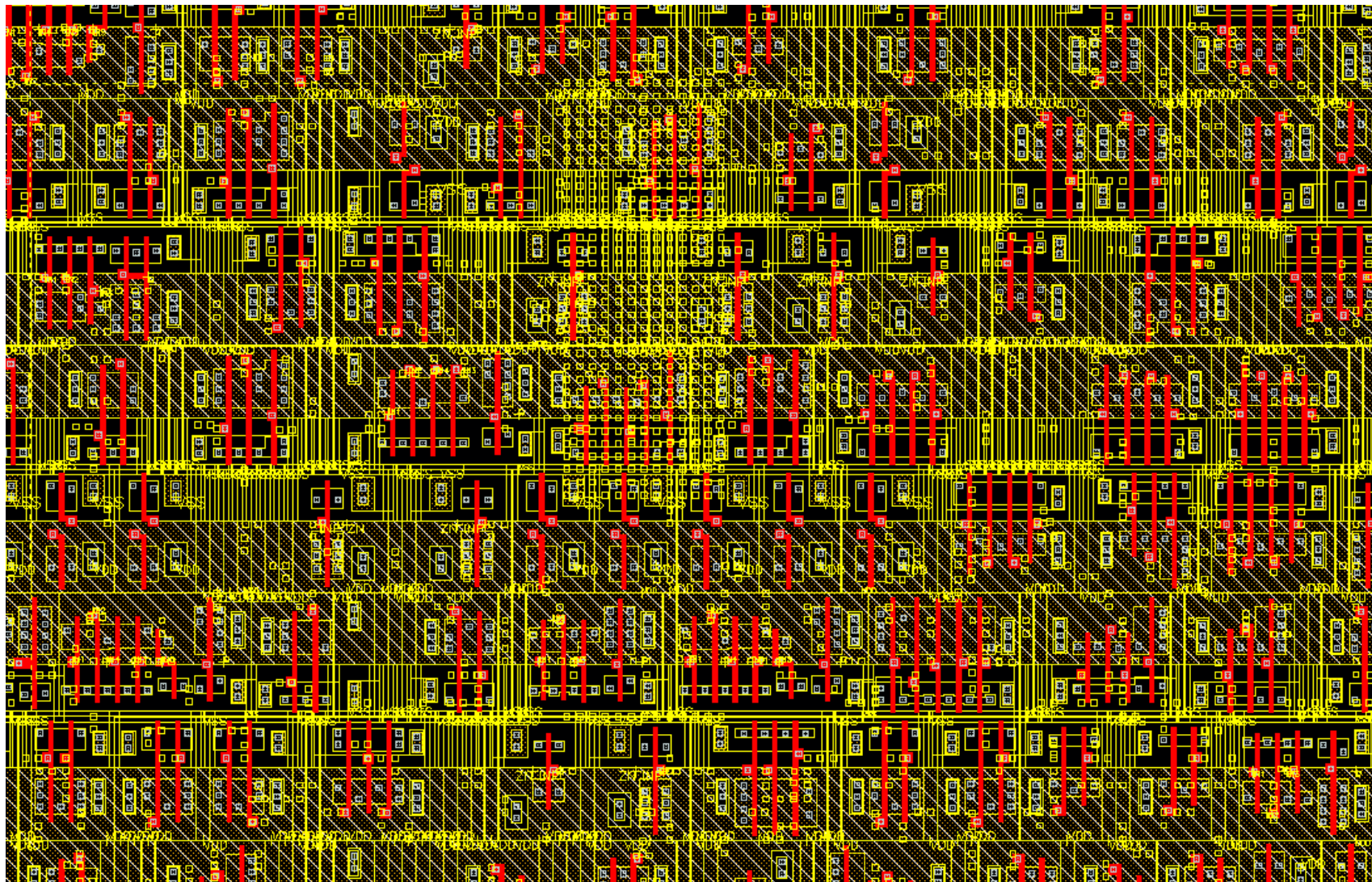
RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片





制造/
封装/
测试



爱芯人课堂/QQ群187291154

RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片



GDS: Geometry Data Standard

描述电路版图的一种格式：包括晶体管大小，数量，物理位置和尺寸信息，连接线的物理尺寸和位置信息等等；

晶体管+连接线，组成复杂庞大的电路逻辑；

为二进制格式，无法用文本编辑器查看，可用 calibre drv, virtuoso, laker等等查看；

可以理解为，GDS包含制造一颗芯片所需的全部信息；

芯片制造厂（fab）只需要IC设计公司提交GDS用于芯片生产；

Notes :

1. 版图 : Layout ;
2. 晶体管 : Transistor , 连接线 : interconnect/wire

RTL文件

后端
设计

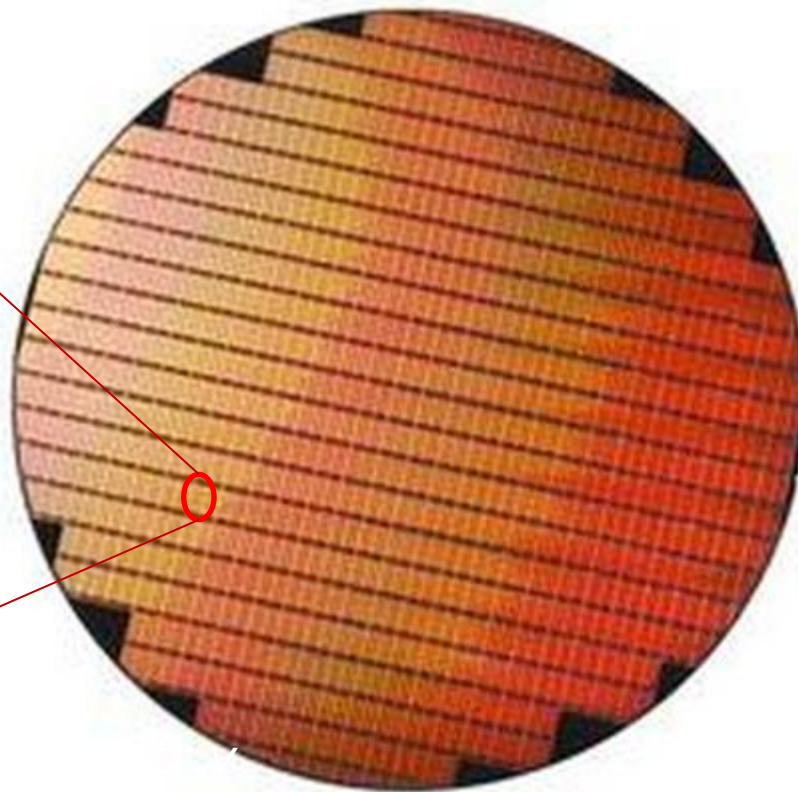
GDS 版图

制造
/封装
测试

芯片



封装前的芯片颗粒
(**Die**)



长满**Die**的晶圆
(**Wafer**)

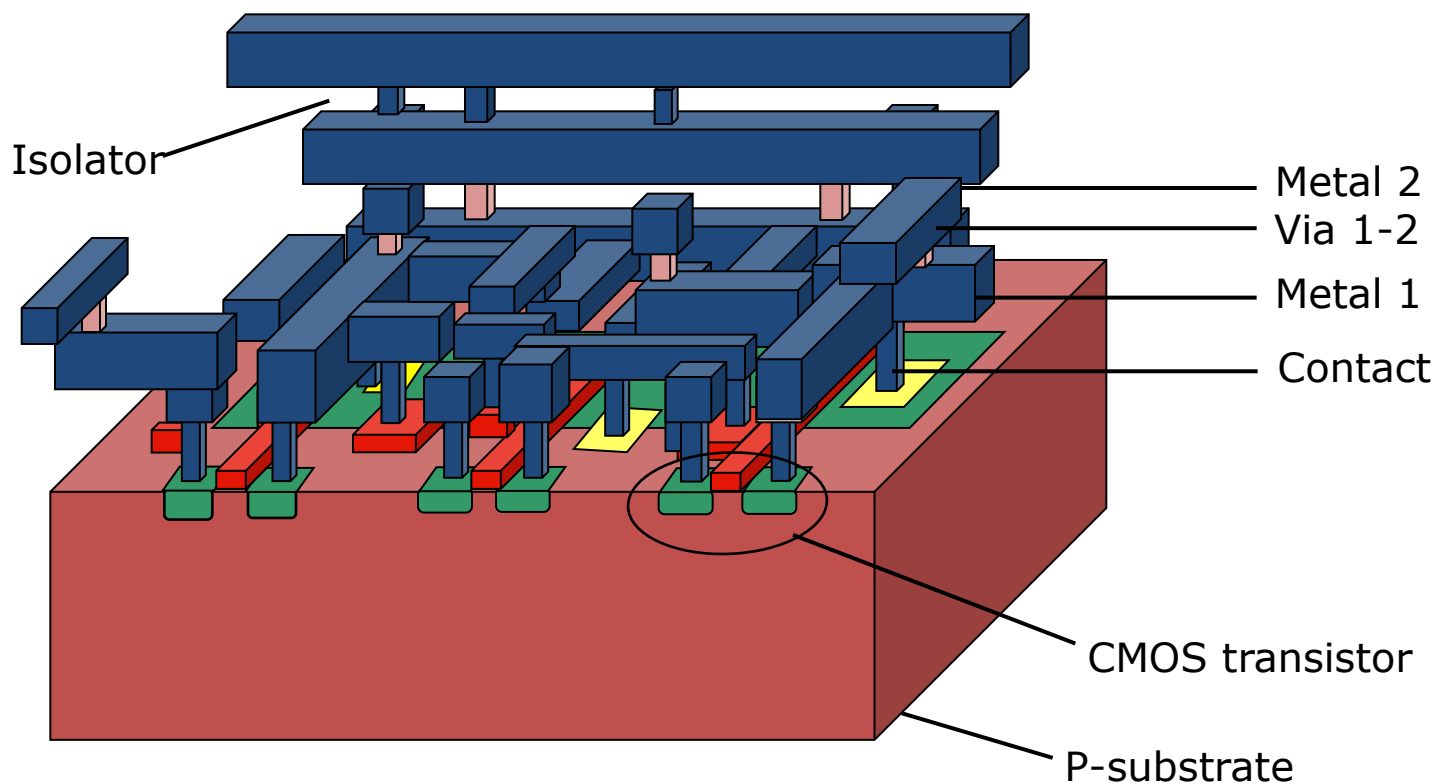
RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片



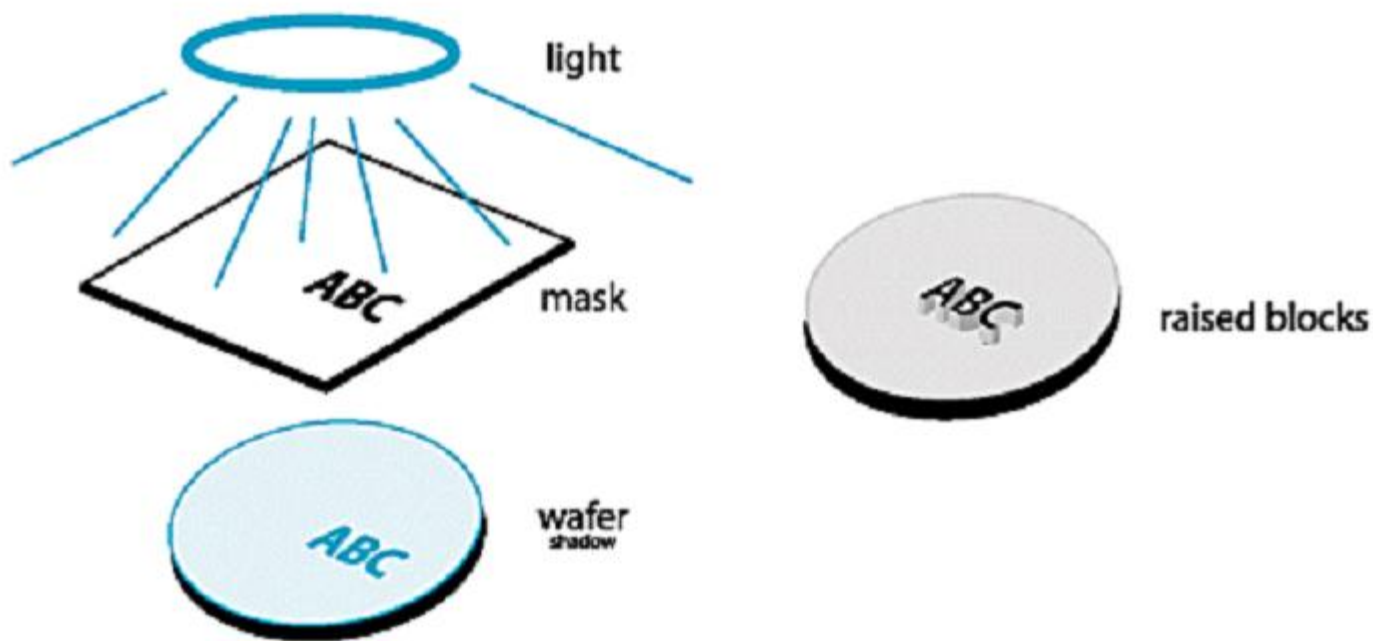
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



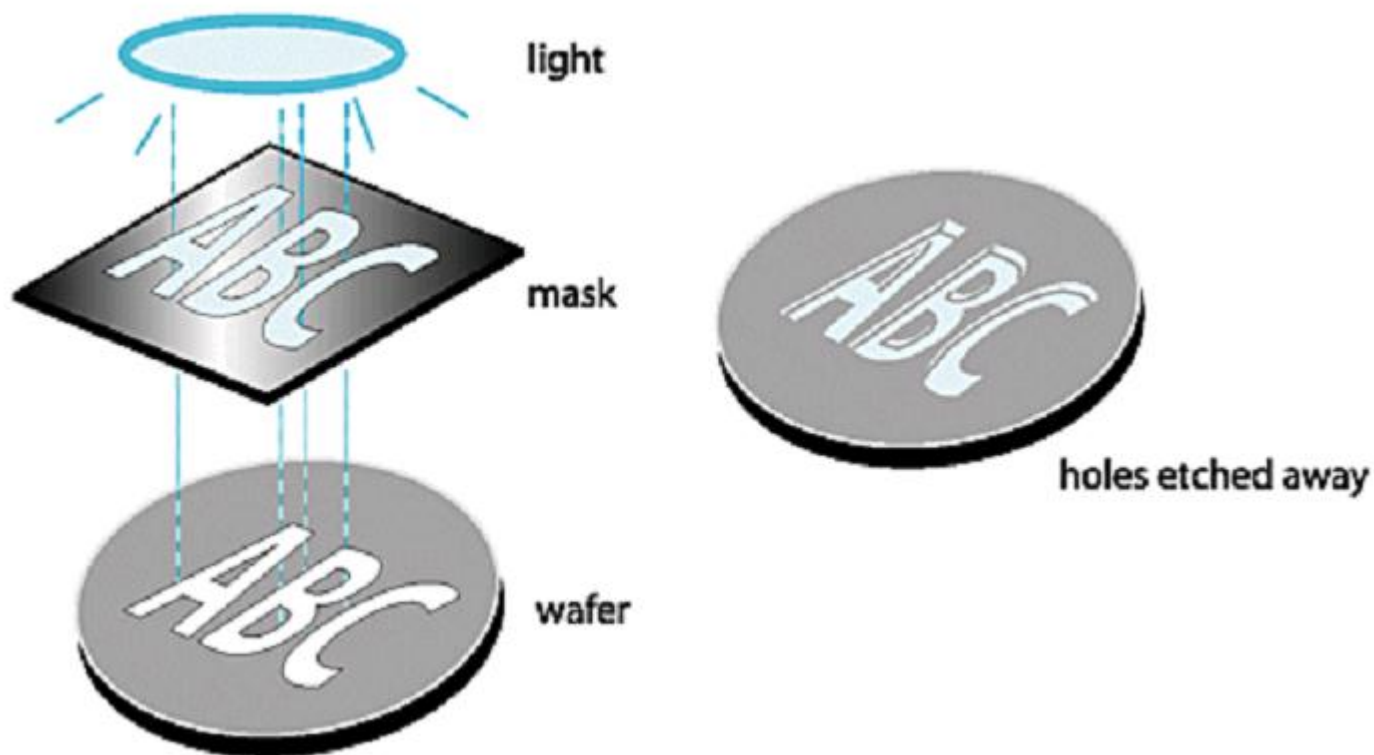
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



RTL文件

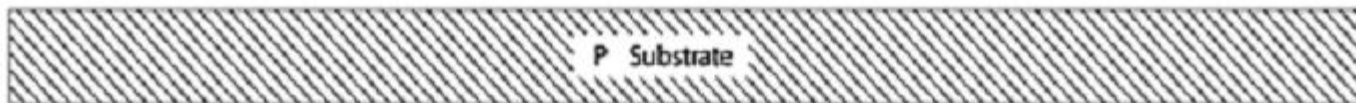
后端
设计

GDS 版图

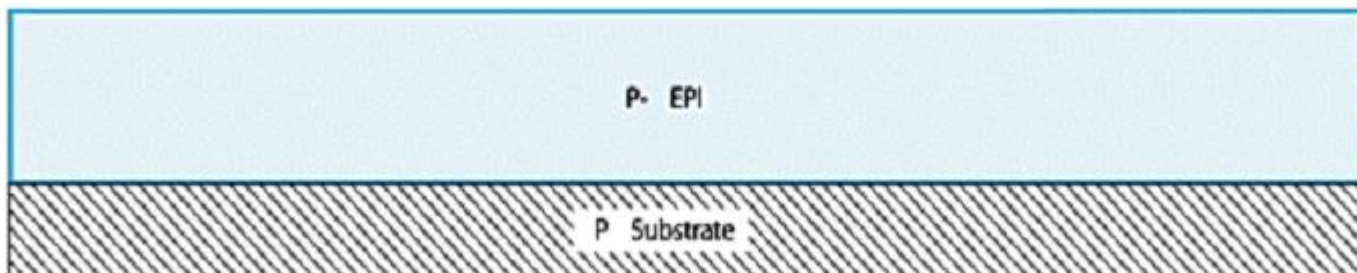
制造
/封装
测试

芯片

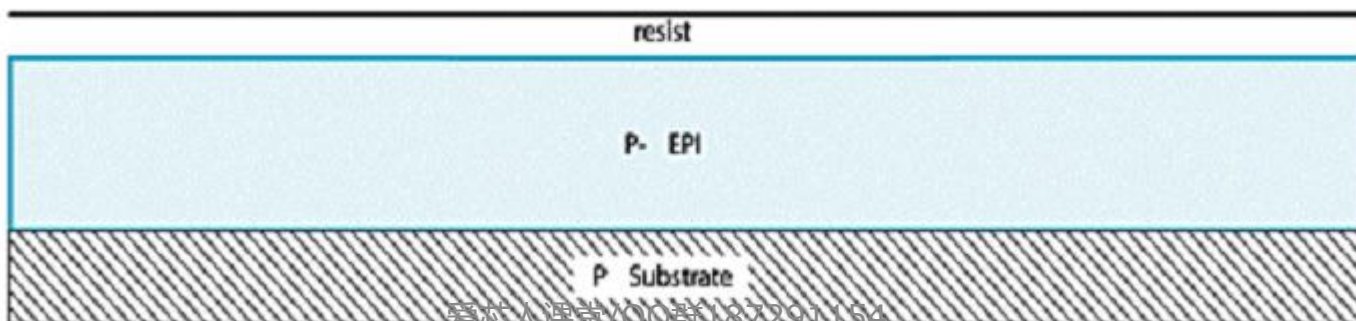
P Type Start Wafer



Grow P- Epitaxial Layer



Spin Resist Coating



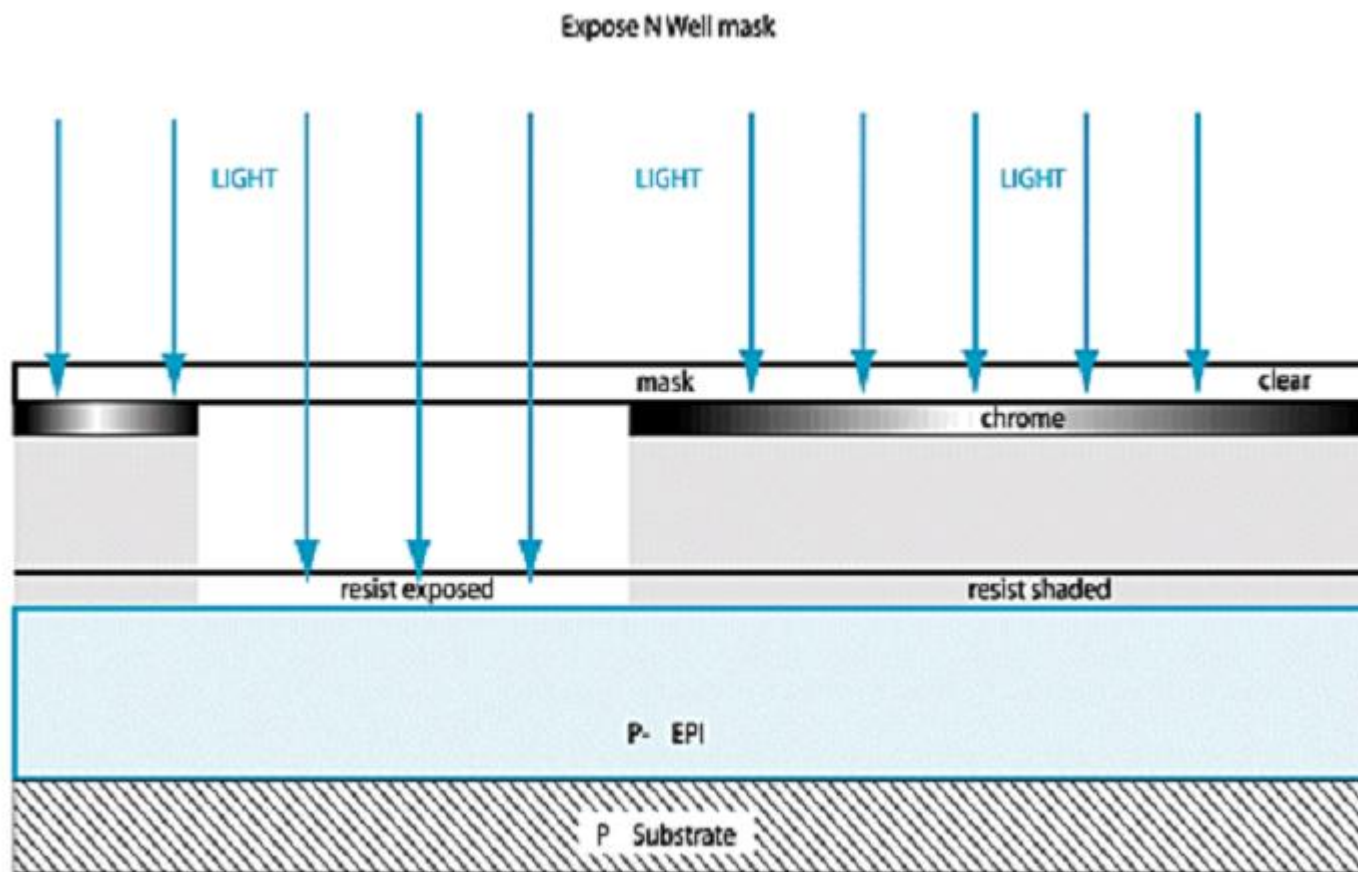
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



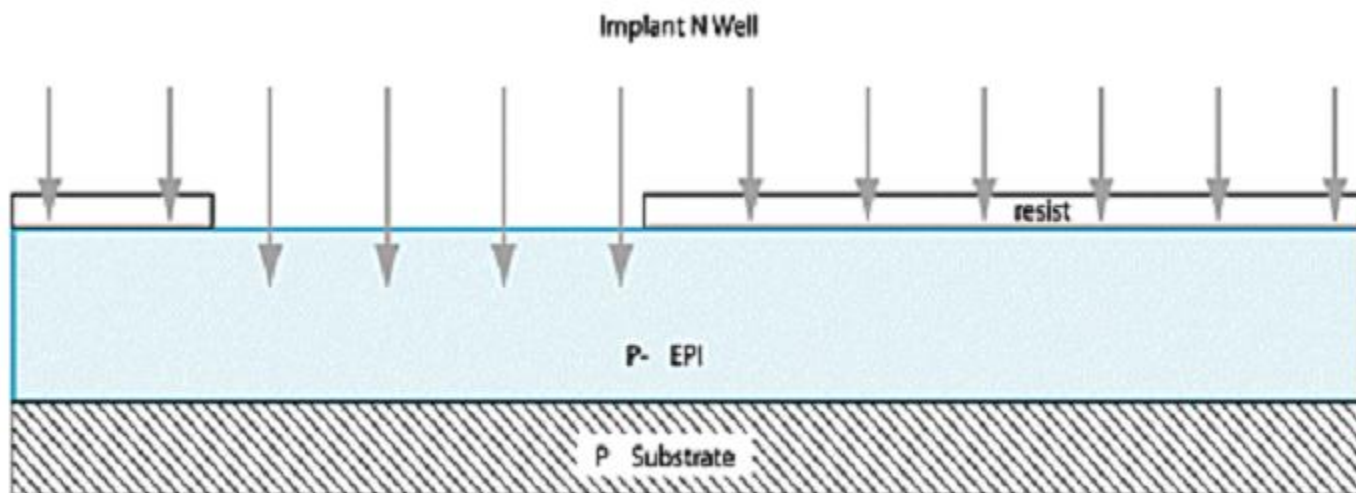
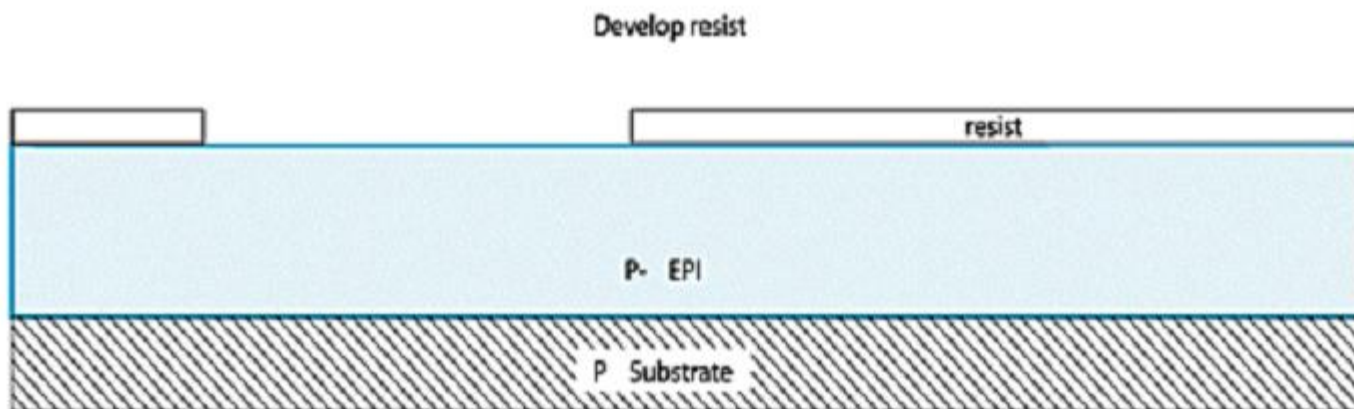
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



RTL文件

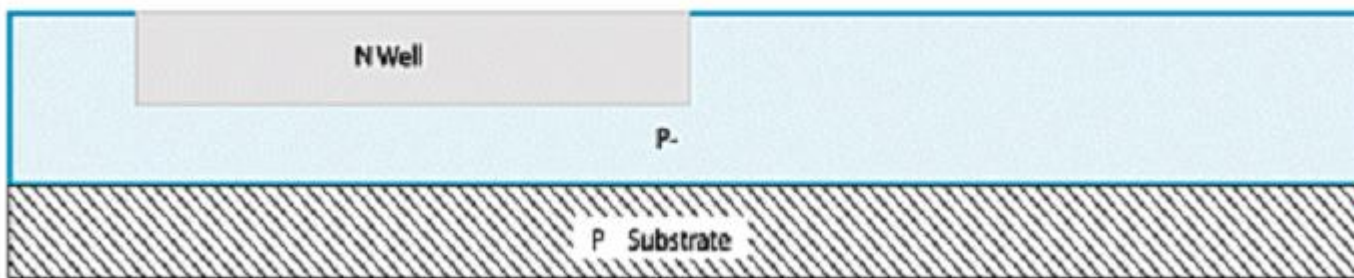
后端
设计

GDS 版图

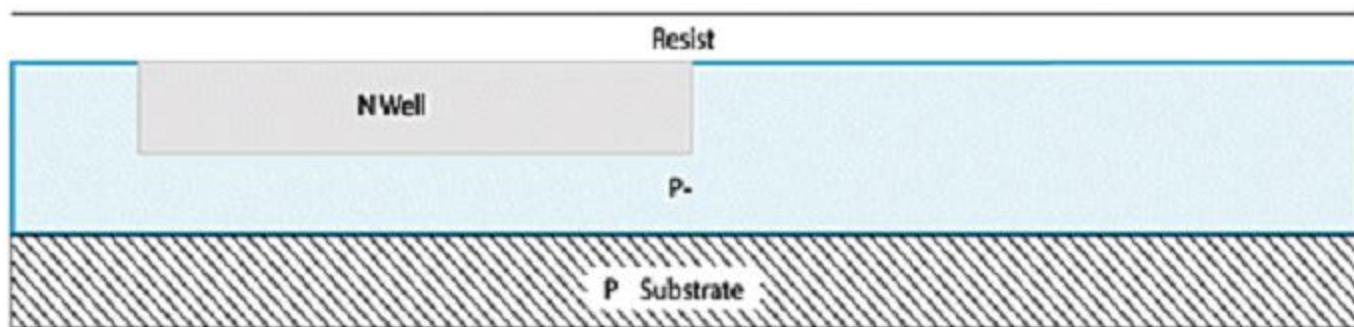
制造
/封装
测试

芯片

Remove oxide from Anneal



Spin resist



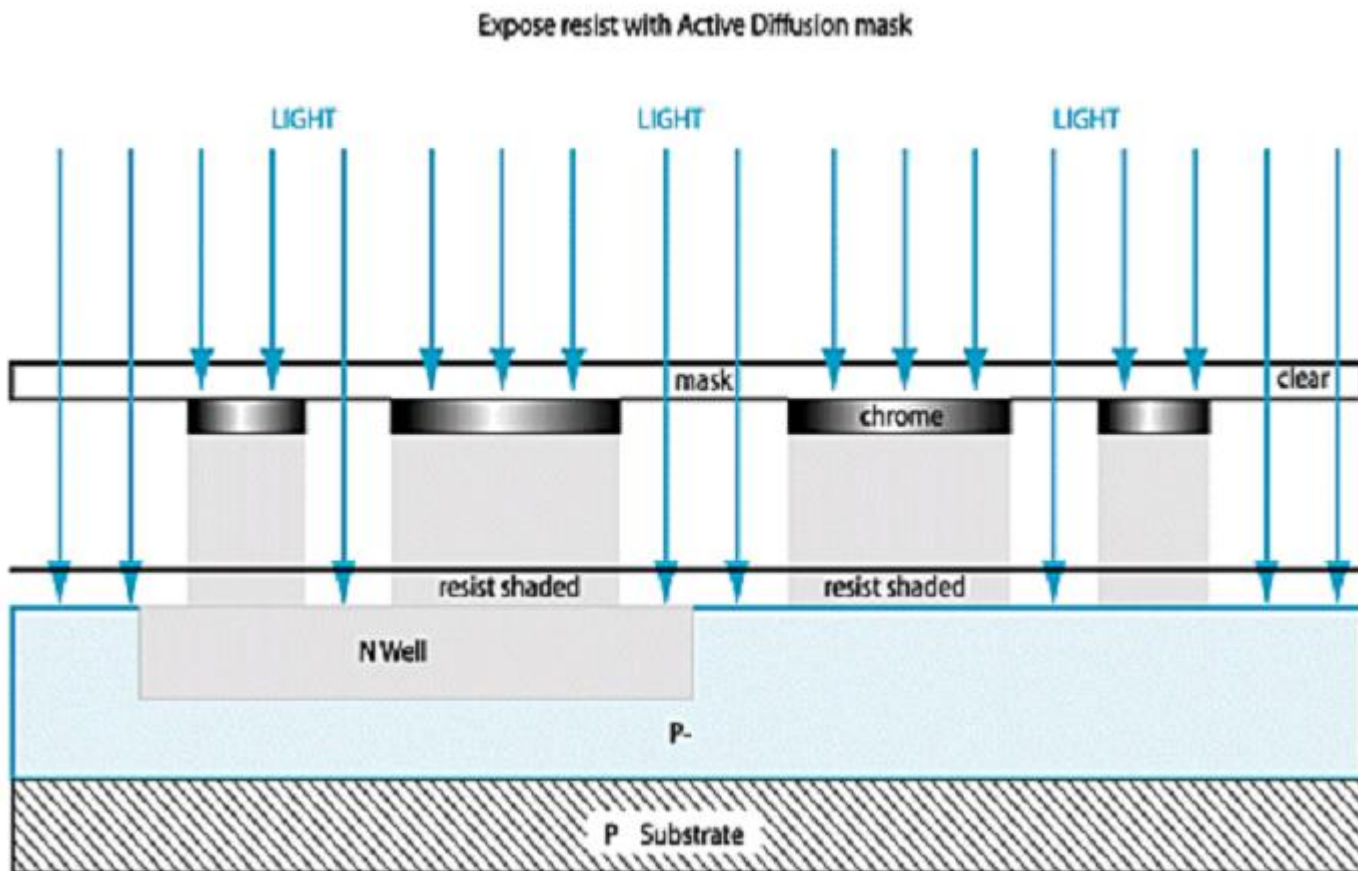
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



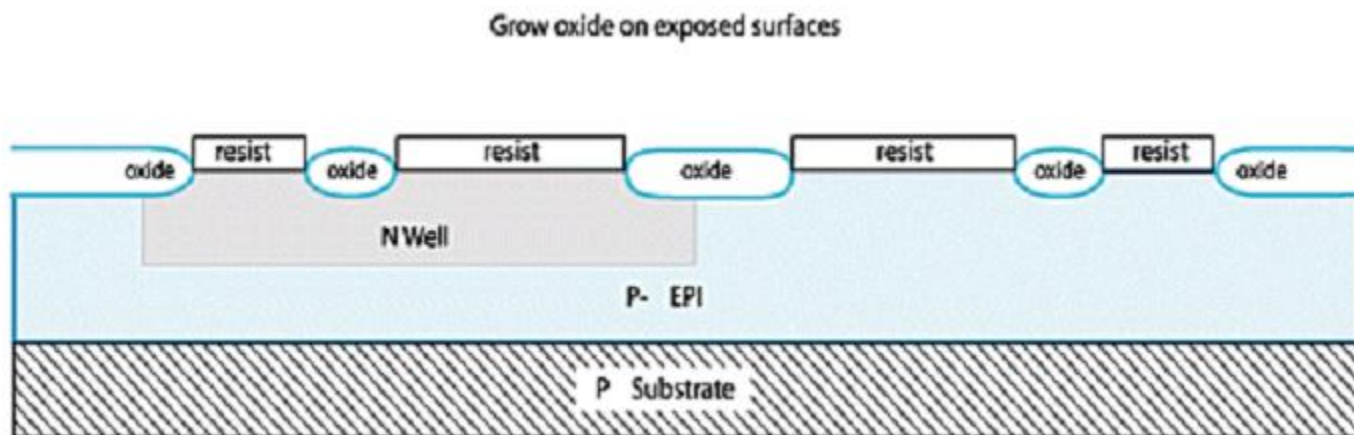
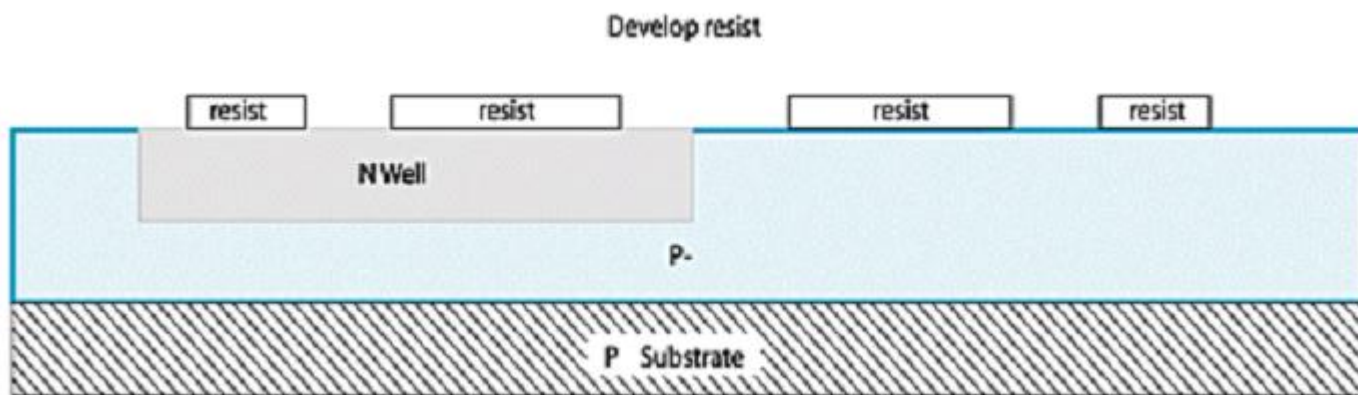
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



RTL文件

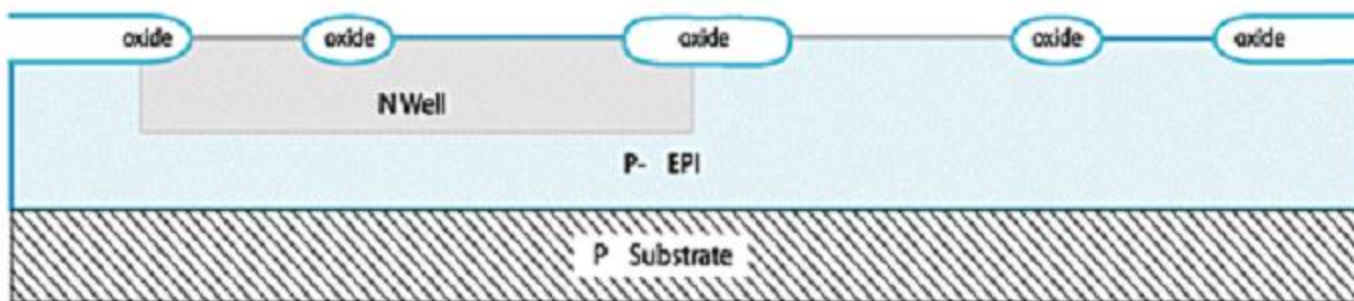
后端
设计

GDS 版图

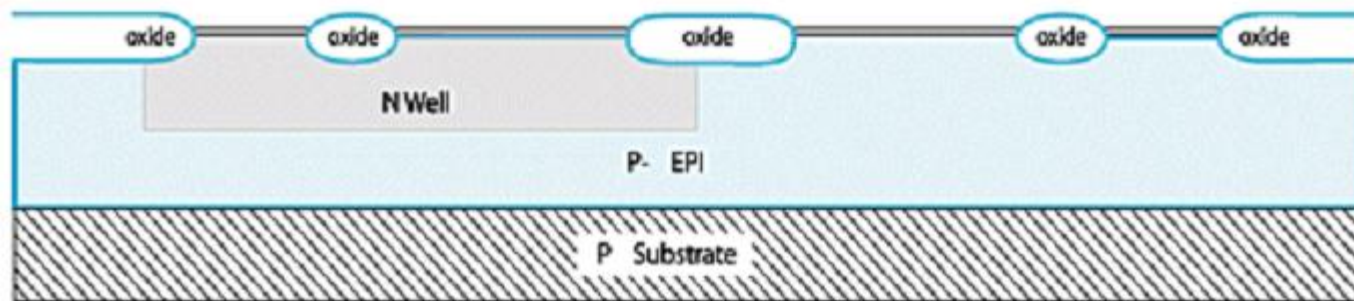
制造
/封装
测试

芯片

Remove resist



Grow thin oxide over silicon surfaces



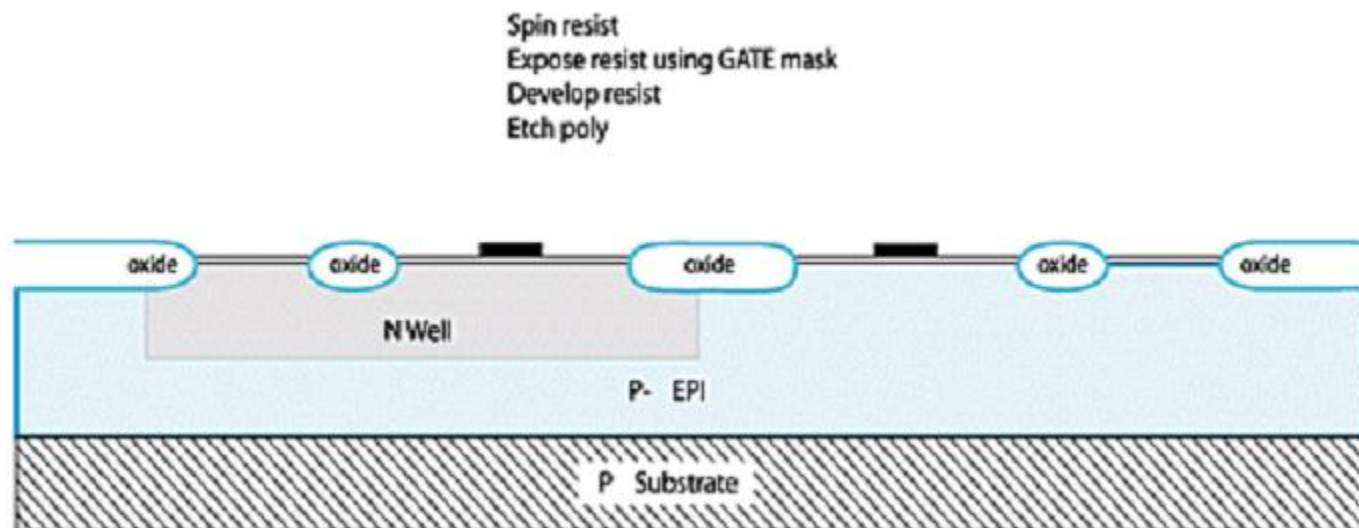
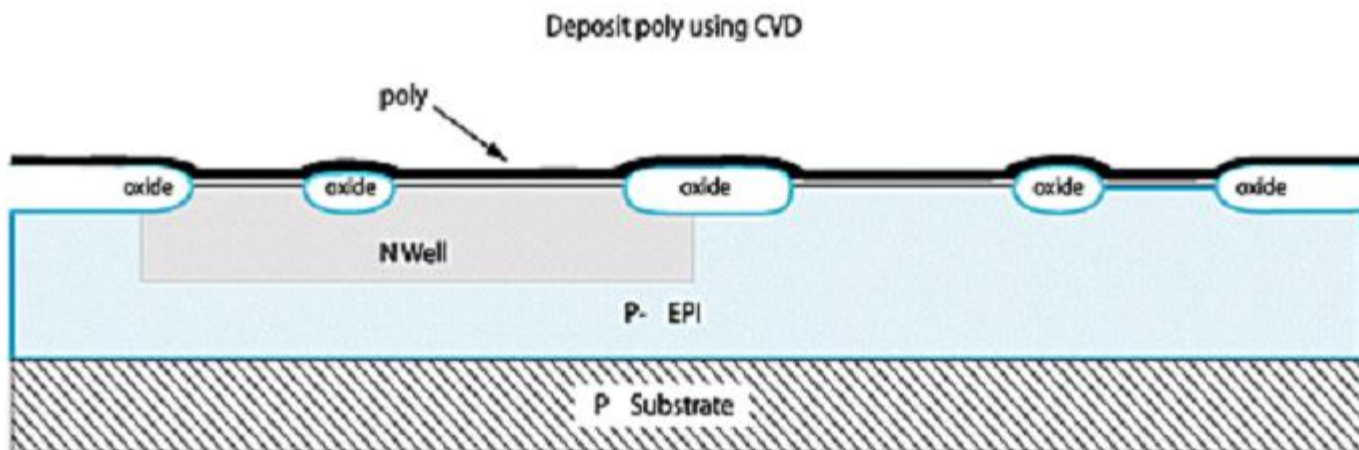
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



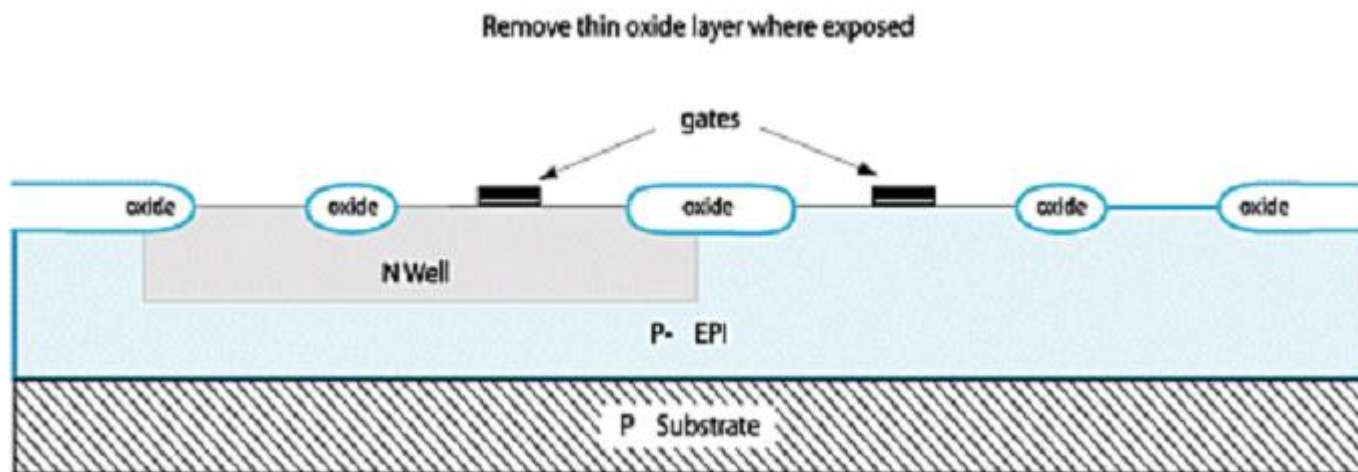
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



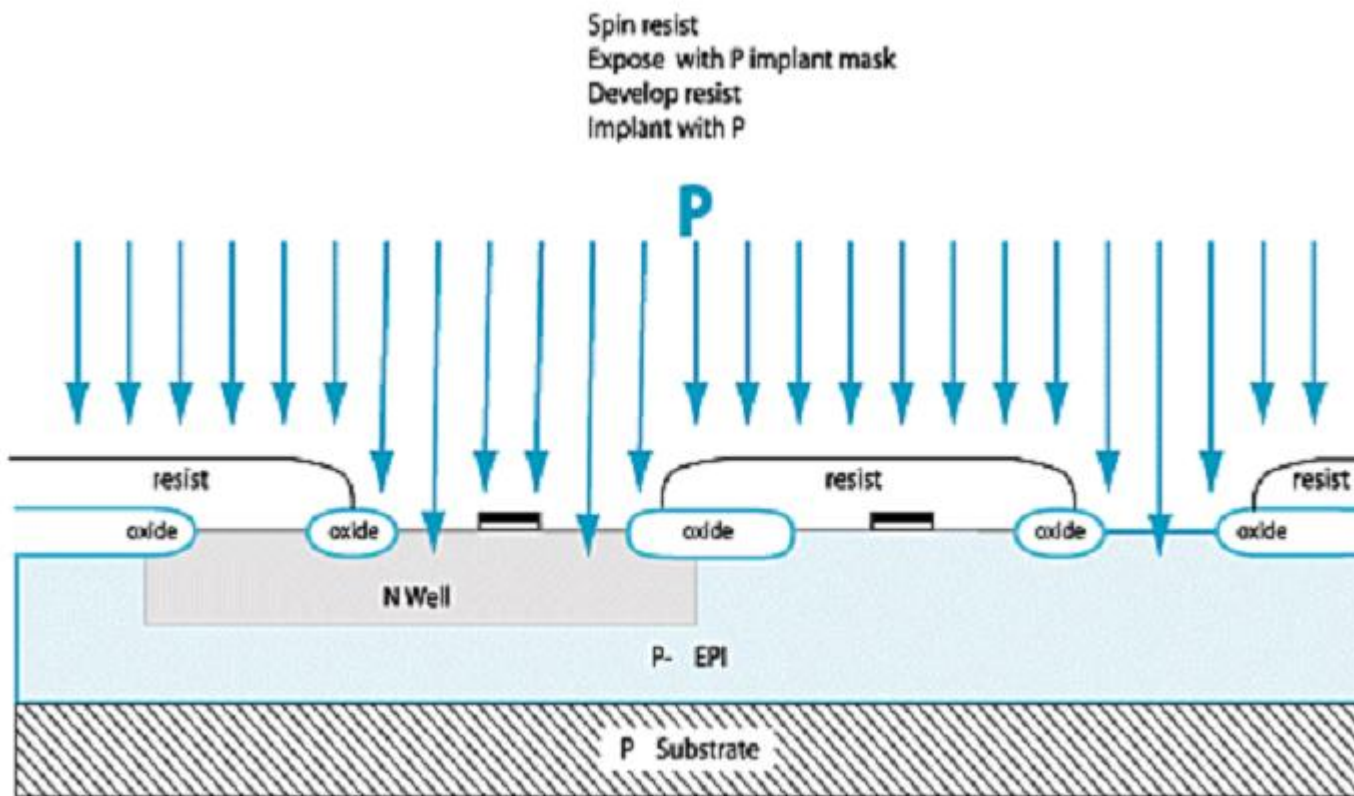
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



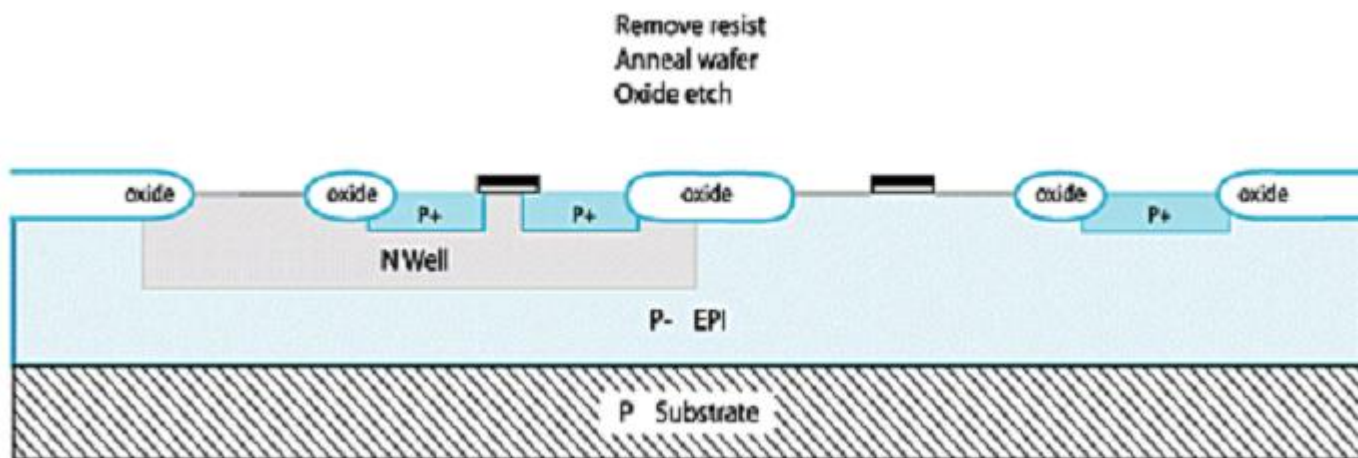
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



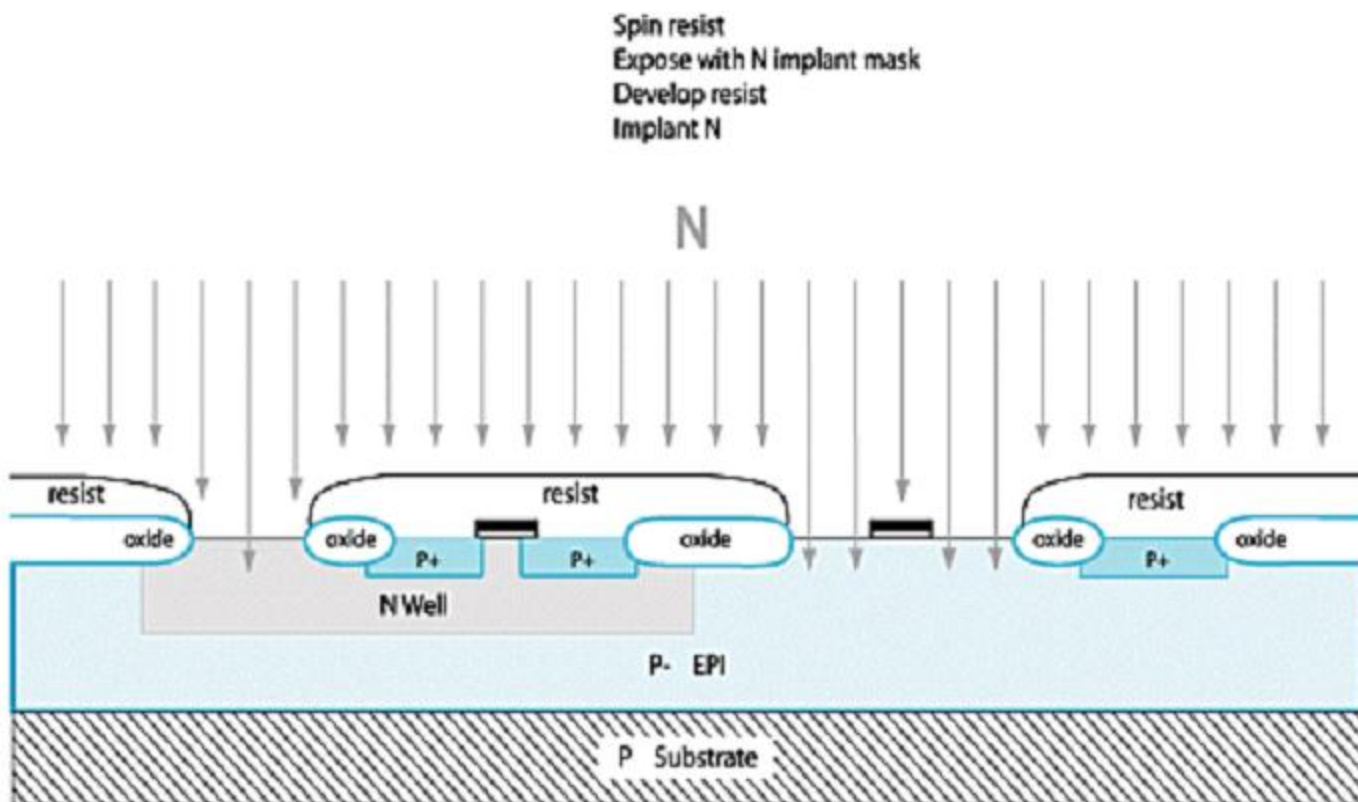
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



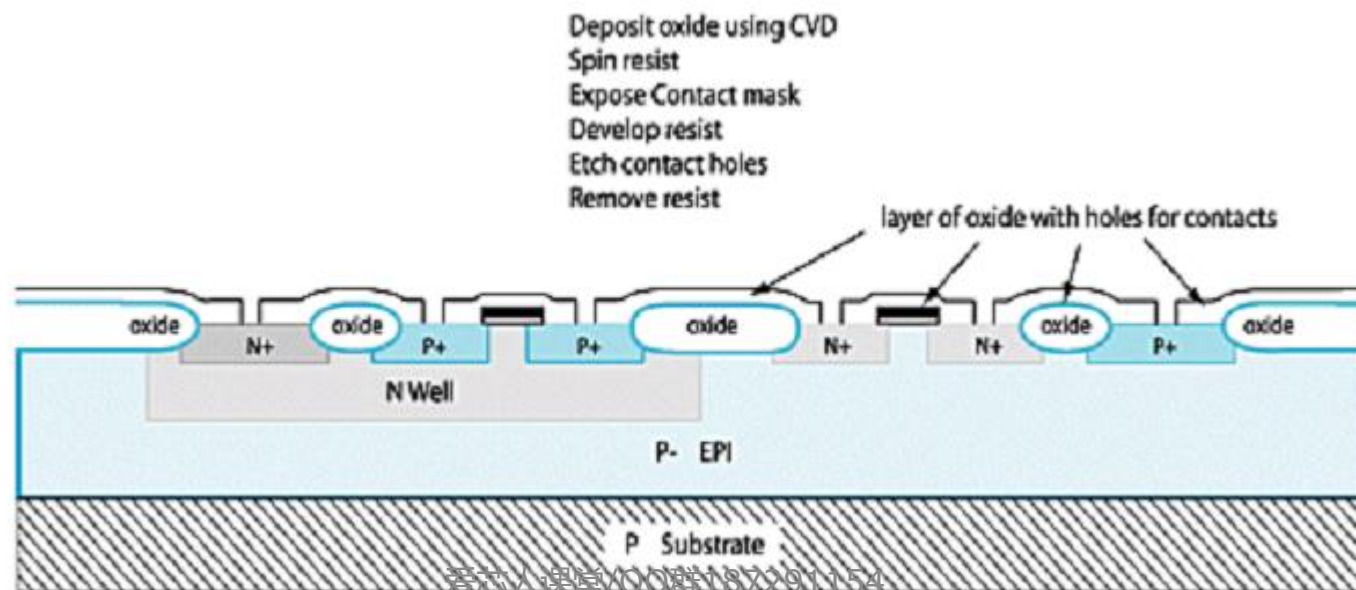
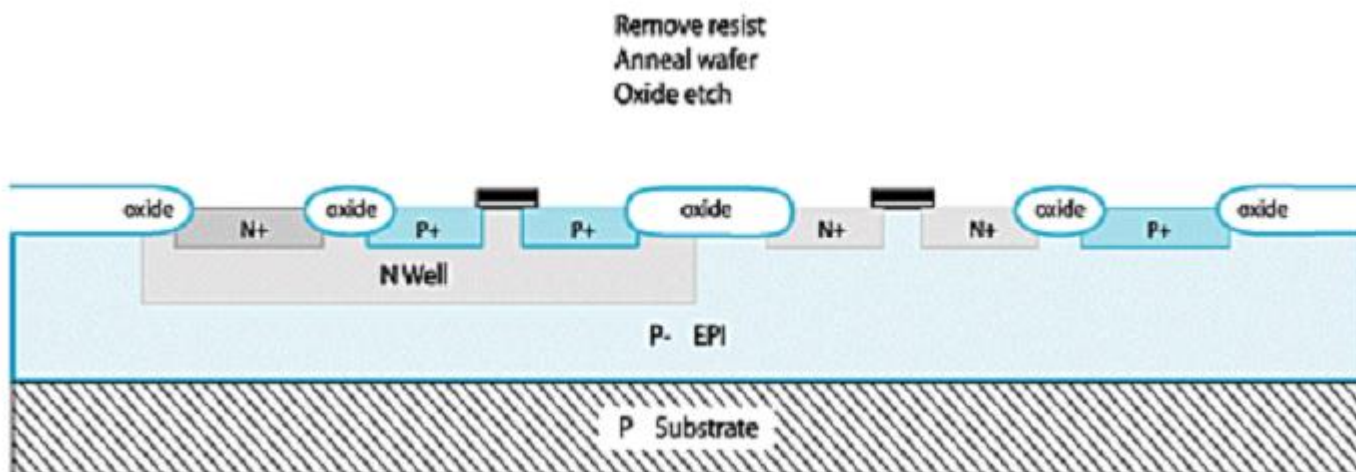
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



RTL文件

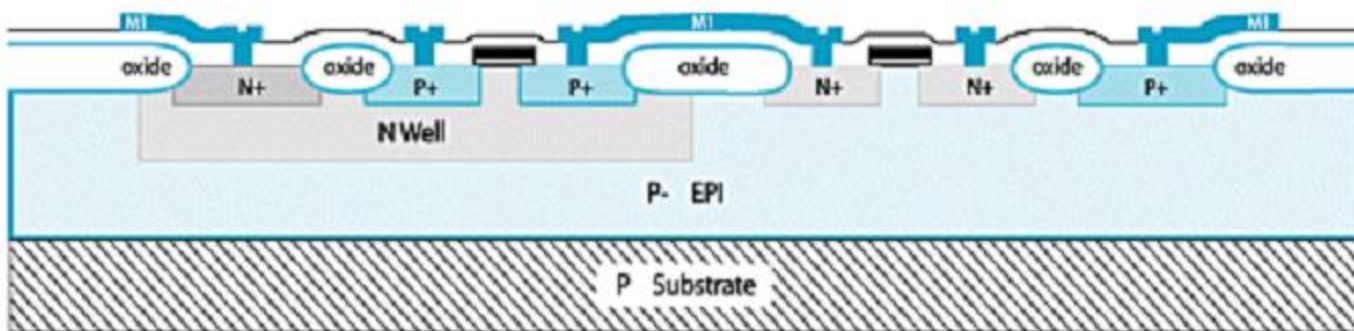
后端
设计

GDS 版图

制造
/封装
测试

芯片

Deposit Metal 1 using sputtering
Spin resist
Expose Metal 1 mask
Etch Metal using RIE
Remove resist



RTL文件

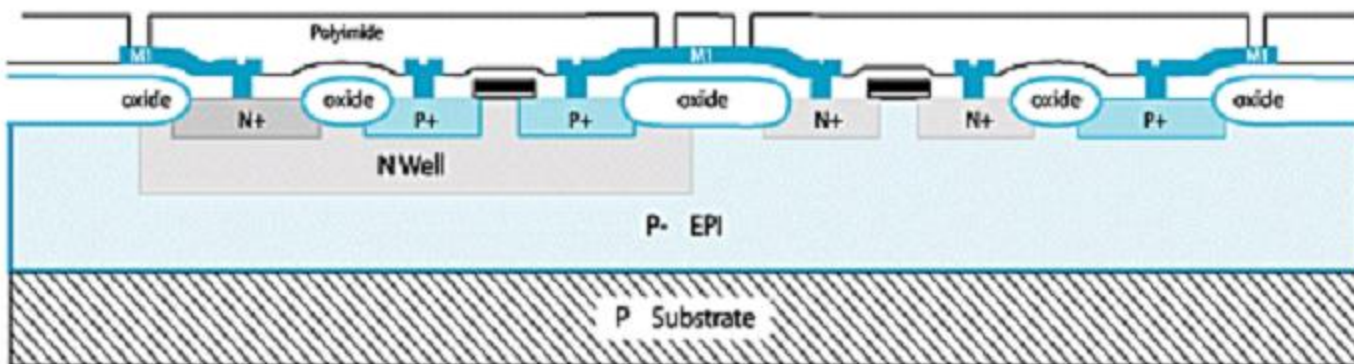
后端
设计

GDS 版图

制造
/封装
测试

芯片

Spin Polyimide
Spin resist
Expose Via 1 mask (vias connect two metals)
Etch Via
Remove resist



RTL文件

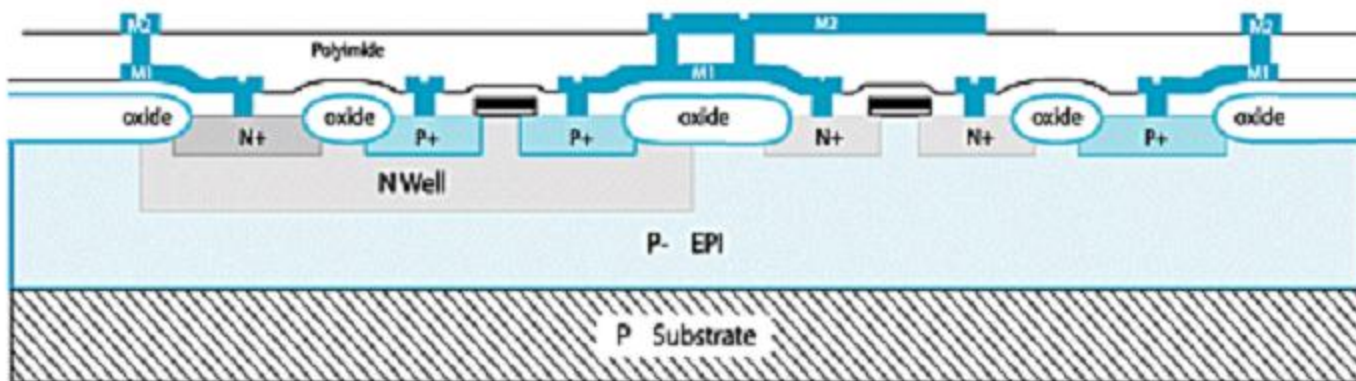
后端
设计

GDS 版图

制造
/封装
测试

芯片

Deposit Metal 2 using sputtering
Spin resist
Expose Metal 2 mask
Etch Metal using RIE
Remove resist



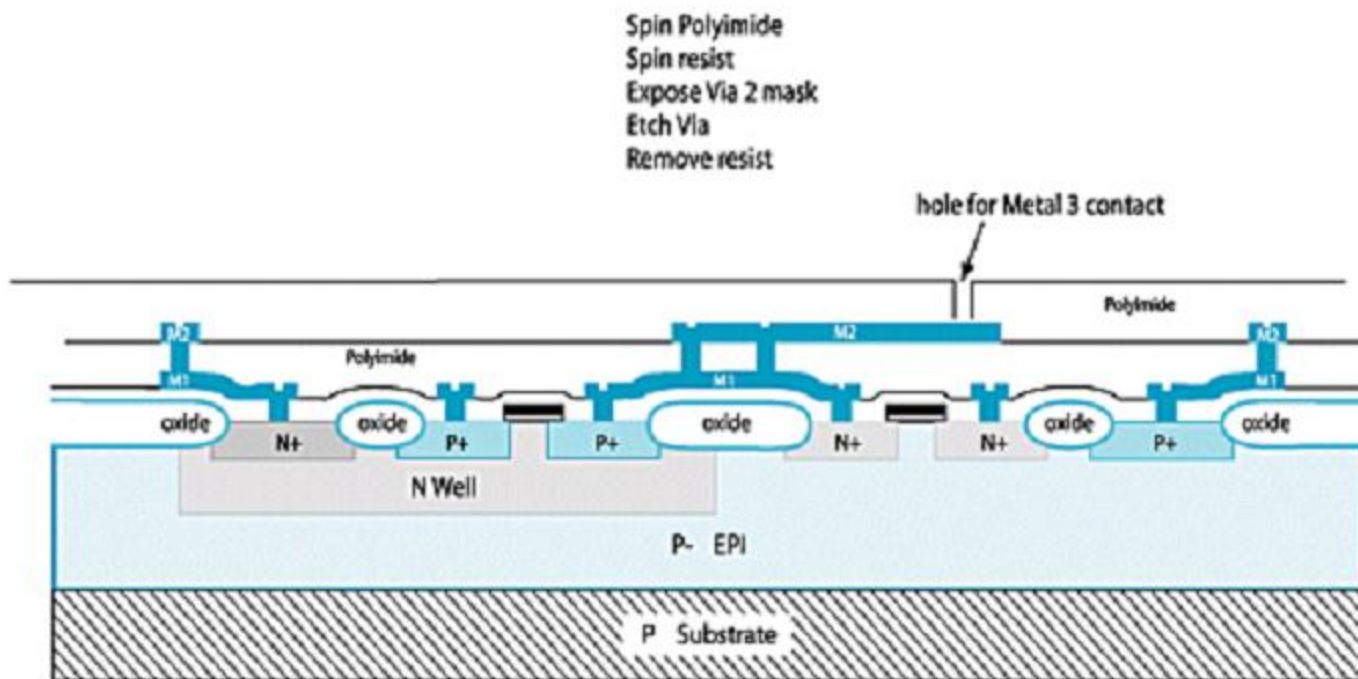
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



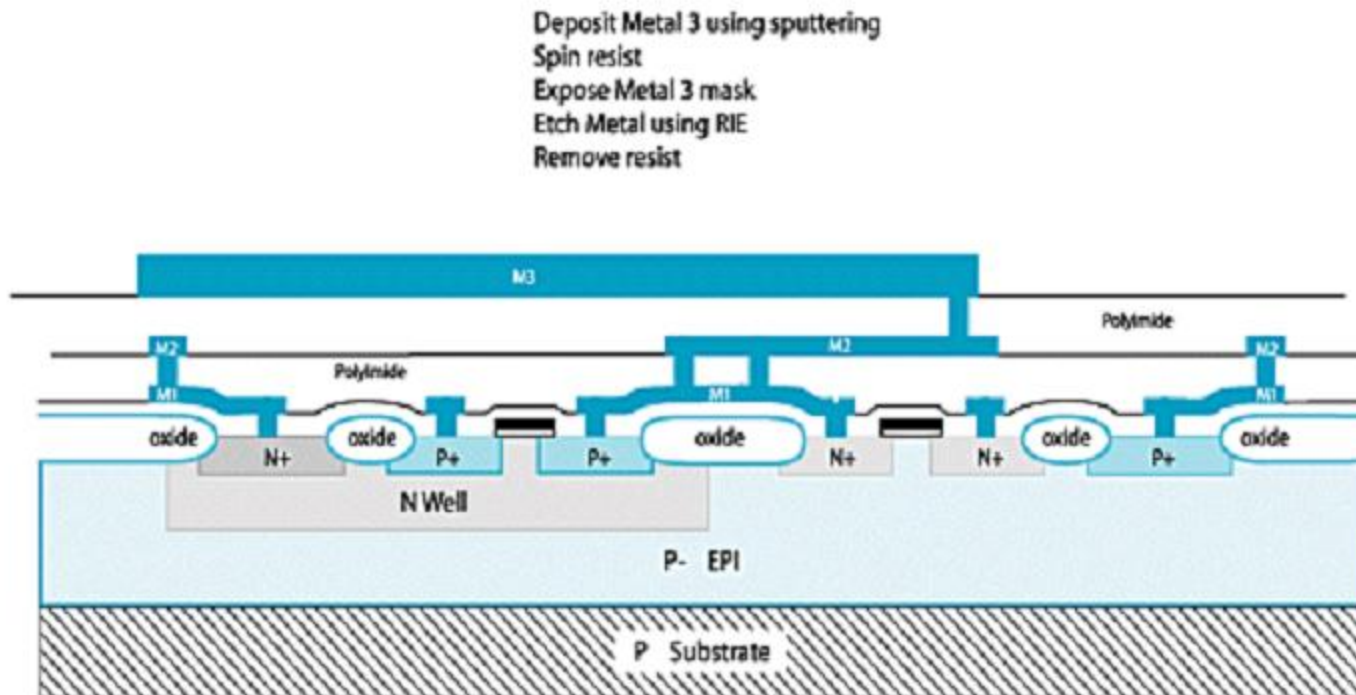
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



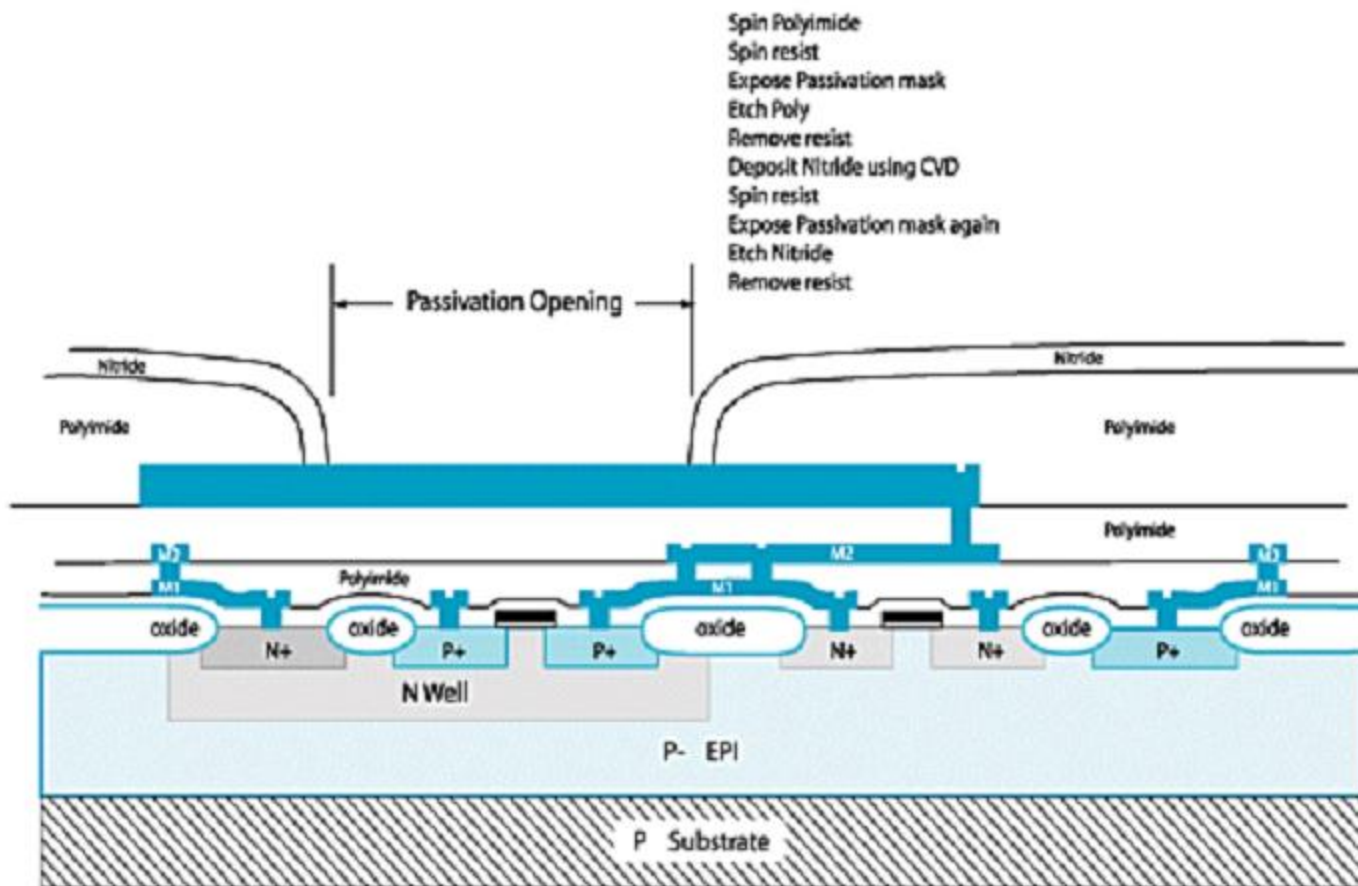
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



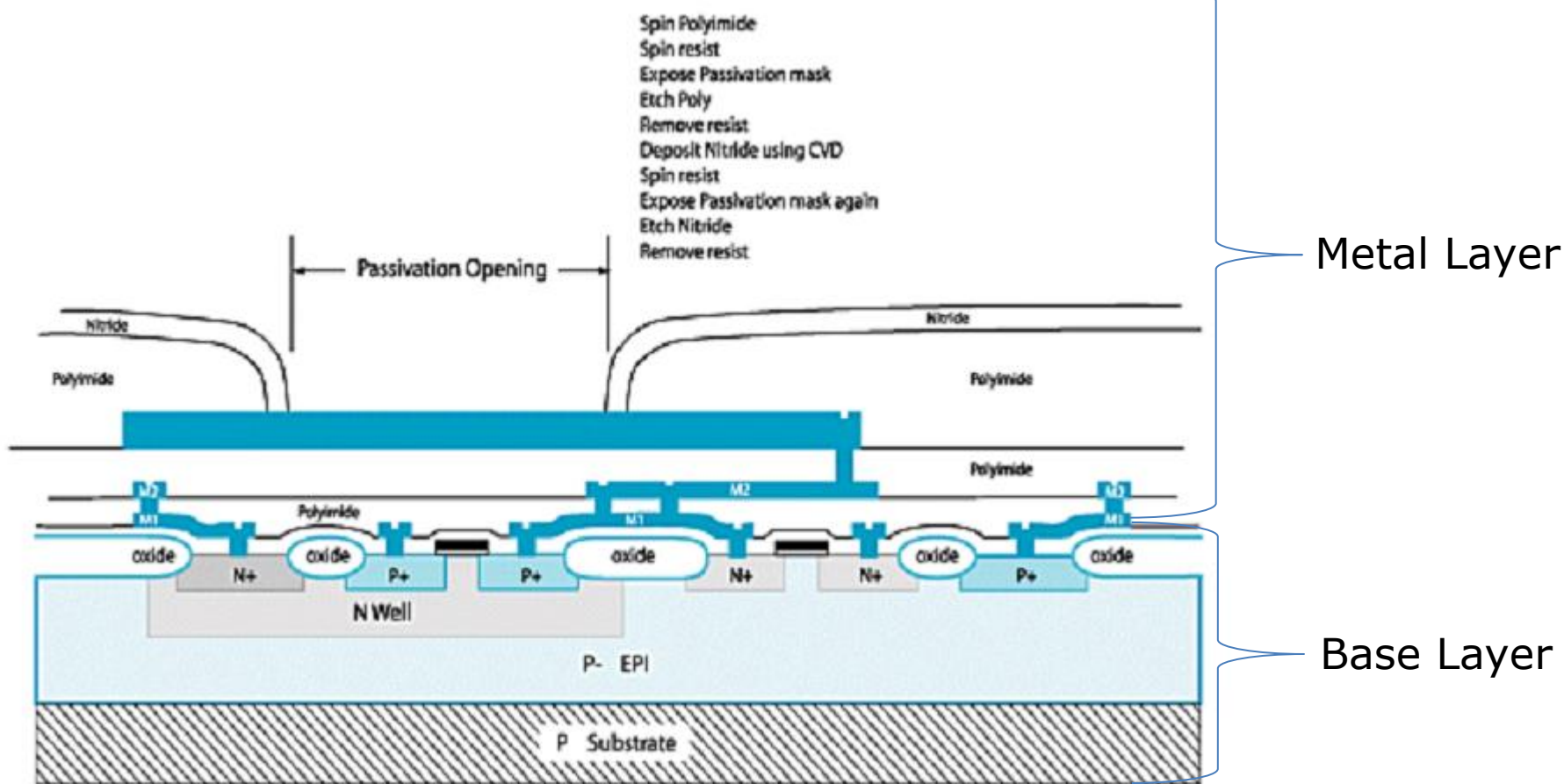
RTL文件

后端
设计

GDS 版图

制造
/封装
测试

芯片



Notes :

1. $\$ \propto$ (Mask 数&Die的面积)

RTL文件

后端
设计

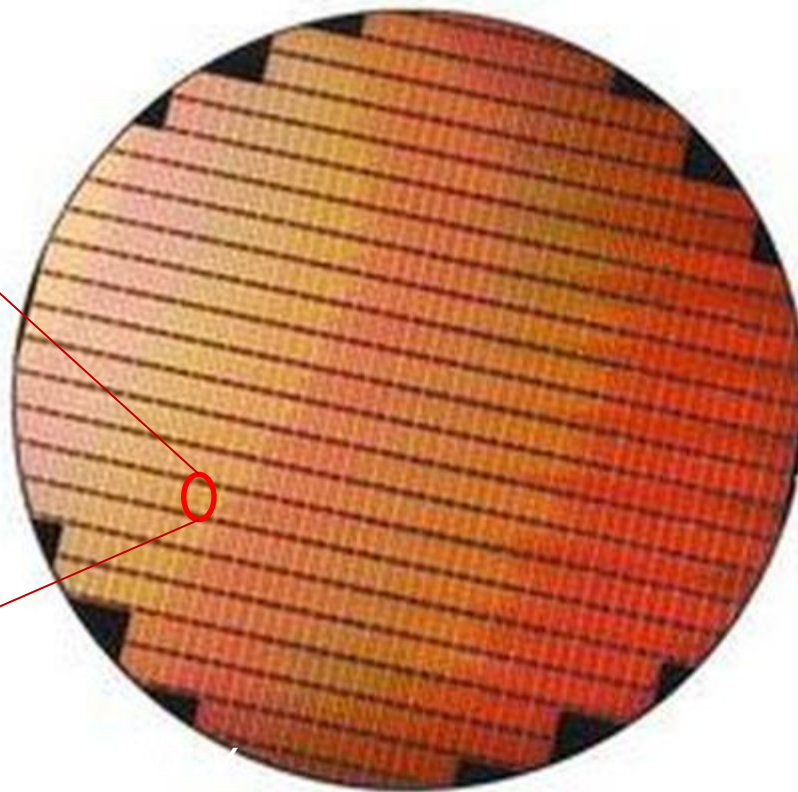
GDS 版图

制造
/封装
测试

芯片



封装前的芯片颗粒
(**Die**)



长满**Die**的晶圆
(**Wafer**)

RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片



Die



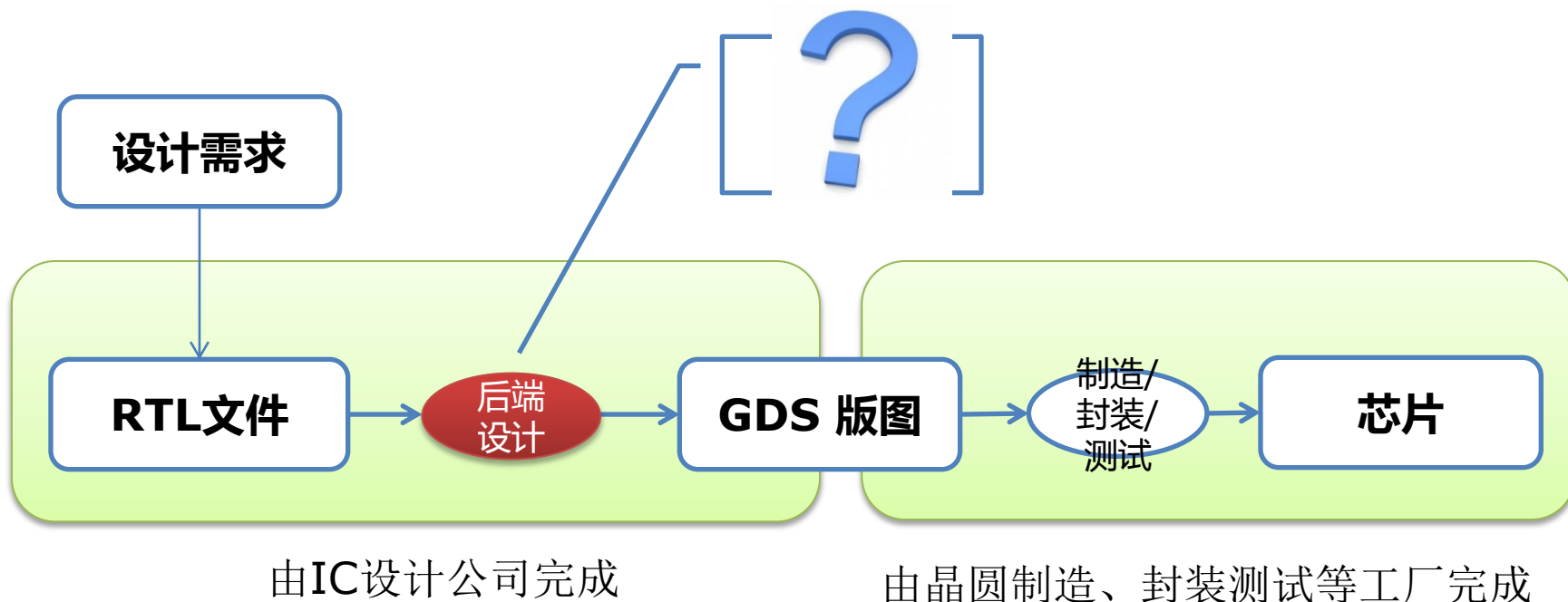
.....

封装&测试



芯片

从需求到芯片



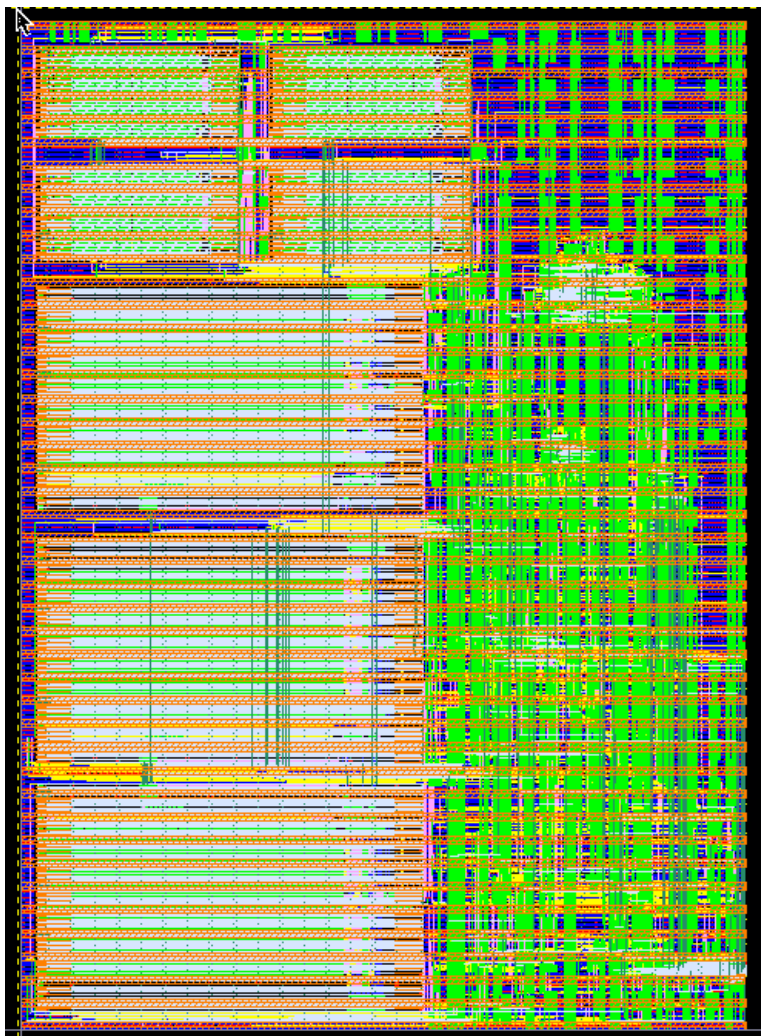
RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片



以CPU为例

RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片

```
module oc8051_dptra(clk, rst, addr, data_in, data2_in, wr, wd2, wr_bit, data_hi, data_lo);  
  
input clk, rst, wr, wr_bit;  
input [2:0] wd2;  
input [7:0] addr, data_in, data2_in;  
  
output [7:0] data_hi, data_lo;  
  
reg [7:0] data_hi, data_lo;  
  
always @(posedge clk or posedge rst)  
begin  
    if (rst) begin  
        data_hi <= #1 `OC8051_RST_DPH;  
        data_lo <= #1 `OC8051_RST_DPL;  
    end else if (wd2==`OC8051_RWS_DPTRA) begin  
        //  
        //write from destination 2 and 1  
        data_hi <= #1 data2_in;  
        data_lo <= #1 data_in;  
    end else if ((addr==`OC8051_SFR_DPTRA_HI) & (wr) & !(wr_bit))  
        //  
        //case of writing to dptra  
        data_hi <= #1 data_in;  
    else if ((addr==`OC8051_SFR_DPTRA_LO) & (wr) & !(wr_bit))  
        data_lo <= #1 data_in;  
    end  
end  
  
endmodule
```

oc8051_dptra.v

RTL: Register Transfer Level

采用硬件描述语言（**HDL**）描述的寄存器传输级电路；

由设计人员（**designer**）完成，传递给数字后端的设计数据；

功能通过仿真验证；

Notes :

1. 在IC设计流程中提到RTL，指的就是这些Verilog/VHDL设计文件；
2. HDL:包括Verilog HDL，VHDL等；



从哪里出发？

RTL

+

SDC

RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片

再看版图

对这个**GDS**有什么要求呢？

1. 功能，要和**RTL**的一样；
2. 性能，要满足预定的性能目标；
3. 物理规则，**Fab**能拿它正常制造；
4. 功耗&面积， **IR Drop**满足要求，
功耗&面积要尽可能小；



我们的目标是什么？

GDS

对这个**GDS**有什么要求呢？

1. 功能，要和**RTL**的一样；
2. 性能，要满足预定的性能目标；
3. 物理规则，**Fab**能拿它正常制造；
4. 功耗&面积，**IR Drop**满足要求，功耗&面积要尽可能小；

数字后端设计流程

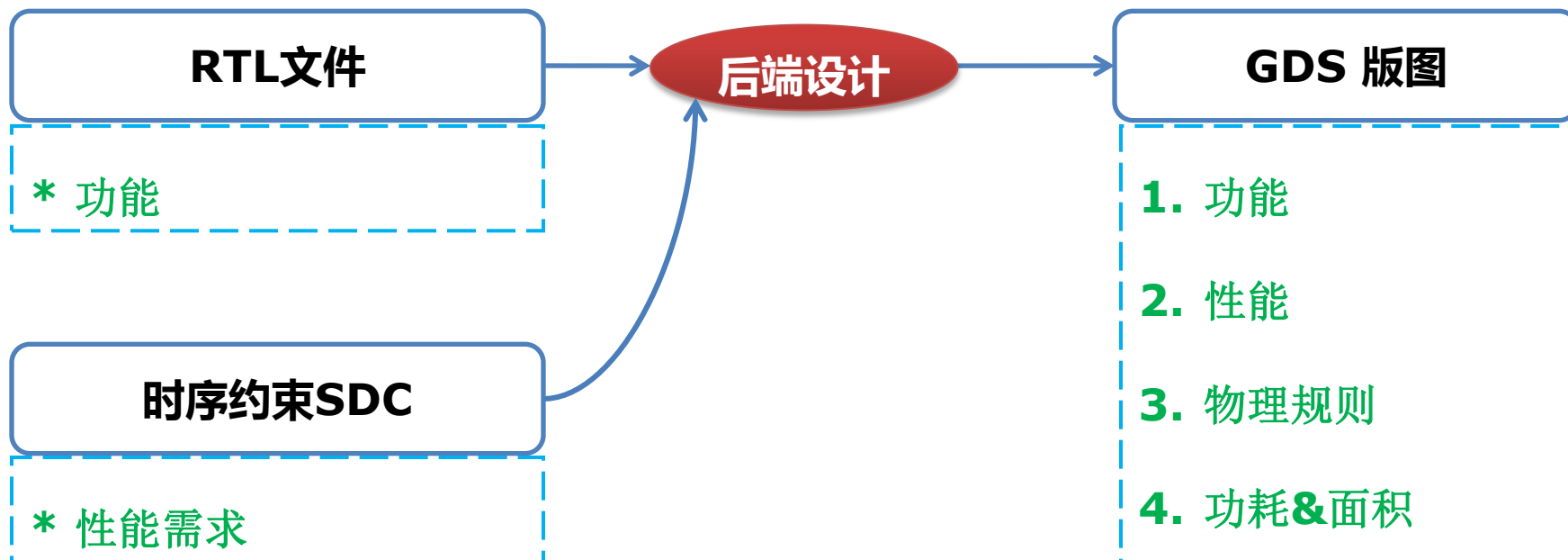
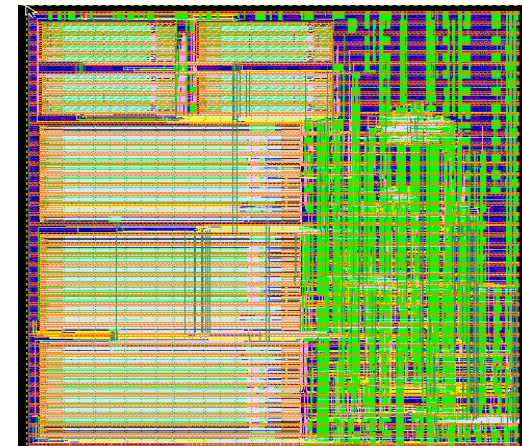
```
module oc8051_dptr(clk, rst, addr, data_in, data2_in, wr, wd2, wr_bit, data_hi, data_lo);
data_lo);

input clk, rst, wr, wr_bit;
input [2:0] wd2;
input [7:0] addr, data_in, data2_in;

output [7:0] data_hi, data_lo;

reg [7:0] data_hi, data_lo;

always @(posedge clk or posedge rst)
begin
    if (rst) begin
        data_hi <= #1 'OC8051_RST_DPH;
        data_lo <= #1 'OC8051_RST_DPL;
    end else if (wd2== 'OC8051_RWS_DPTR) begin
        //write from destination 2 and 1
        data_hi <= #1 data2_in;
        data_lo <= #1 data_in;
    end else if ((addr== 'OC8051_SFR_DPTR_HI) & (wr) & !(wr_bit))
        //
        //case of writing to dptr
        data_hi <= #1 data_in;
        else if ((addr== 'OC8051_SFR_DPTR_LO) & (wr) & !(wr_bit))
            data_lo <= #1 data_in;
    end
end
endmodule
```



Notes: SDC, Synopsys Design Constraints

爱芯人课堂/QQ群187291154

1.逻辑综合，从RTL到Gate-Level Netlist

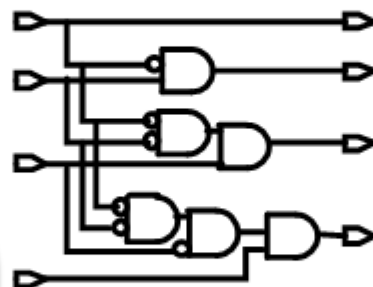
Synthesis = Translation + Optimization + Mapping

```
residue = 16'h0000;  
if (high_bits == 2'b10)  
    residue = state_table[index];  
else  
    state_table[index] = 16'h0000;
```

HDL Source

RTL

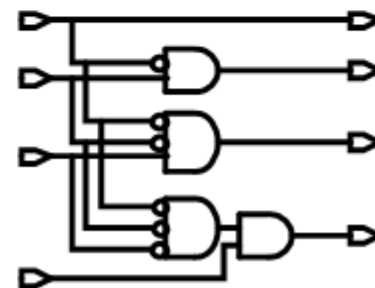
Translate



**Generic Boolean
(GTECH)**

**GTECH;
逻辑表达式
 $Y=A\&B + C\&D$**

Optimize + Map



Target Technology

与工艺库相关的Netlist

Notes :

1. 逻辑综合 :

Logic Synthesis / Synthesis

2. 门级网表 :

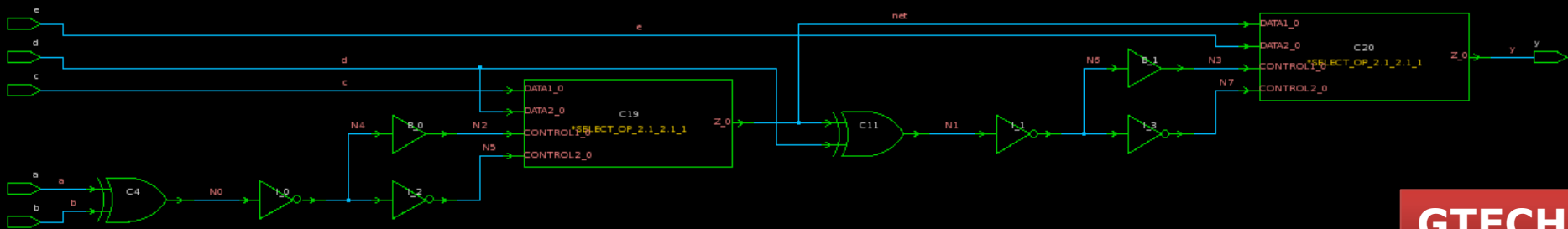
Gate-Level Netlist / Netlist

1.逻辑综合，从RTL到Gate-Level Netlist

```
module top (a,b,c,d,e,y);  
  input a,b,c,d,e;  
  output y;  
  
  wire net;  
  assign net = (a==b)?c:d;  
  assign y = (net==d)?net:e;  
  
endmodule
```

RTL

Translation



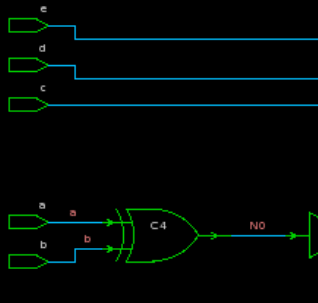
GTECH

1.逻辑综合，从RTL到Gate-Level Netlist

```
module top (a,b,c,d,e,y);  
  input a,b,c,d,e;  
  output y;  
  
  wire net;  
  assign net = (a==b)?c:d;  
  assign y = (net==d)?net:e;  
  
endmodule
```

RTL

Translation



```
module top ( a, b, c, d, e, y );  
  input a, b, c, d, e;  
  output y;  
  wire  N0, N1, N2, N3, N4, N5, net, N6, N7;  
  
  GTECH_XOR2 C4 ( .A(a), .B(b), .Z(N0) );  
  GTECH_NOT I_0 ( .A(N0), .Z(N4) );  
  GTECH_XOR2 C11 ( .A(net), .B(d), .Z(N1) );  
  GTECH_NOT I_1 ( .A(N1), .Z(N6) );  
  SELECT_OP C19 ( .DATA1(c), .DATA2(d), .CONTROL1(N2), .CONTROL2(N5), .Z(net)  
    );  
  GTECH_BUF B_0 ( .A(N4), .Z(N2) );  
  SELECT_OP C20 ( .DATA1(net), .DATA2(e), .CONTROL1(N3), .CONTROL2(N7), .Z(y)  
    );  
  GTECH_BUF B_1 ( .A(N6), .Z(N3) );  
  GTECH_NOT I_2 ( .A(N4), .Z(N5) );  
  GTECH_NOT I_3 ( .A(N6), .Z(N7) );  
  
endmodule
```

GTECH

1.逻辑综合，从RTL到Gate-Level Netlist

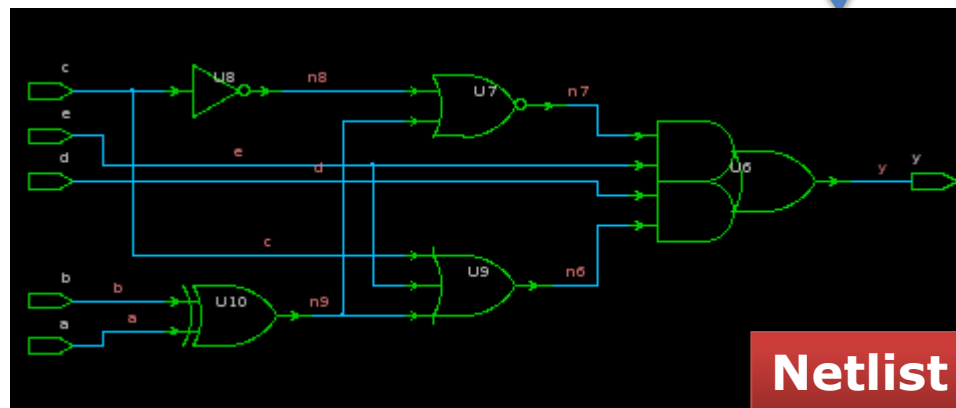
```
module top ( a, b, c, d, e, y );  
  input a, b, c, d, e;  
  output y;  
  wire  N0, N1, N2, N3, N4, N5, net, N6, N7;
```

GTECH

```
  GTECH_XOR2 C4 ( .A(a), .B(b), .Z(N0) );  
  GTECH_NOT I_0 ( .A(N0), .Z(N4) );  
  GTECH_XOR2 C11 ( .A(net), .B(d), .Z(N1) );  
  GTECH_NOT I_1 ( .A(N1), .Z(N6) );  
  SELECT_OP C19 ( .DATA1(c), .DATA2(d), .CONTROL1(N2), .CONTROL2(N5), .Z(net)  
    );  
  GTECH_BUF B_0 ( .A(N4), .Z(N2) );  
  SELECT_OP C20 ( .DATA1(net), .DATA2(e), .CONTROL1(N3), .CONTROL2(N7), .Z(y)  
    );  
  GTECH_BUF B_1 ( .A(N6), .Z(N3) );  
  GTECH_NOT I_2 ( .A(N4), .Z(N5) );  
  GTECH_NOT I_3 ( .A(N6), .Z(N7) );  
endmodule
```

+ SDC

Map&Optimization



1.逻辑综合，从RTL到Gate-Level Netlist

```
module top ( a, b, c, d, e, y );
  input a, b, c, d, e;
  output y;
  wire  N0, N1, N2, N3, N4, N5, net, N6, N7;
```

GTECH

```
GTECH_XOR2 C4 ( .A(a), .B(b), .Z(N0) );
GTECH_NOT I_0 ( .A(N0), .Z(N4) );
GTECH_XOR2 C11 ( .A(net), .B(d), .Z(N1) );
GTECH_NOT I_1 ( .A(N1), .Z(N6) );
SELECT_OP C19 ( .DATA1(c), .DATA2(d), .CONTROL1(N2), .CONTROL2(N5), .Z(net)
);
GTECH_BUF B_0 ( .A(N4), .Z(N2) );
SELECT_OP C20 ( .DATA1(net), .DATA2(e), .CONTROL1(N3), .CONTROL2(N7), .Z(y)
);
GTECH_BUF B_1 ( .A(N6), .Z(N3) );
GTECH_NOT I_2 ( .A(N4), .Z(N5) );
GTECH_NOT I_3 ( .A(N6), .Z(N7) );
endmodule
```

逻辑综合的结果要满足sdc的时序约束

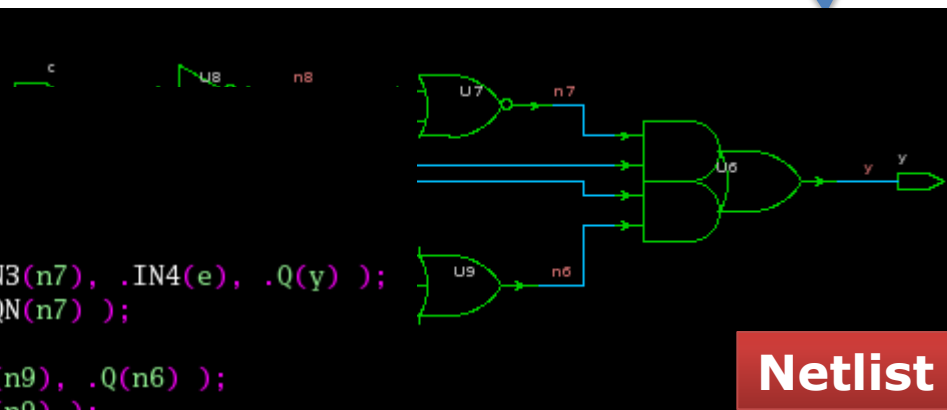
+ SDC

Map&Optimization

修改记录：svf

```
module top ( a, b, c, d, e, y );
  input a, b, c, d, e;
  output y;
  wire  n6, n7, n8, n9;

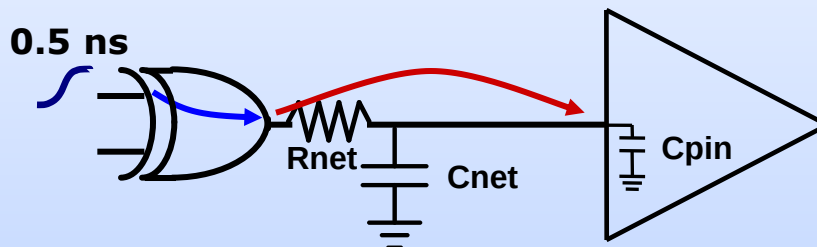
  A022X1 U6 ( .IN1(d), .IN2(n6), .IN3(n7), .IN4(e), .Q(y) );
  NOR2X0 U7 ( .IN1(n8), .IN2(n9), .QN(n7) );
  INVX0 U8 ( .INP(c), .ZN(n8) );
  OR3X1 U9 ( .IN1(c), .IN2(e), .IN3(n9), .Q(n6) );
  XOR2X1 U10 ( .IN1(b), .IN2(a), .Q(n9) );
endmodule
```



Netlist

1.逻辑综合，从RTL到Gate-Level Netlist

dc如何计算delay呢？



Cell Delay = $f(\text{Input Transition Time}, C_{net} + C_{pin})$

Net Delay = $f(R_{net}, C_{net} + C_{pin})$

- 需要计算Cell和Net的delay;
- Cell和Net的延时都与Net的寄生参数（parasitic parameter）有关，即net的R和C;

1.逻辑综合，从RTL到Gate-Level Netlist

此时还不到物理实现，连接线的r和c，dc如何估算呢？

Example: Standard Format

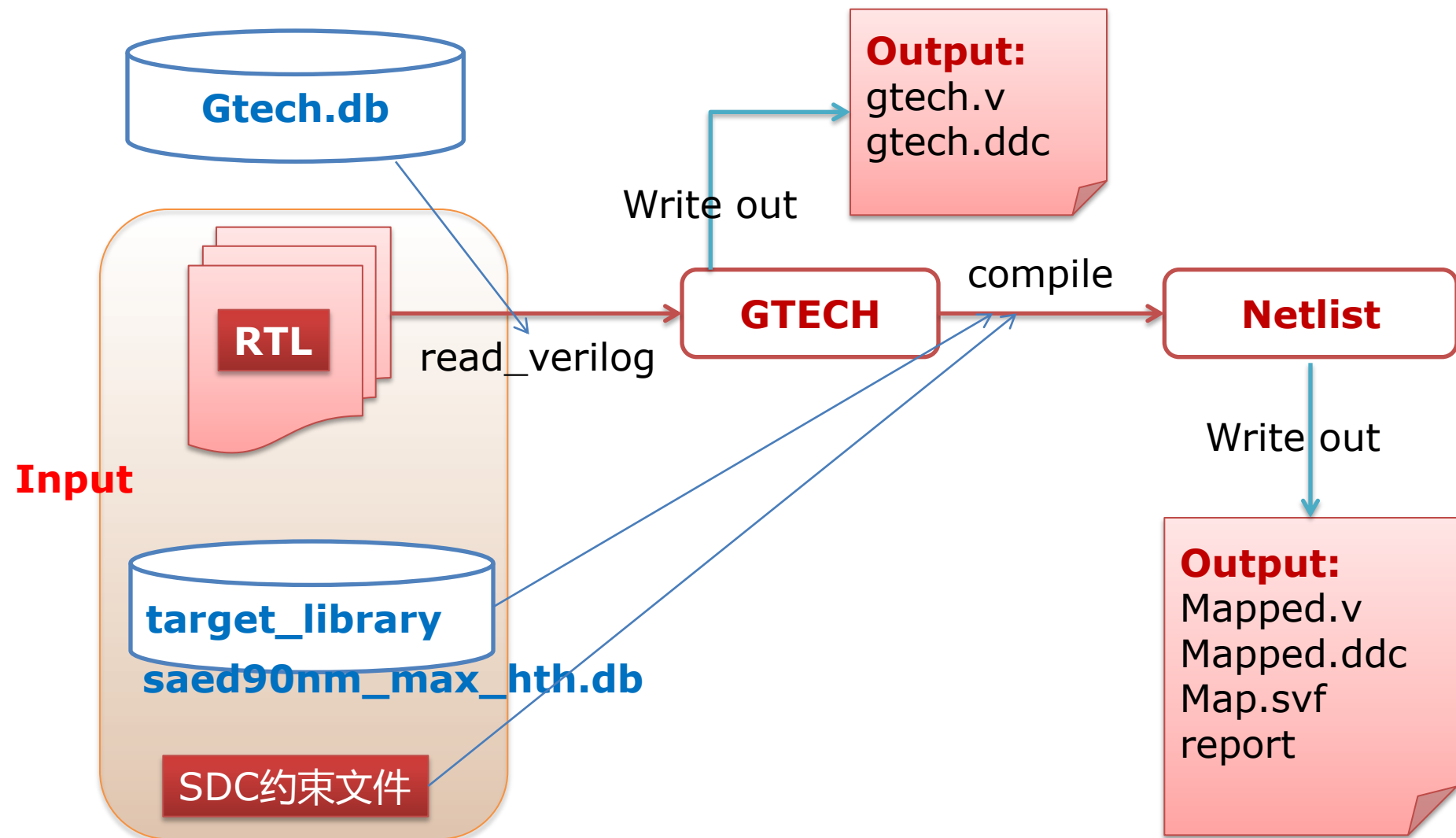
Wire-Load-Model

```
Name          : 160KGATES
Location       : ssc_core_slow
Resistance     : 0.000271
Capacitance    : 0.00017
Area          : 0
Slope         : 50.3104
Fanout        Length
-----
      1      31.44
      2      81.75
      3     132.07
      4     182.38
      5     232.68
Time Unit      : 1ns
Capacitive Load Unit : 1.000000pf
Pulling Resistance Unit : 1kilo-ohm
```

Annotations:

- R per unit length* (points to Resistance)
- C per unit length* (points to Capacitance)
- Extrapolation slope* (points to Slope)

1.逻辑综合，从RTL到Gate-Level Netlist

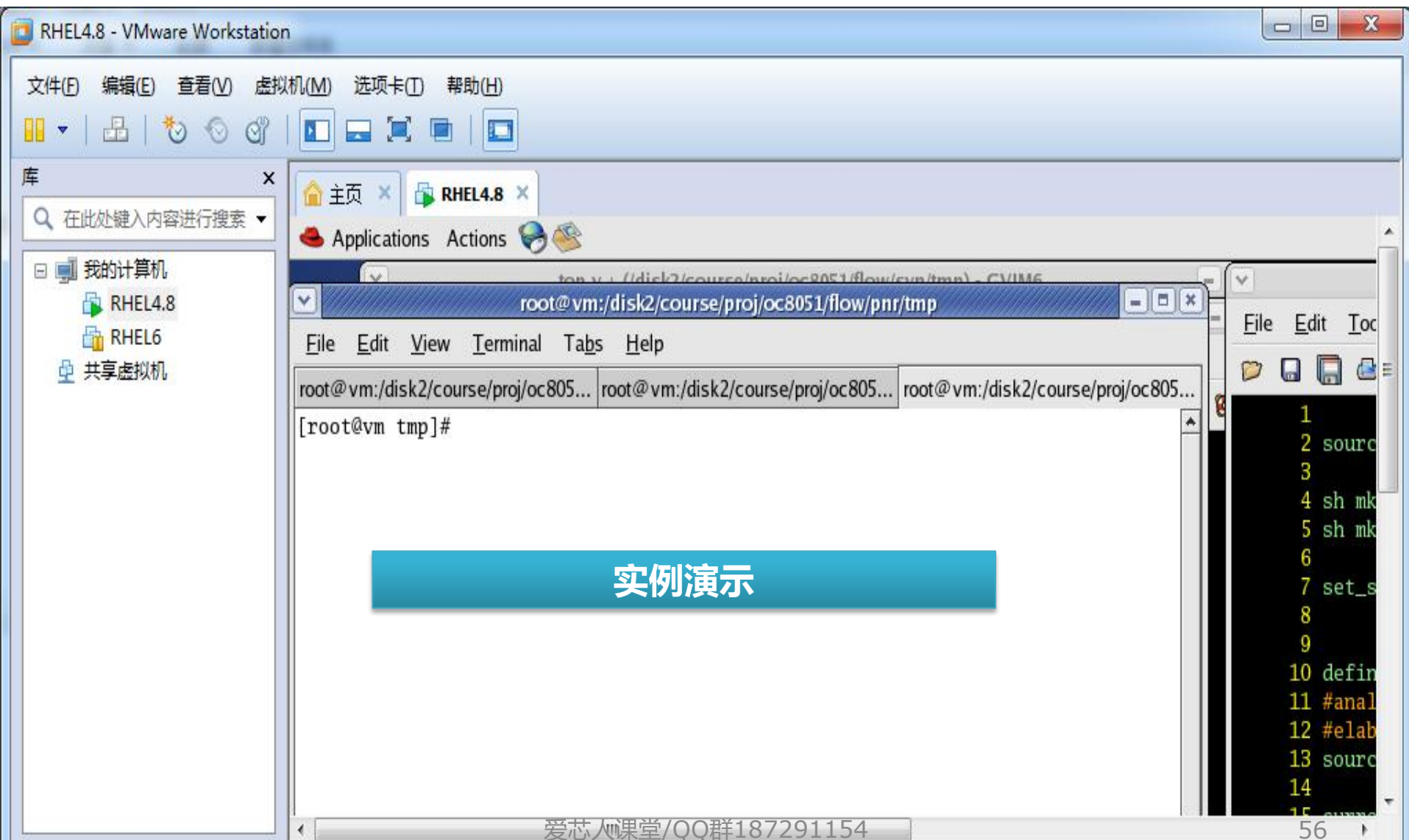


1.逻辑综合，从RTL到Gate-Level Netlist

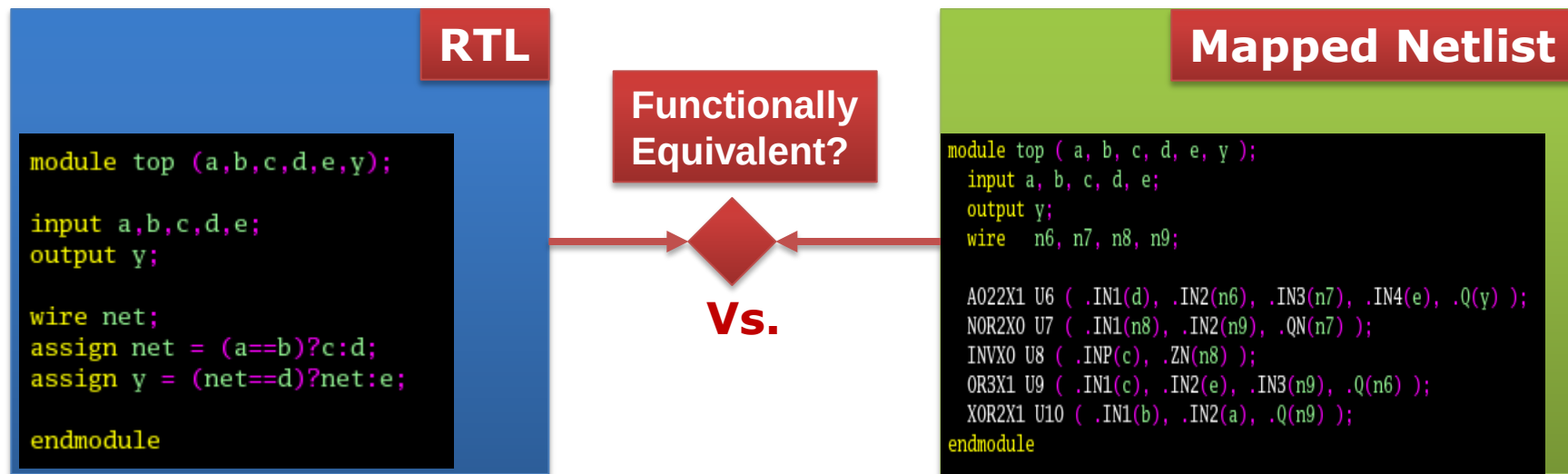
Notes :

1. 简单地说，逻辑综合的目标就是把Design从RTL映射到Gate Level Netlist，同时满足设置的约束（包括时序，面积，功耗等约束）；
2. 逻辑综合要满足在sdc约束下的setup timing，不需要理会hold timing；
3. 常用的工具有synopsys的design compiler（dc）和cadence的RTL Compiler (rc);

1.逻辑综合，从RTL到Gate-Level Netlist



2.形式验证 , RTL vs. Mapped.v

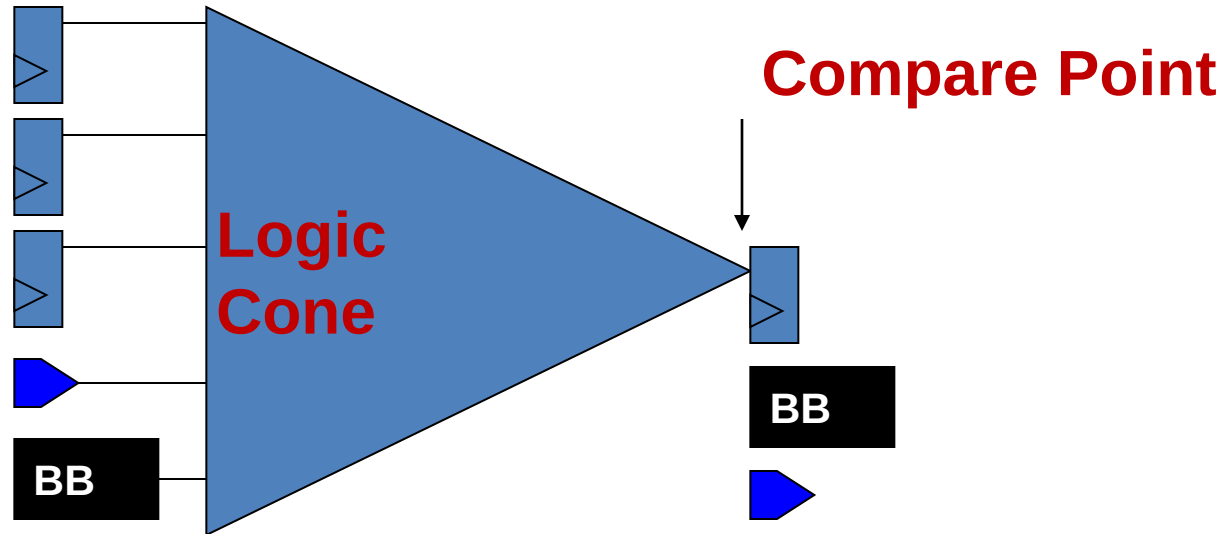


Notes :

1. 形式验证 : Formality/Formal
2. 形式验证工具既不是仿真工具 , 也不是测试向量生成工具 , 更不是时序分析工具。它是比较两个设计数据在功能上是否等价的工具 ;
3. 常用的工具有synopsys的formality (fm) 和Cadence的Conformal (lec) ;

2.形式验证

A design contains **Logic Cones** and **Compare Points**



Inputs

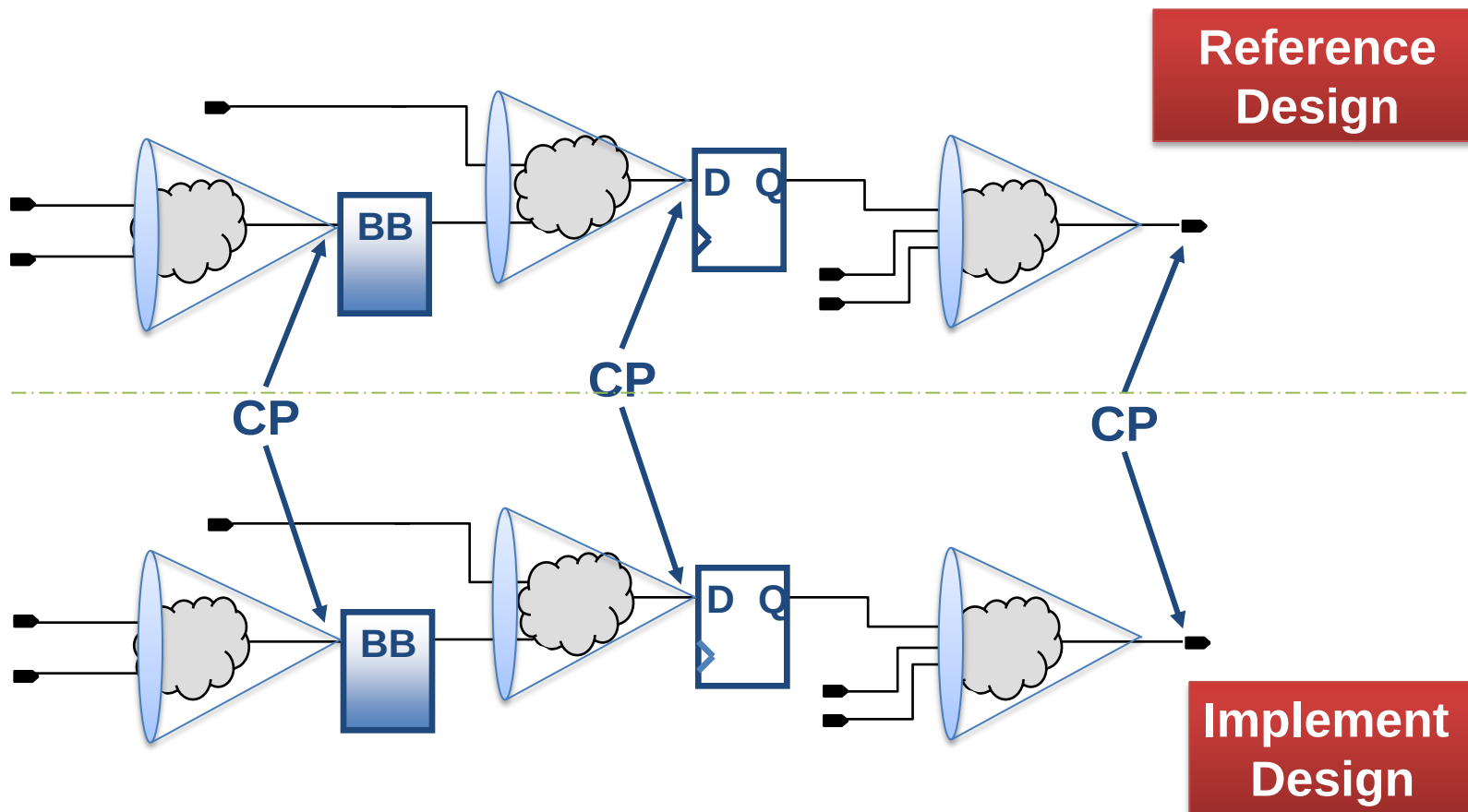
- Outputs from Registers
- Primary Input Ports
- Outputs from Black Boxes

Compare Points

- Inputs to Registers
- Primary Output Ports
- Inputs to Black Boxes

2.形式验证

Match



2.形式验证

Verify

- For each matched pair of compare points, Formality tries to :

Either

- Confirms same response for all possible input combinations.
- Marks point as “**passed**”

Or

- Finds a “counter example” that shows different response
- Marks point as “**failed**”

2.形式验证

Debug

Formality (R) Console - Synopsys Inc.

File Edit View Designs Run Window Help

Diagnosis Complete

1. Reference 2. Implementation 3. Setup 4. Match 5. Verify 6. Debug

Failing Points Passing Points Aborted Points Error Candidates

	Type	Reference	Size	Implementation	Size
1	treg	db:/WORK/mR4000/Instruction_reg[0]		sim:/WORK/mR4000/Instruction_reg[0]	
2	treg	db:/WORK/mR4000/Instruction_reg[1]		sim:/WORK/mR4000/Instruction_reg[1]	
3	port	db:/WORK/mR4000/ReadAdd[0]		sim:/WORK/mR4000/ReadAdd[0]	
4	port	db:/WORK/mR4000/ReadAdd[1]		sim:/WORK/mR4000/ReadAdd[1]	

Number of Failing Points: 4

Filter: none

Diagnose All Diagnose Selected Point

Status: Diagnosing sim:/WORK/mR4000 vs db:/WORK/mR4000...
Status: Diagnosis initializing...
Info: Limiting number of input patterns to 256.
Status: Analyzing patterns...
Analysis completed
Diagnosis completed
1

Log Errors Warnings History Last Command

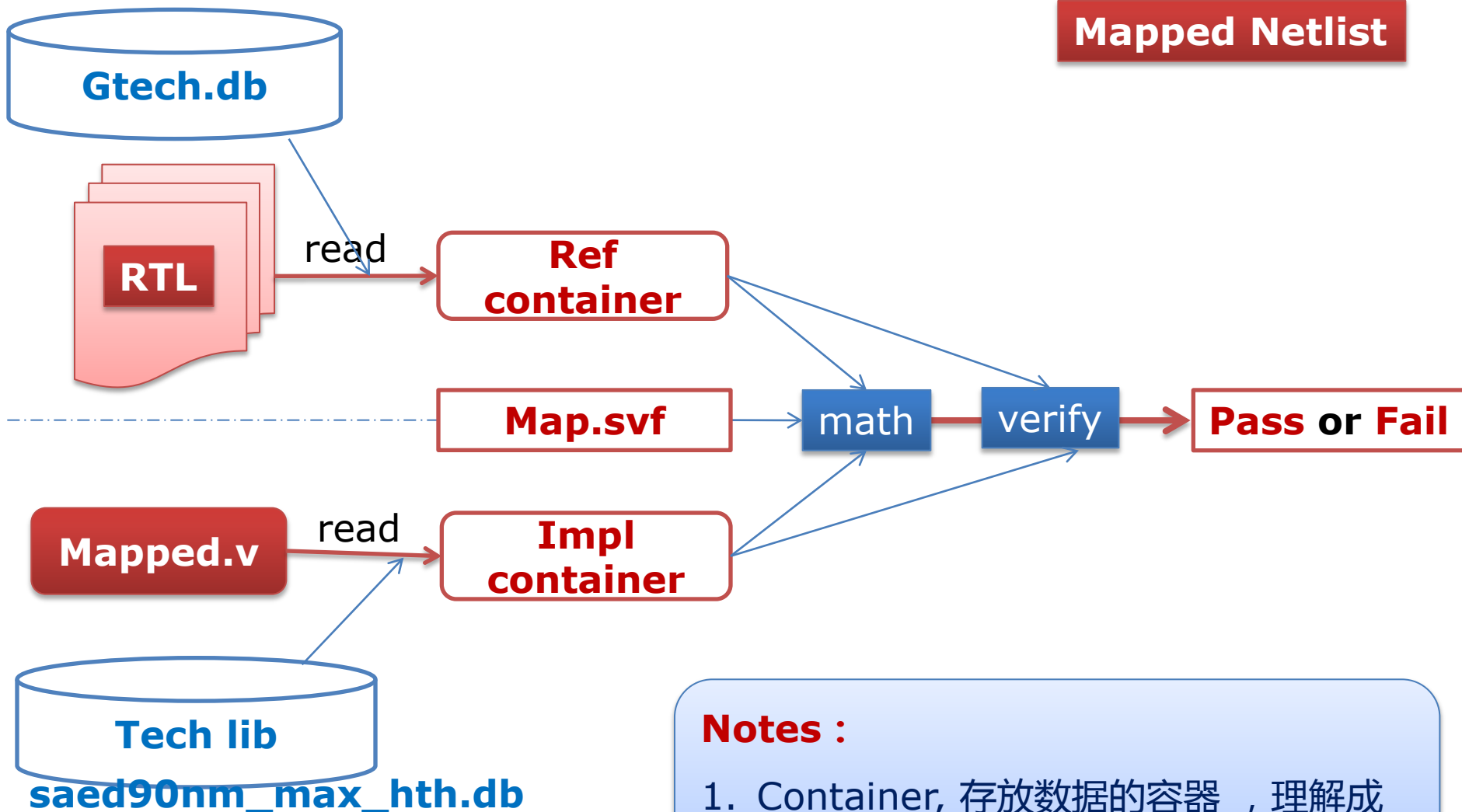
Formality (verify)>

Ready

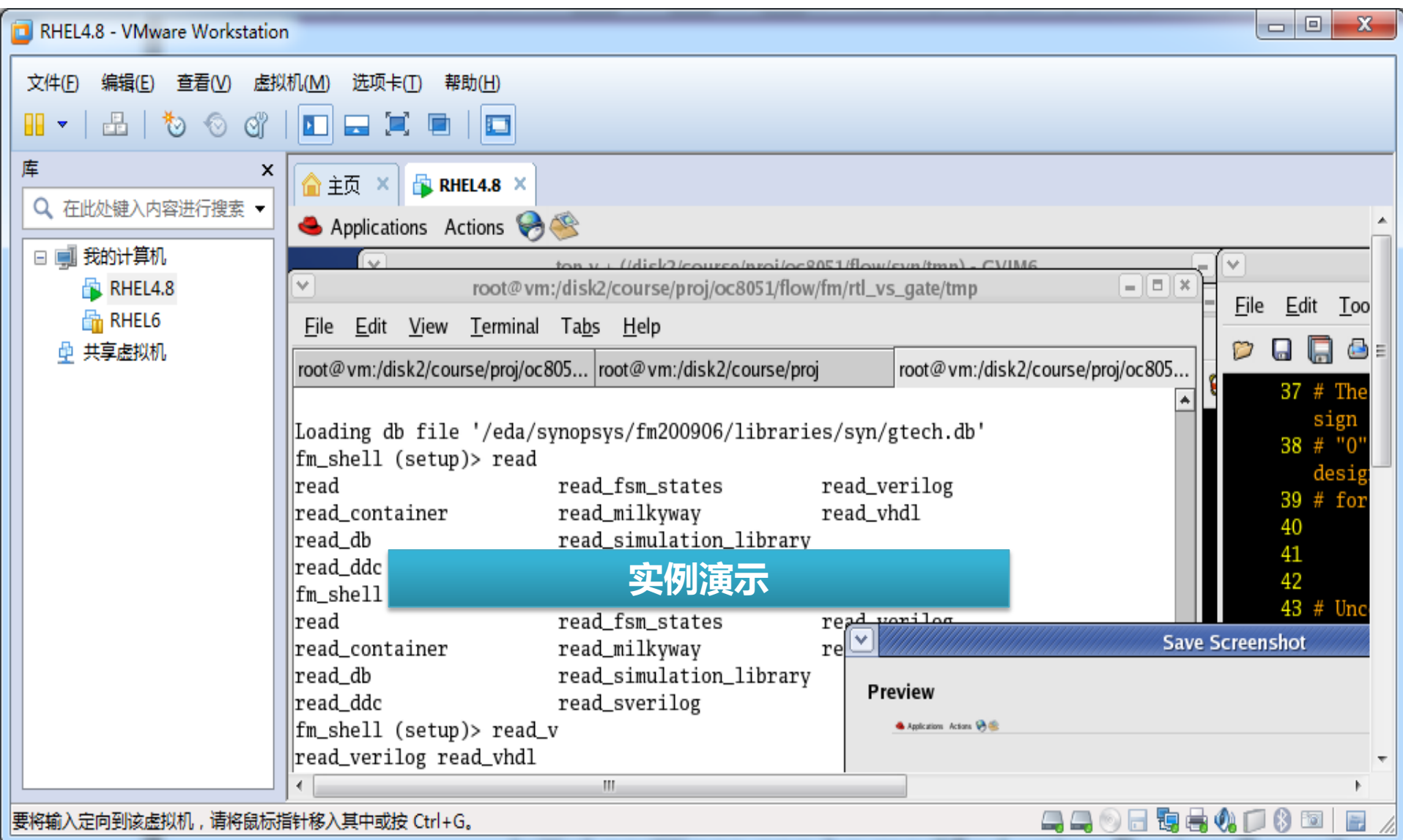
Shell State: Verify

2.形式验证 , RTL vs. Mapped.v

Mapped Netlist



2.形式验证 , RTL vs. Mapped.v

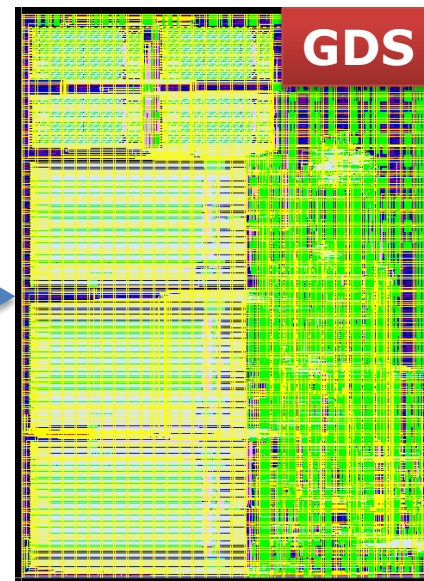


3.物理实现，从Netlist到GDS

```
module top ( a, b, c, d, e, y );  
  input a, b, c, d, e;  
  output y;  
  wire  n6, n7, n8, n9;  
  
  A022X1 U6 ( .IN1(d), .IN2(n6), .IN3(n7), .IN4(e), .Q(y) );  
  NOR2X0 U7 ( .IN1(n8), .IN2(n9), .QN(n7) );  
  INVX0 U8 ( .INP(c), .ZN(n8) );  
  OR3X1 U9 ( .IN1(c), .IN2(e), .IN3(n9), .Q(n6) );  
  XOR2X1 U10 ( .IN1(b), .IN2(a), .Q(n9) );  
endmodule
```

Mapped Netlist

物理实现



GDS

Notes :

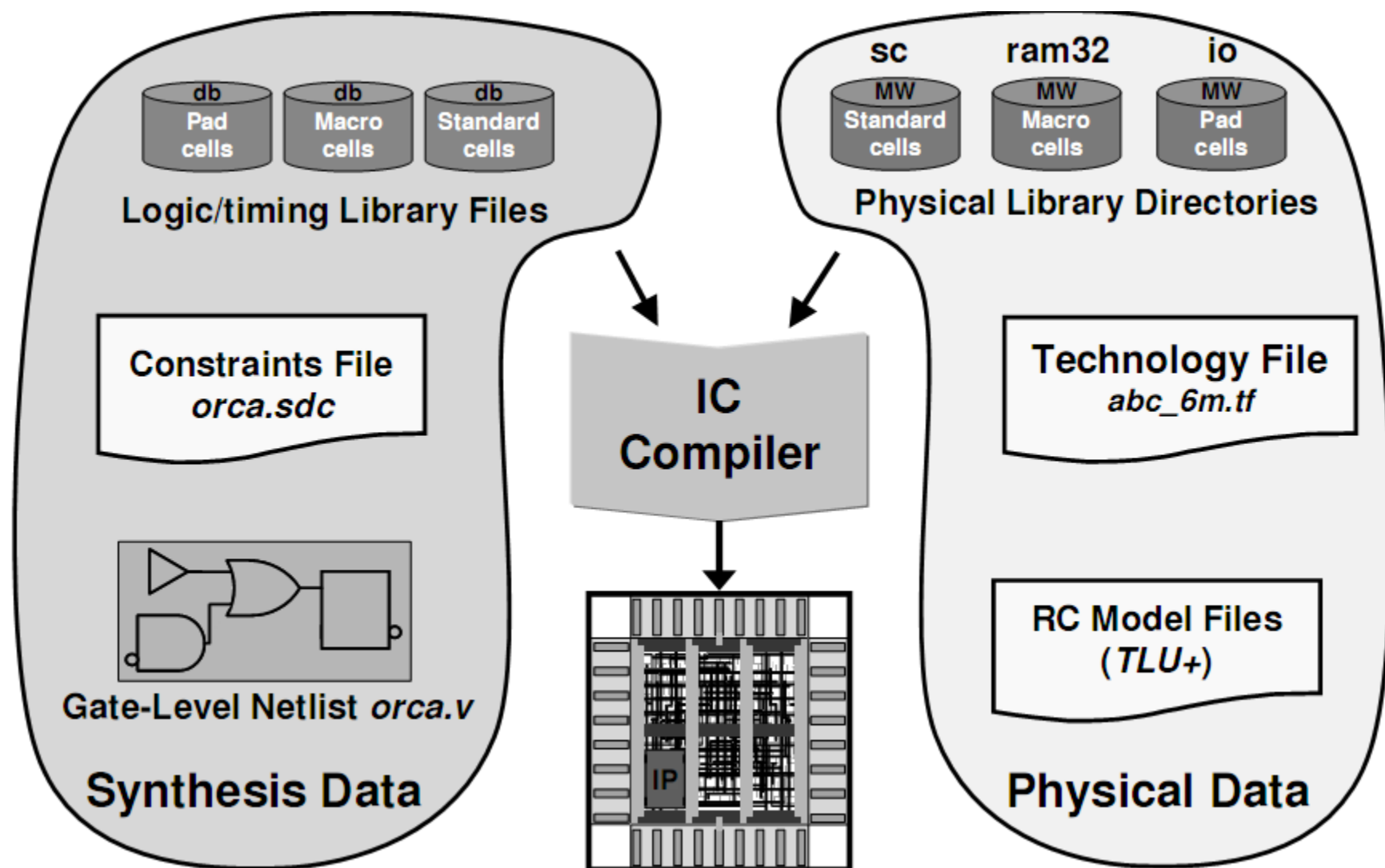
1. 物理实现 : Physical Implementation/Impl/P&R/PR;

3.物理实现，从Netlist到GDS



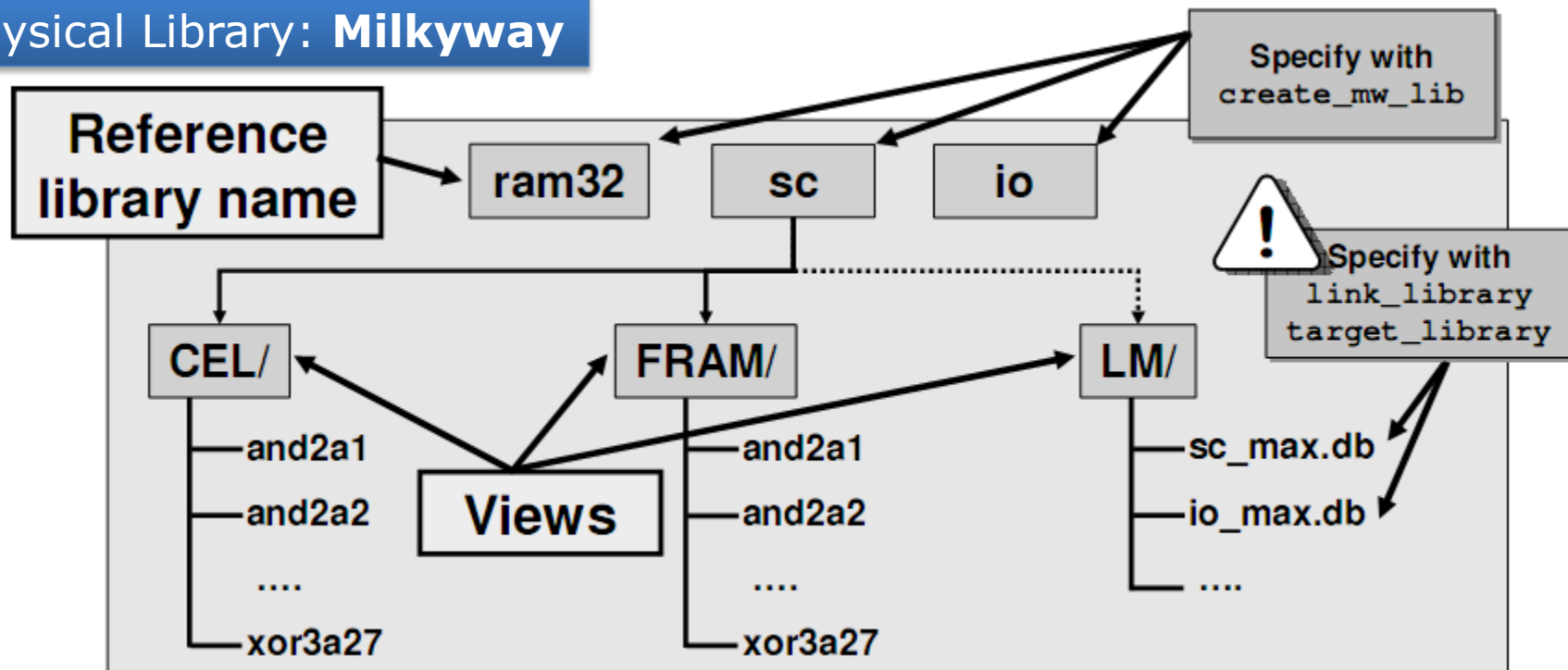
3.物理实现

Logic Library & Physical Library & Tech File



3.物理实现

Physical Library: **Milkyway**



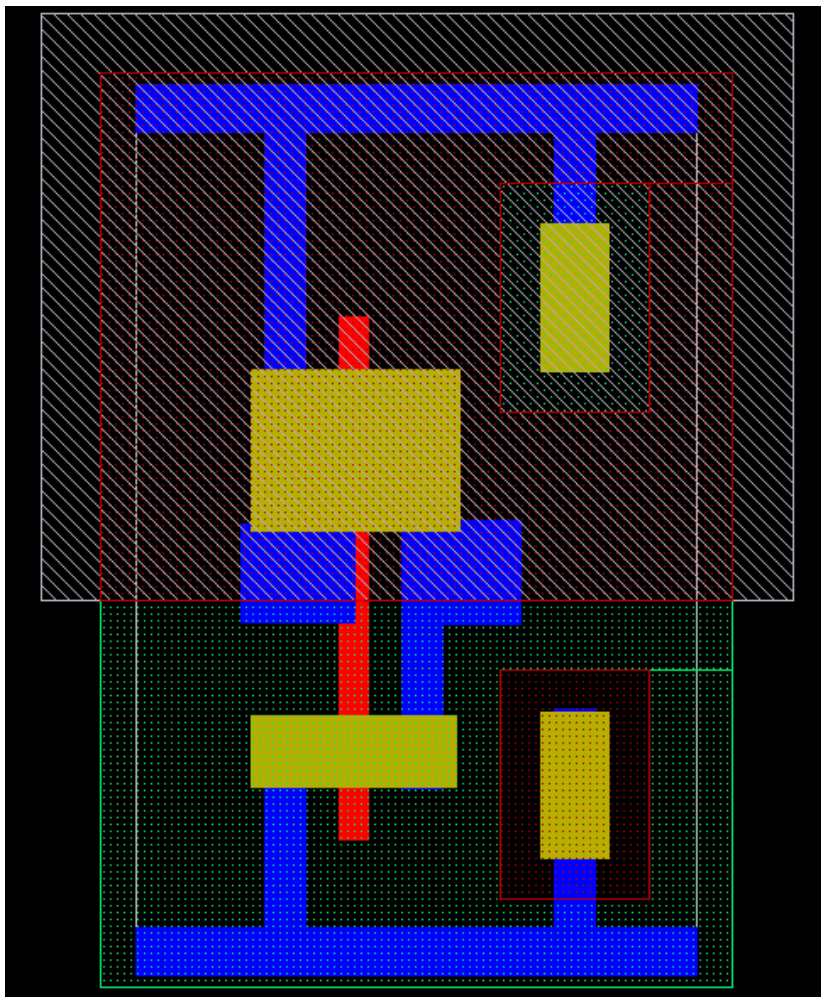
```
[root@vm pnr]# ls output_data/v0/oc8051_top_LIB/  
CEL  FILL  FRAM  lib  lib_1  lib_2  lib_3  lib_bck  ROUTE
```

Notes :

1. ICC中，所有物理库都以milkyway的格式存在，包括stdcel、memory等的物理库，也包括设计本身；所有物理信息都装在milkyway中；

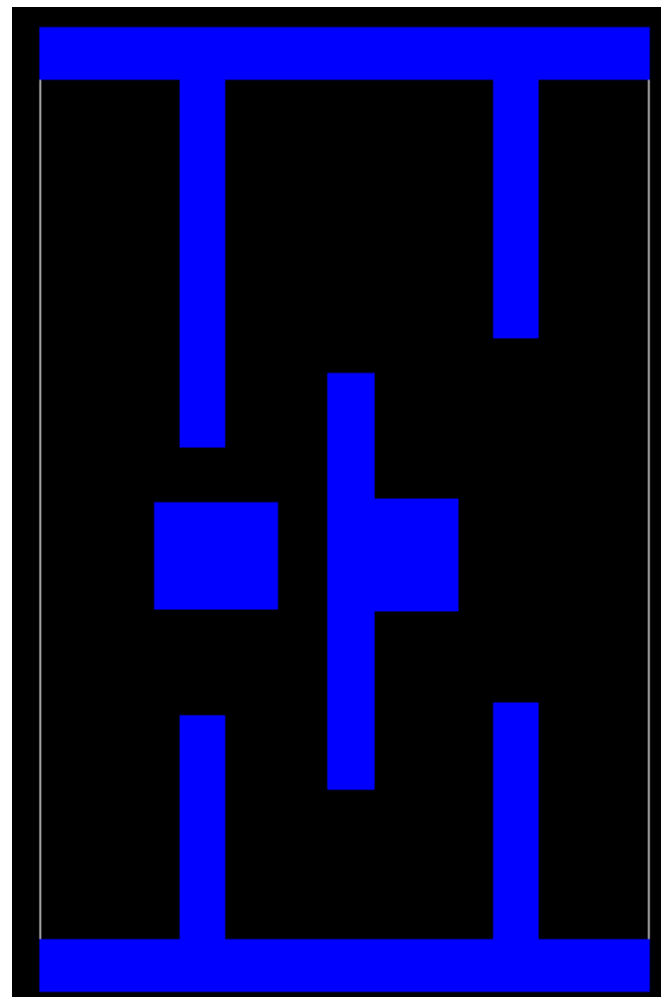
3.物理实现

Milkyway : CEL View vs. FRAM View



INV.CEL

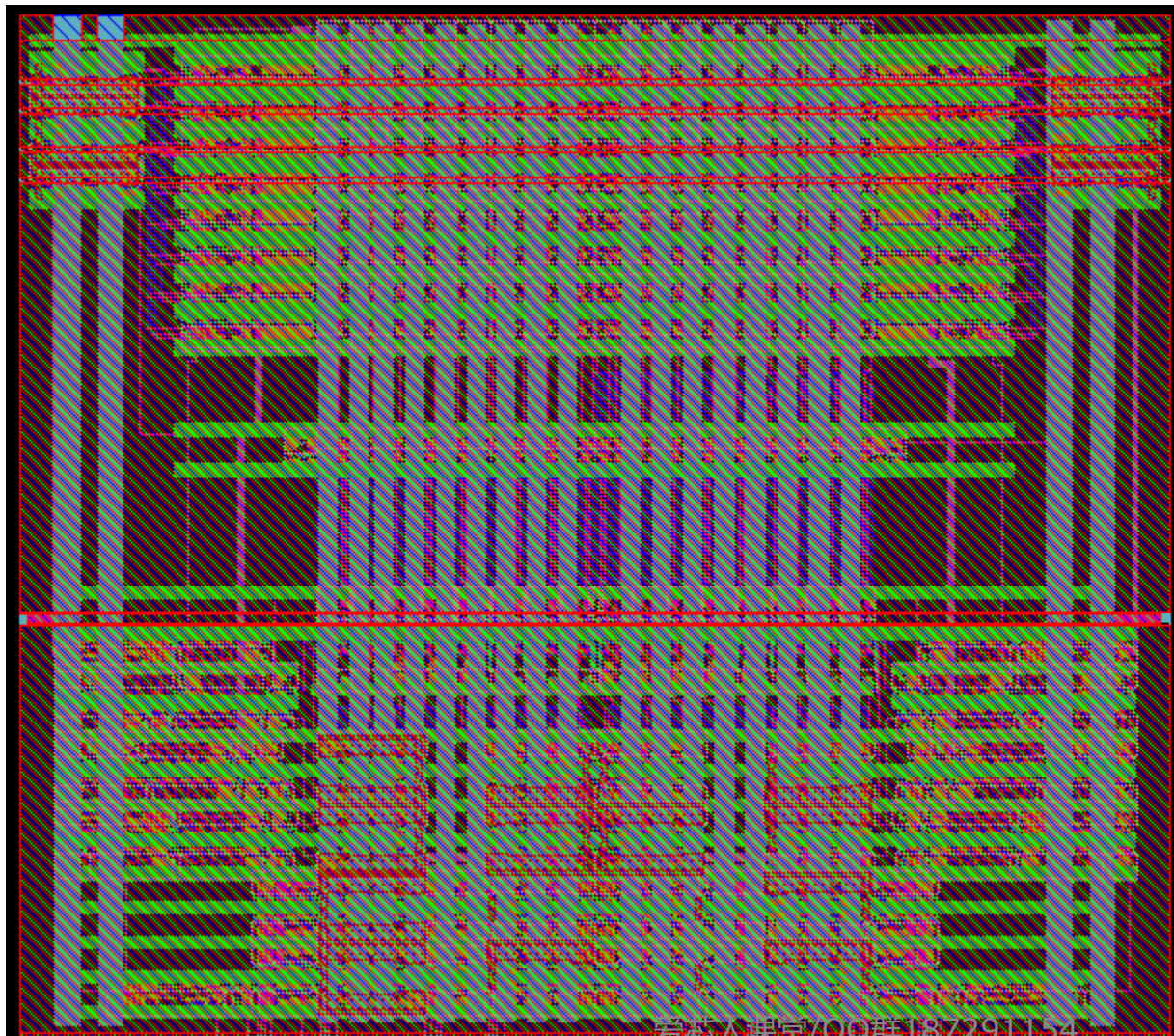
爱芯人课堂/QQ群187291154



INV.FRAME

3.物理实现

Milkyway : CEL View vs. FRAM View

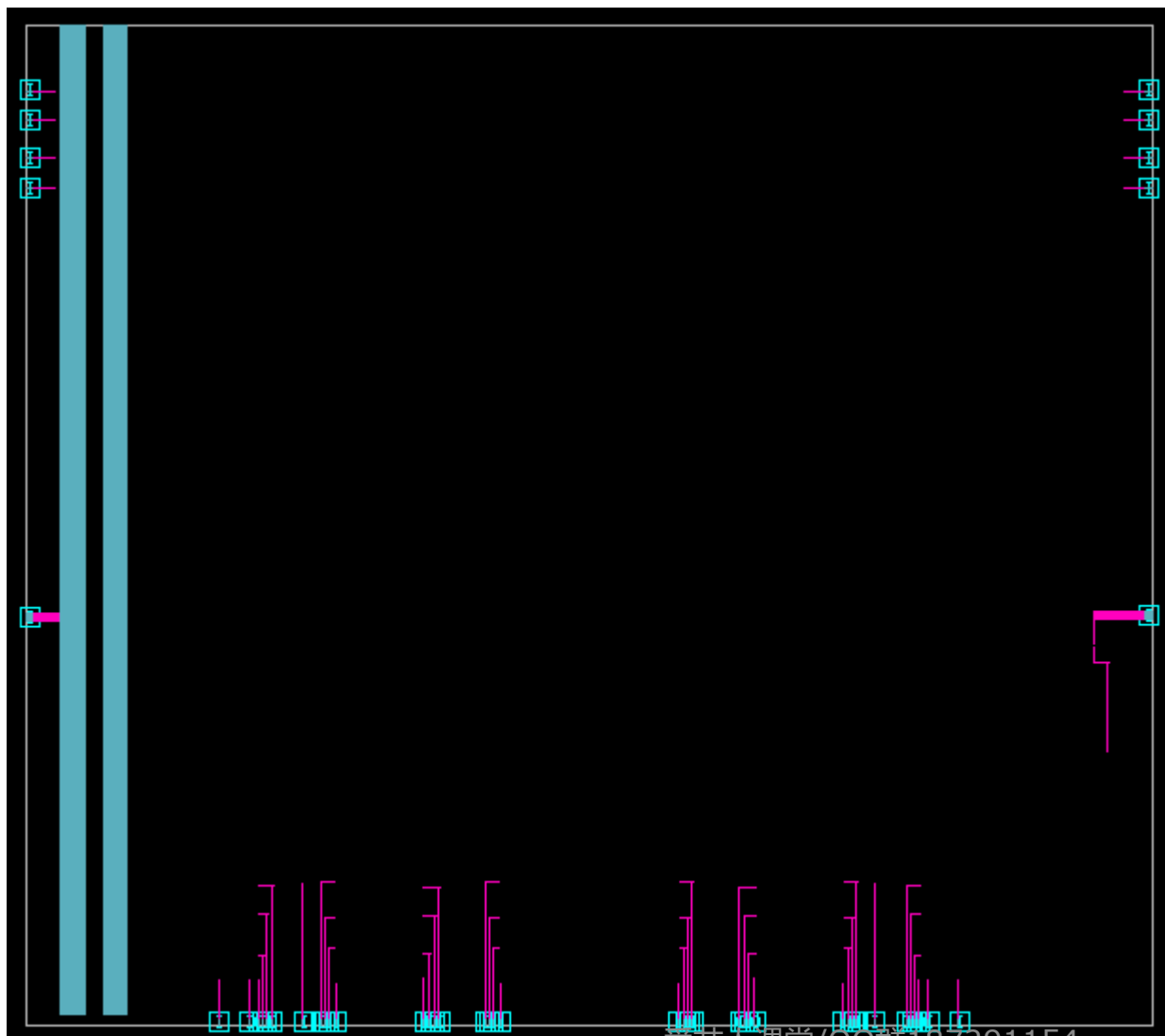


SRAM8x16.CEL

Top Metal = M5 ;
包括所有的物理信息 ;

3.物理实现

Milkyway : CEL View vs. FRAM View



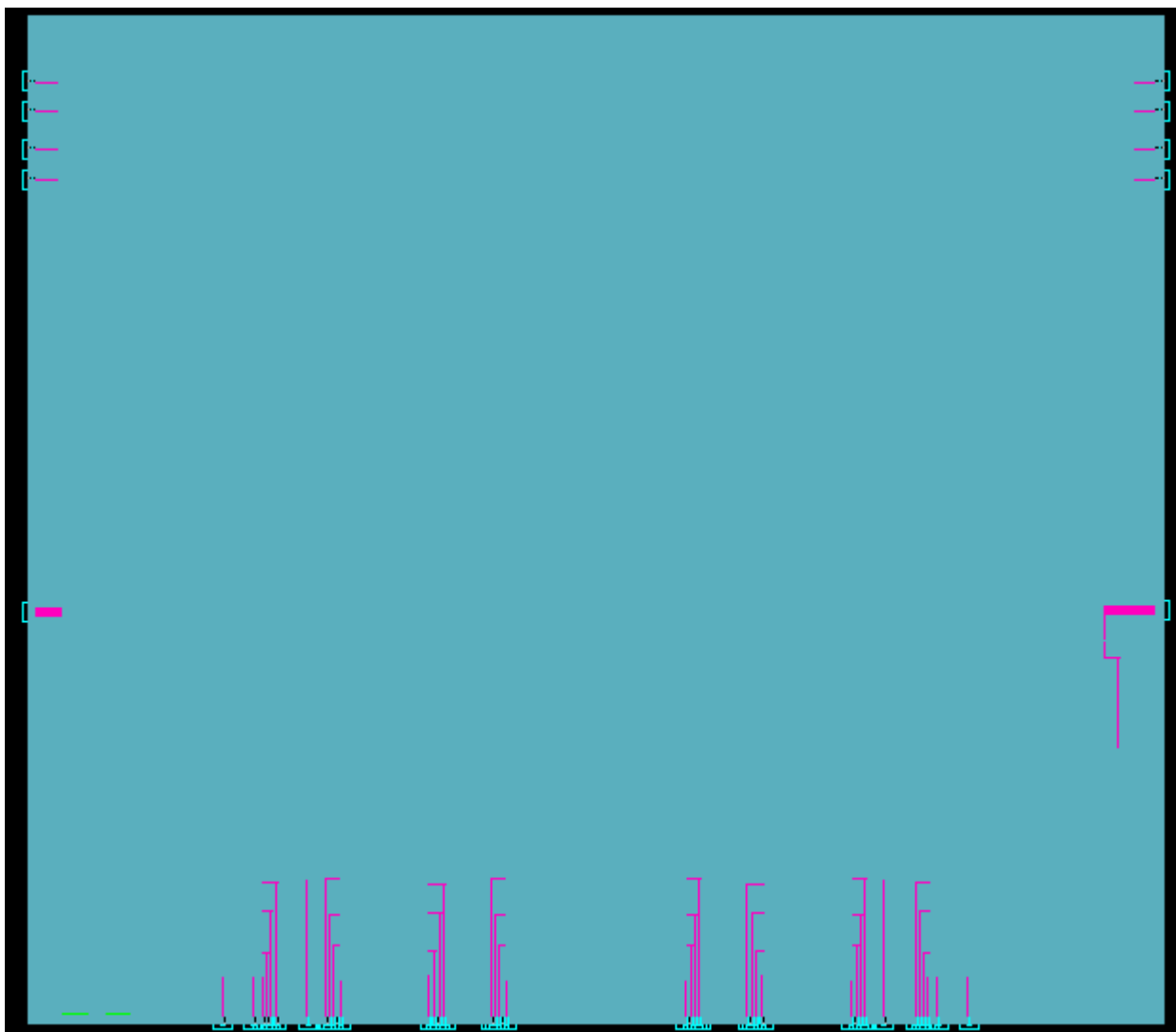
SRAM8x16.FRAME

Top Metal = M5 ;
包括pin的位置和形状,
routing blockage ;

把blockage去掉后

3.物理实现

Milkyway : CEL View vs. FRAM View



SRAM8x16.FRAME

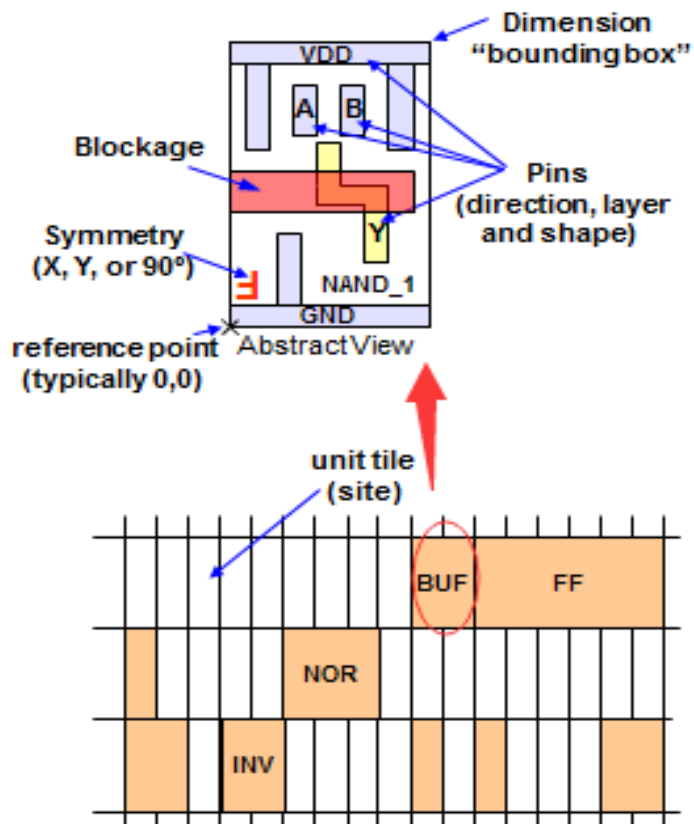
Top Metal = M5 ;
包括pin的位置和形状,
routing blockage ;

在PnR过程中, ICC
只会把FRAM View
读进memory, CEL
View数据量太大,
CEL View只在最后
写GDS时用;

Routing blockage :
避免route时把连线
伸进去, 造成短路;

3.物理实现

Milkyway : FRAM View



3.物理实现

Technology File: .tf

- The *technology file* is unique to each technology
- Contains metal layer technology parameters:
 - Number and name designations for each layer/via
 - Physical and electrical characteristics of each layer/via
 - Design rules for each layer/Via (Minimum wire widths and wire-to-wire spacing, etc.)
 - Units and precision for electrical units
 - Colors and patterns of layers for display
 - ...

.tf文件来定义物理规则；

3.物理实现

Technology File: .tf

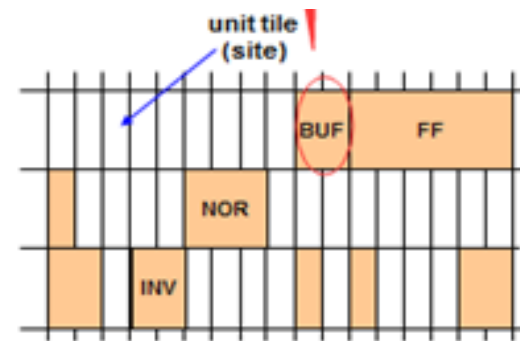
```
Technology {
    name                = "saed90"
    date                 = "May 25 2007"
    dielectric           = 3.61e-05
    unitTimeName         = "ns"
    timePrecision        = 1000
    unitLengthName       = "micron"
    lengthPrecision      = 1000
    gridResolution       = 5
    unitVoltageName      = "V"
    voltagePrecision     = 1000
    unitCurrentName      = "uA"
    currentPrecision     = 1000
    unitPowerName        = "pw"
    powerPrecision       = 1000
    unitResistanceName   = "kohm"
    resistancePrecision  = 10000000
    unitCapacitanceName  = "pf"
    capacitancePrecision = 10000000
    unitInductanceName   = "nh"
    inductancePrecision  = 100
    minBaselineTemperature = 25
    nomBaselineTemperature = 25
    maxBaselineTemperature = 25
}
```

3.物理实现

Technology File: .tf

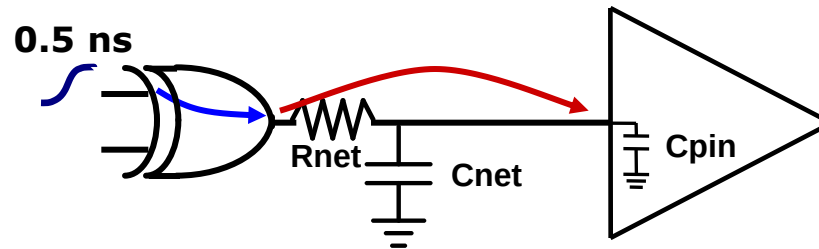
```
Layer      "M1" {  
    layerNumber      = 11  
    maskName         = "metal1"  
    isDefaultLayer   = 1  
    visible          = 1  
    selectable       = 1  
    blink            = 0  
    color             = "blue"  
    lineStyle        = "solid"  
    pattern           = "rectangleX"  
    pitch             = 0.32  
    defaultWidth     = 0.14  
    minWidth          = 0.14  
    minSpacing        = 0.14  
    maxWidth          = 1000  
    minArea           = 0.06  
}
```

```
Tile      "unit" {  
    width      = 0.32  
    height     = 2.88  
}
```



3.物理实现

Parasitic RC Model: TLU+ Technology File

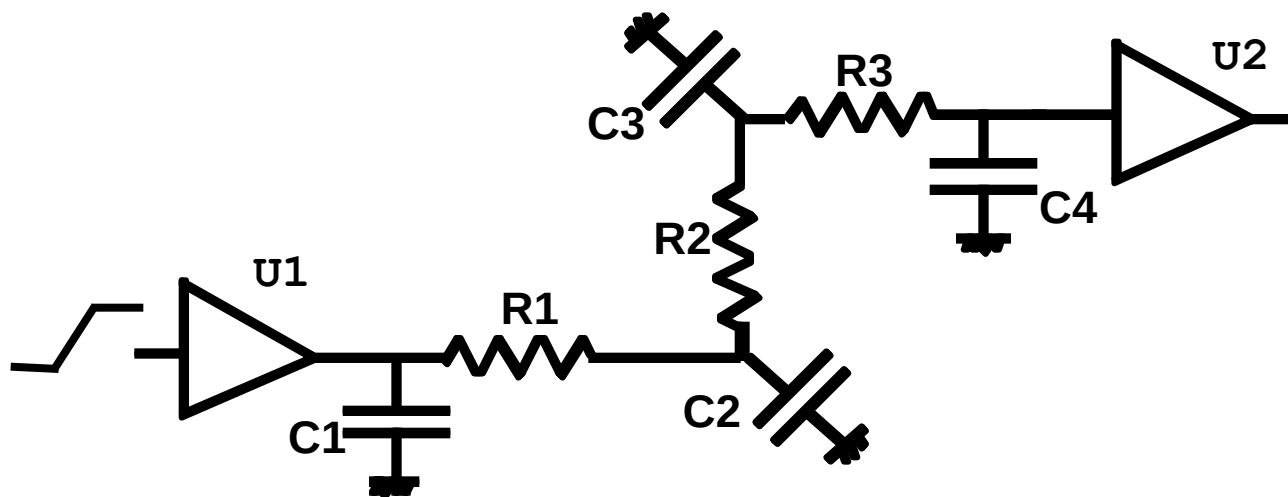


$$\text{Cell Delay} = f(\text{Input Transition Time}, C_{\text{net}} + C_{\text{pin}})$$

$$\text{Net Delay} = f(R_{\text{net}}, C_{\text{net}} + C_{\text{pin}})$$

3.物理实现

Parasitic RC Model: TLU+ Technology File



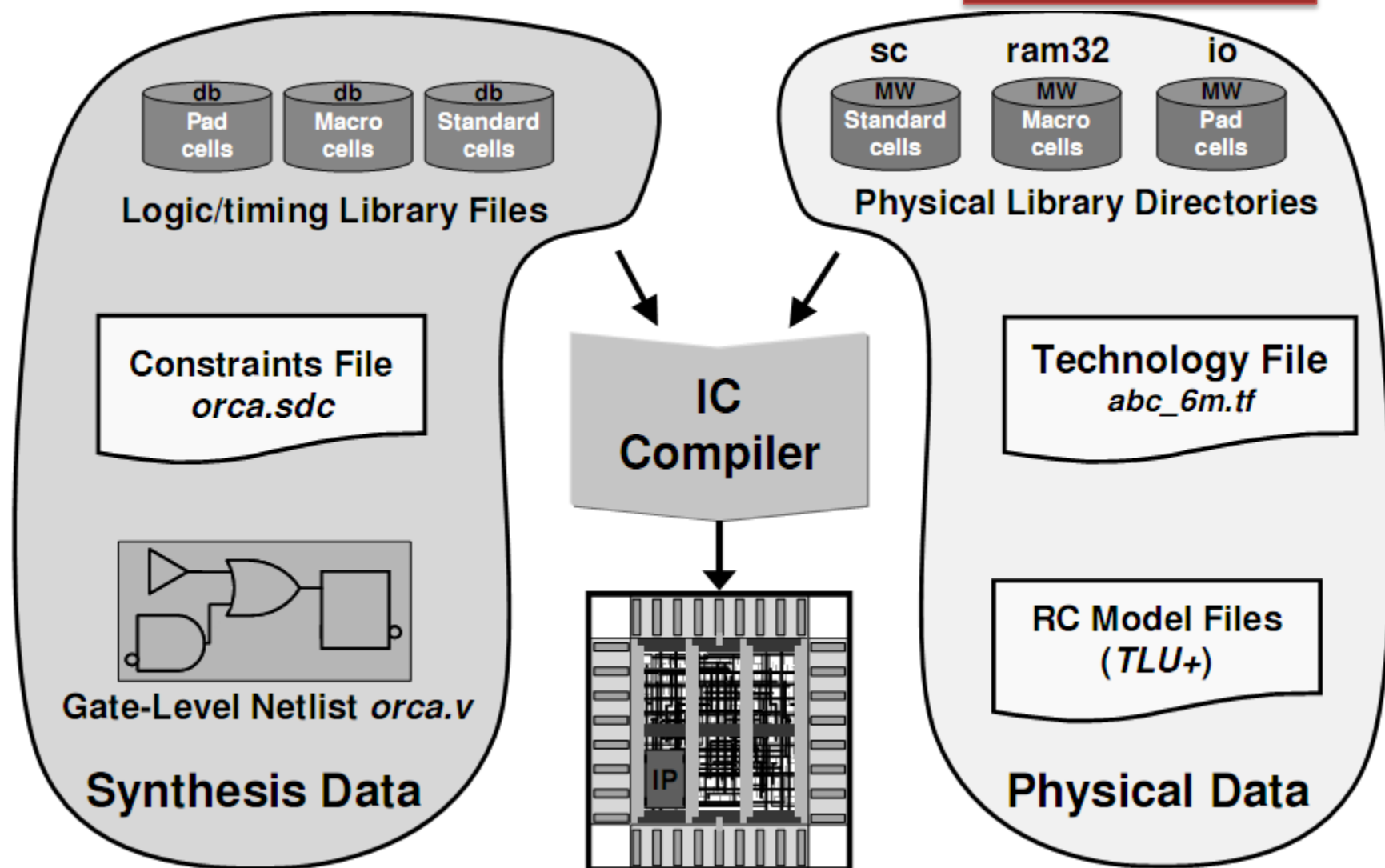
```
set_tlu_plus_files
    -max_tluplus abc_max.tlup
    -min_tluplus abc_min.tlup
    -tech2itf_map abc.map
```

TLU+用于计算wire的寄生参数的，包括电阻，电容，耦合电容等等；

3.物理实现：Design Setup & In

Logic Library & Physical Library & Tech File

stdcel,macro , io

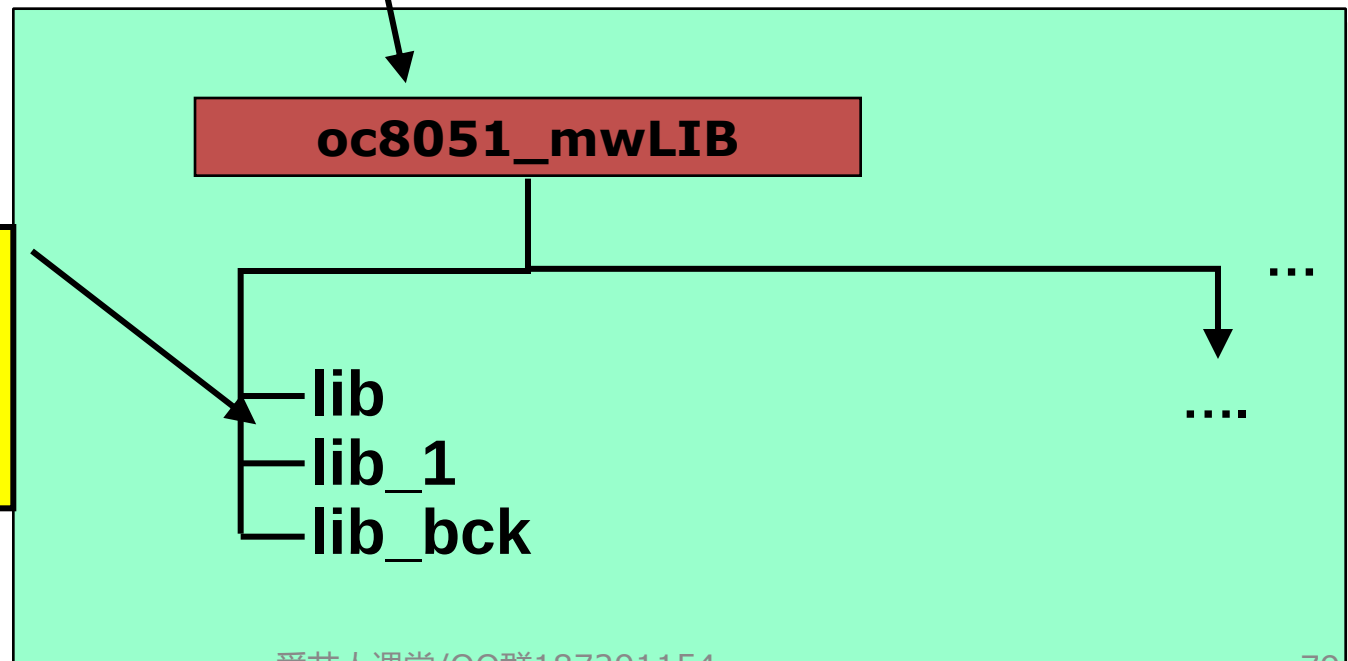


3.物理实现：Design Setup & In

```
create_mw_lib oc8051_mwLIB -open \  
    -technology techfile.tf \  
    -mw_reference_library "mw/sc mw/io mw/ram32"
```

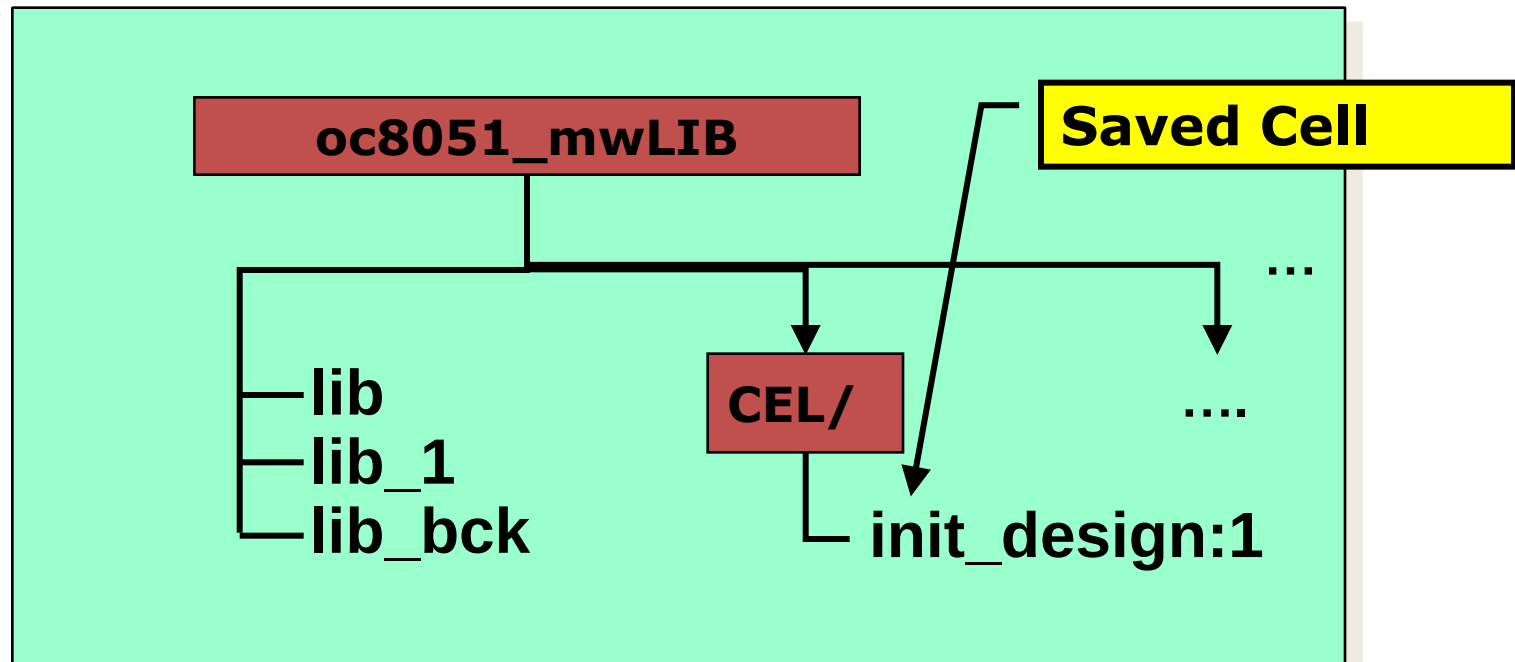
MW Design Library

**Database
T.O.C,
technology
data etc.**

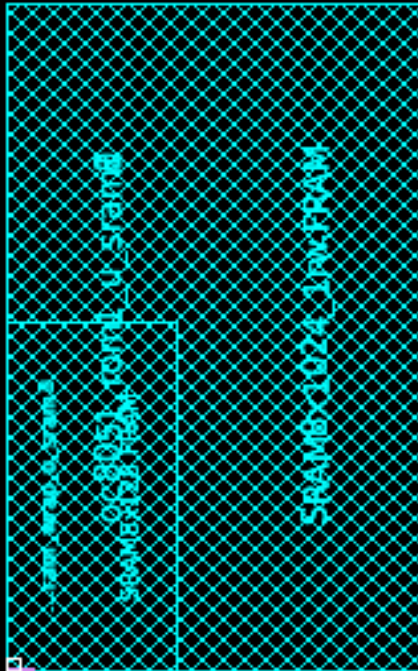


3.物理实现：Design Setup & In

```
read_verilog -top oc8051_top oc8051_top.v  
read_sdc oc8051.sdc  
save_mw_cel -as init_design
```

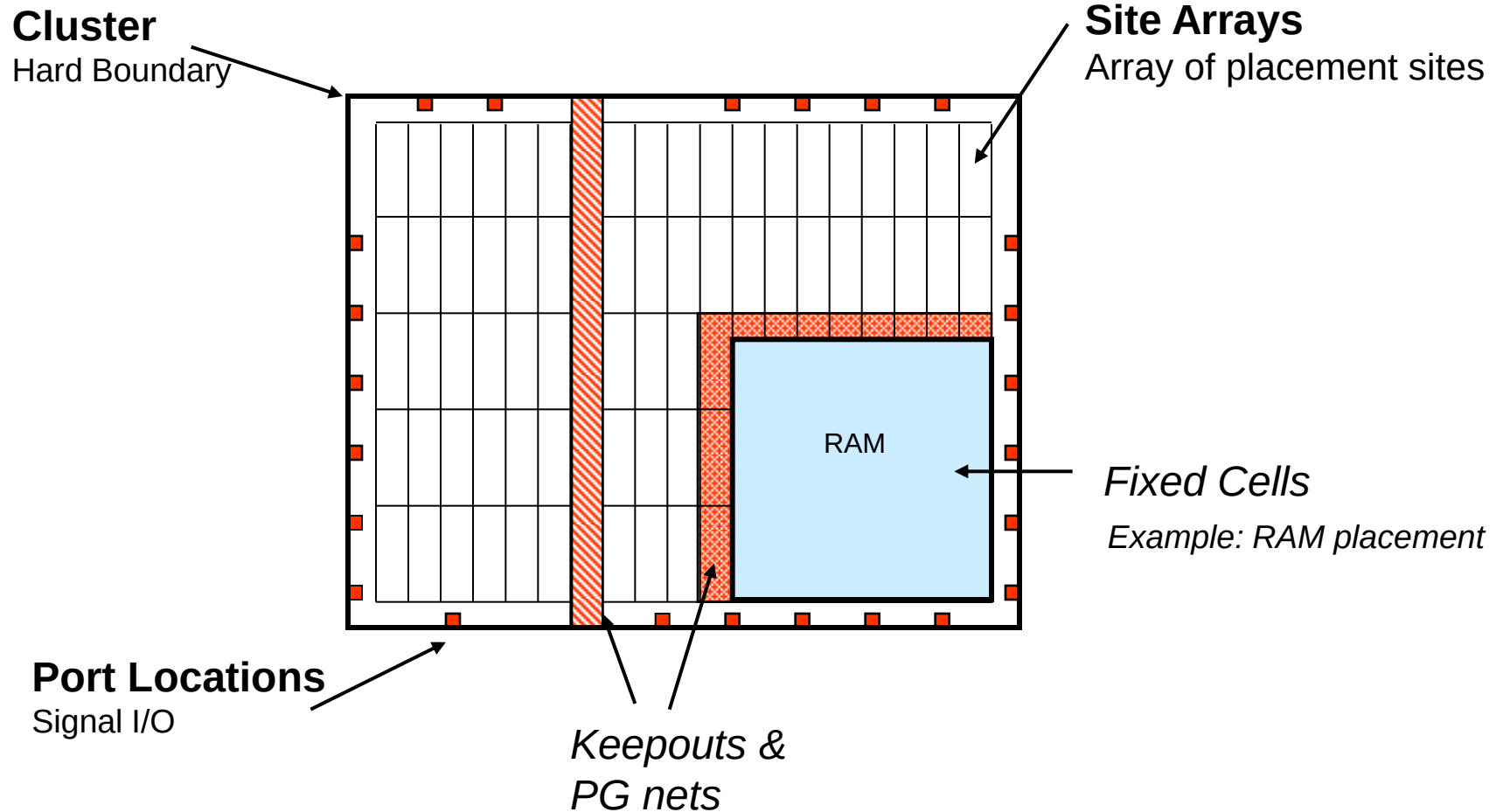


3.物理实现：Design Setup & In



Befor floorplan

3.物理实现:Floorplan

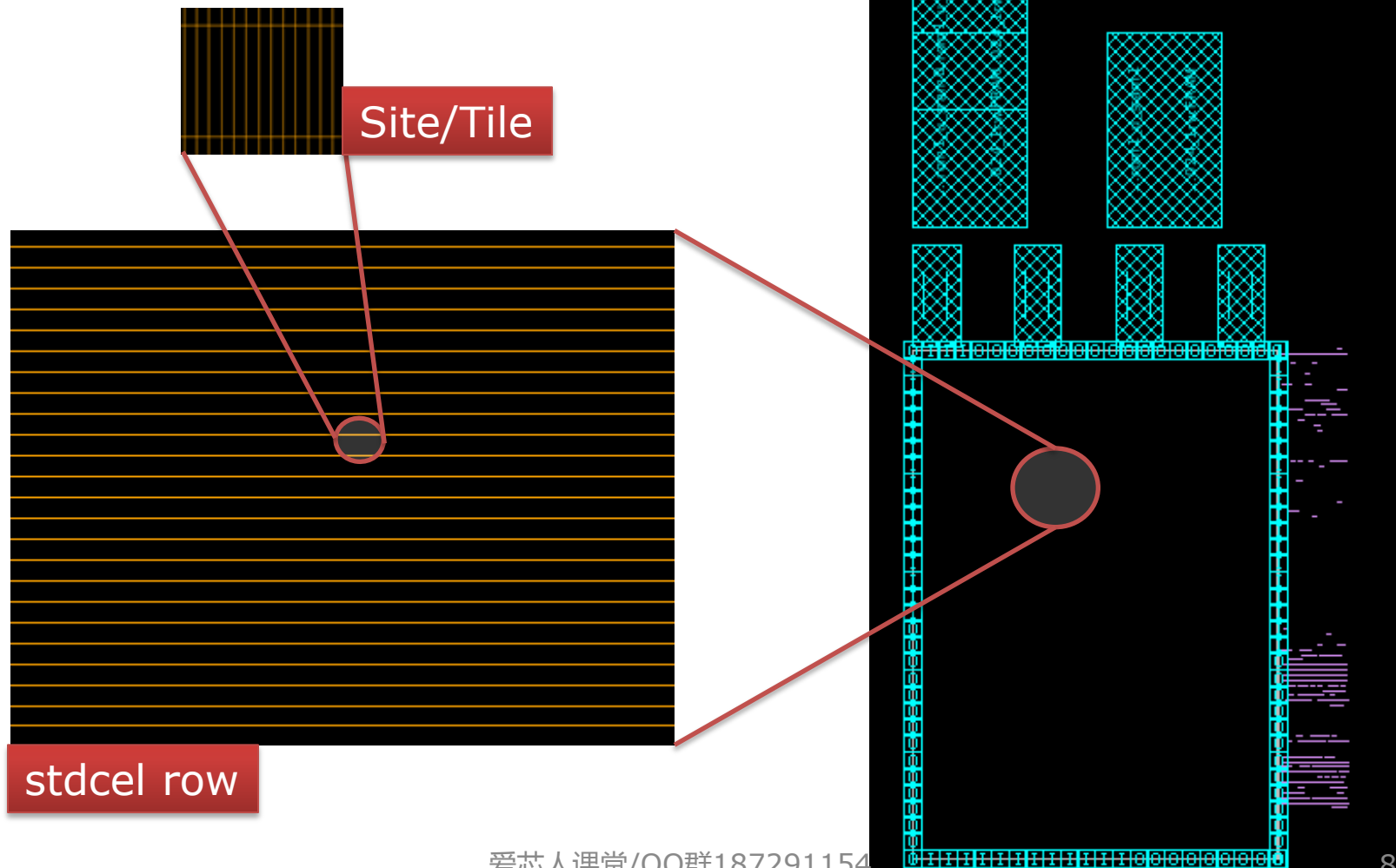


Floorplan的目标

爱芯人课堂/QQ群187291154

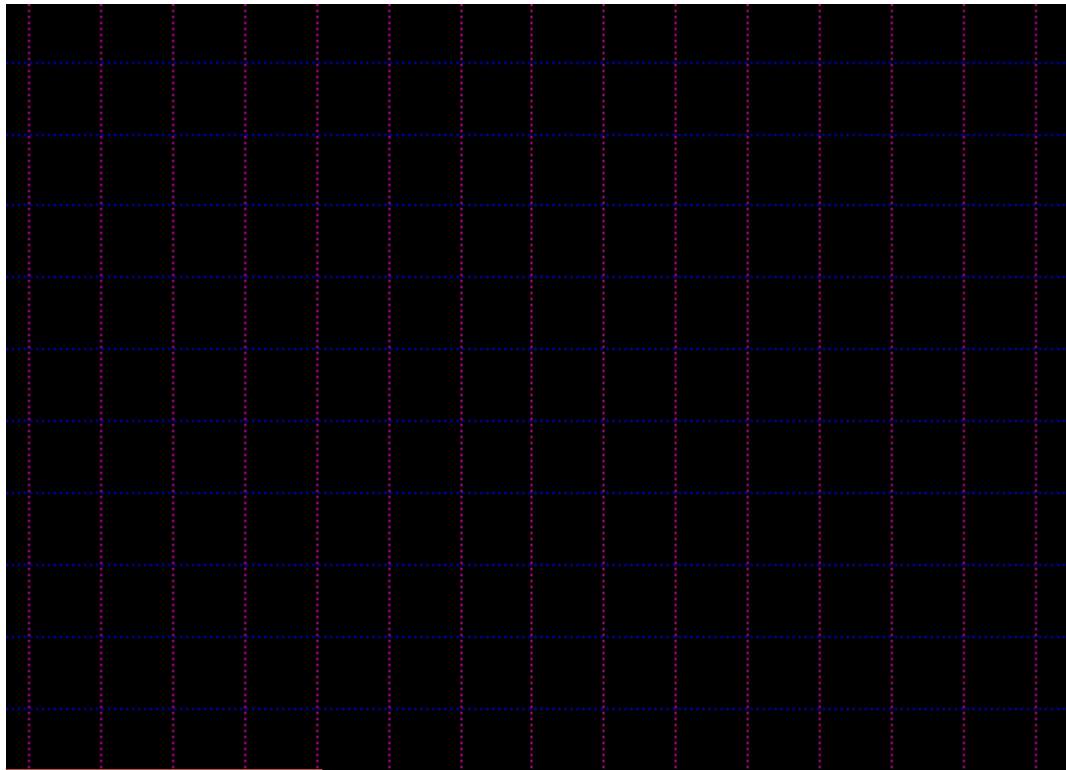
3.物理实现:Floorplan

`initialize_floorplan`

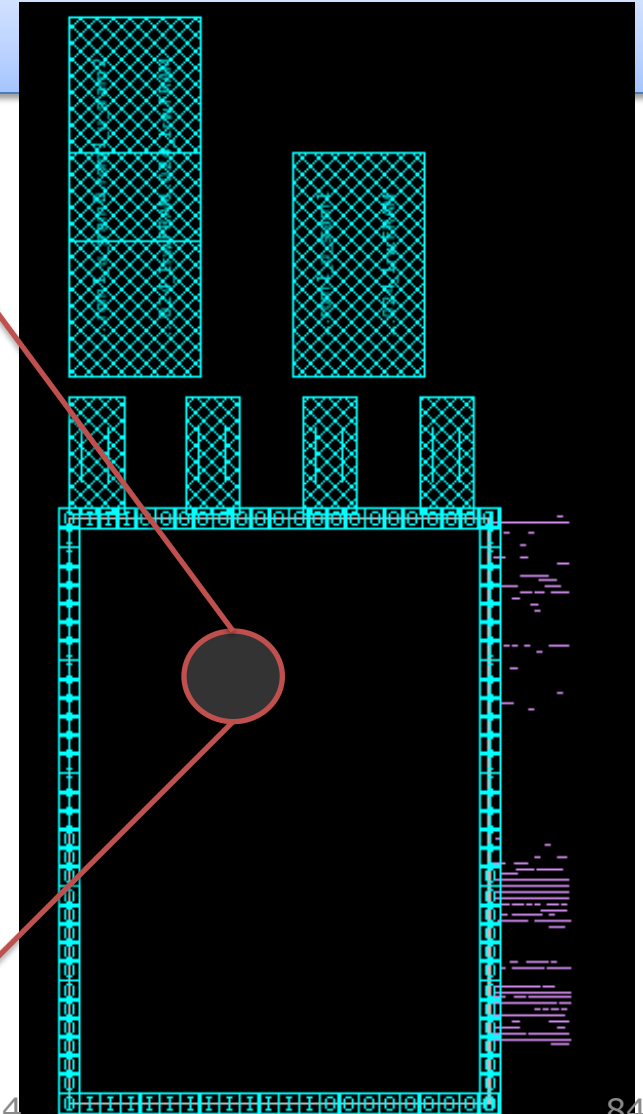


3.物理实现:Floorplan

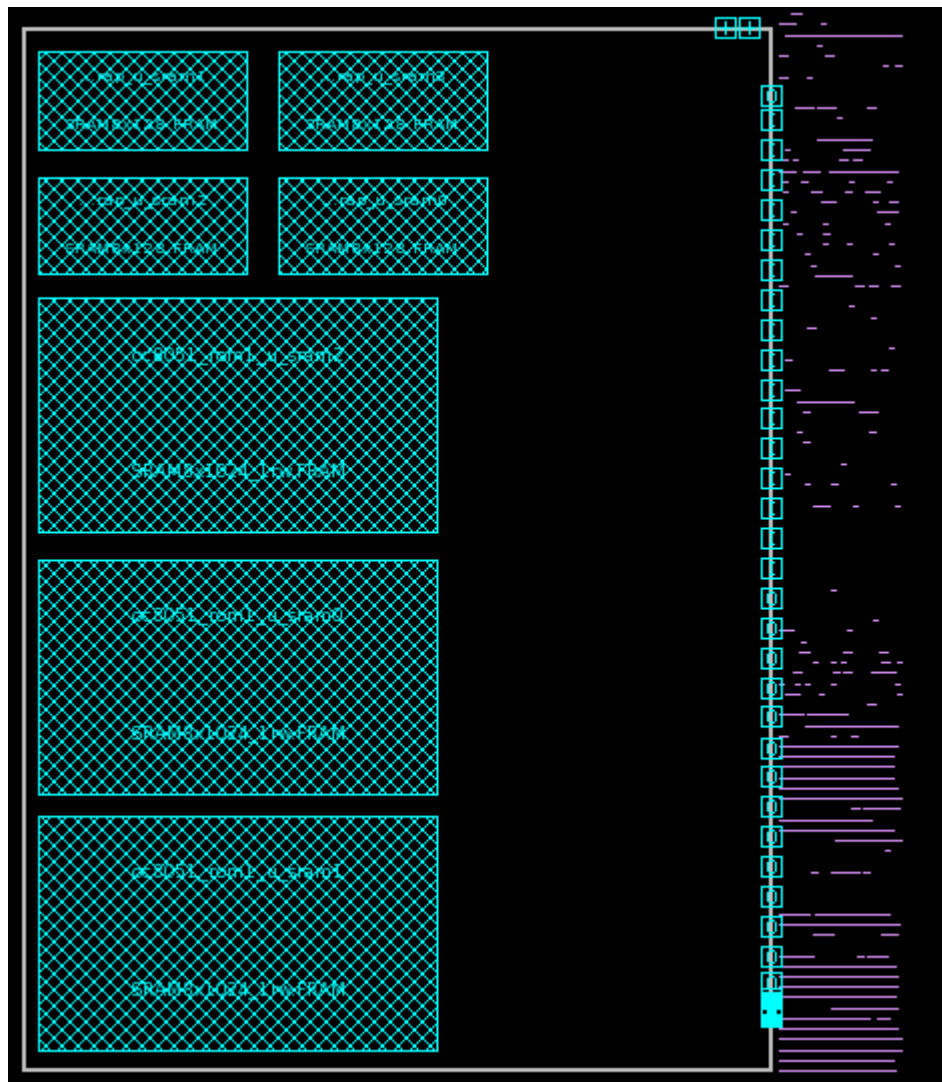
`initialize_floorplan`



Metal Track



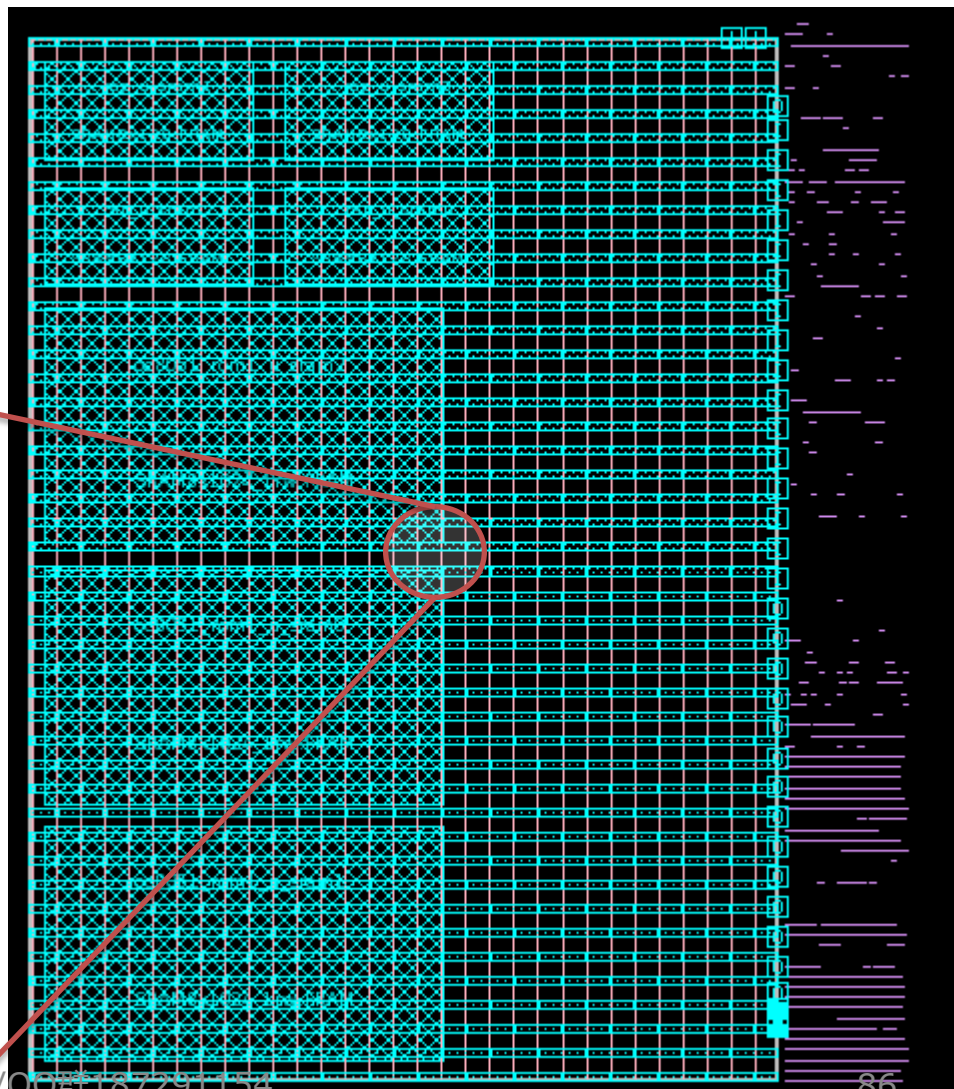
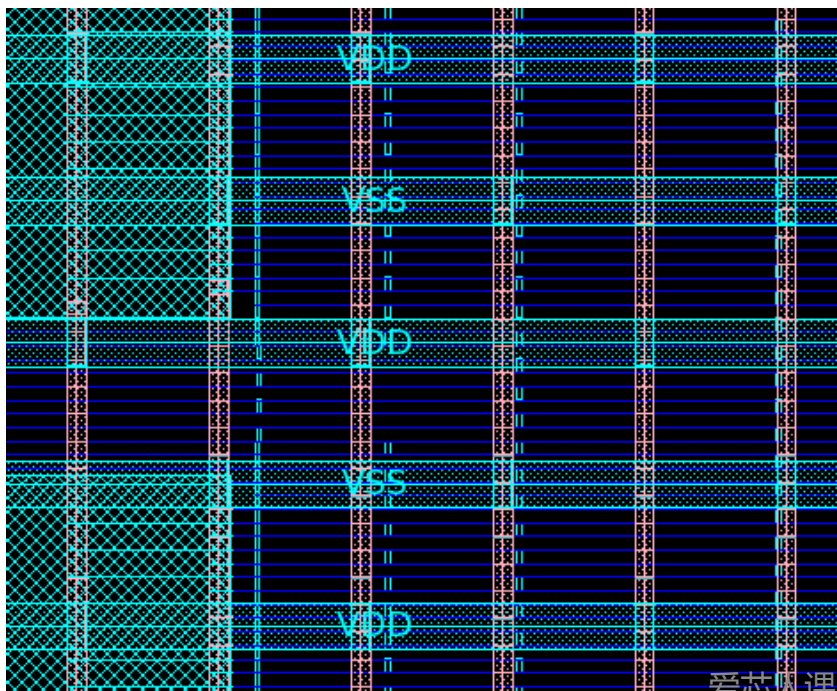
3.物理实现:Floorplan



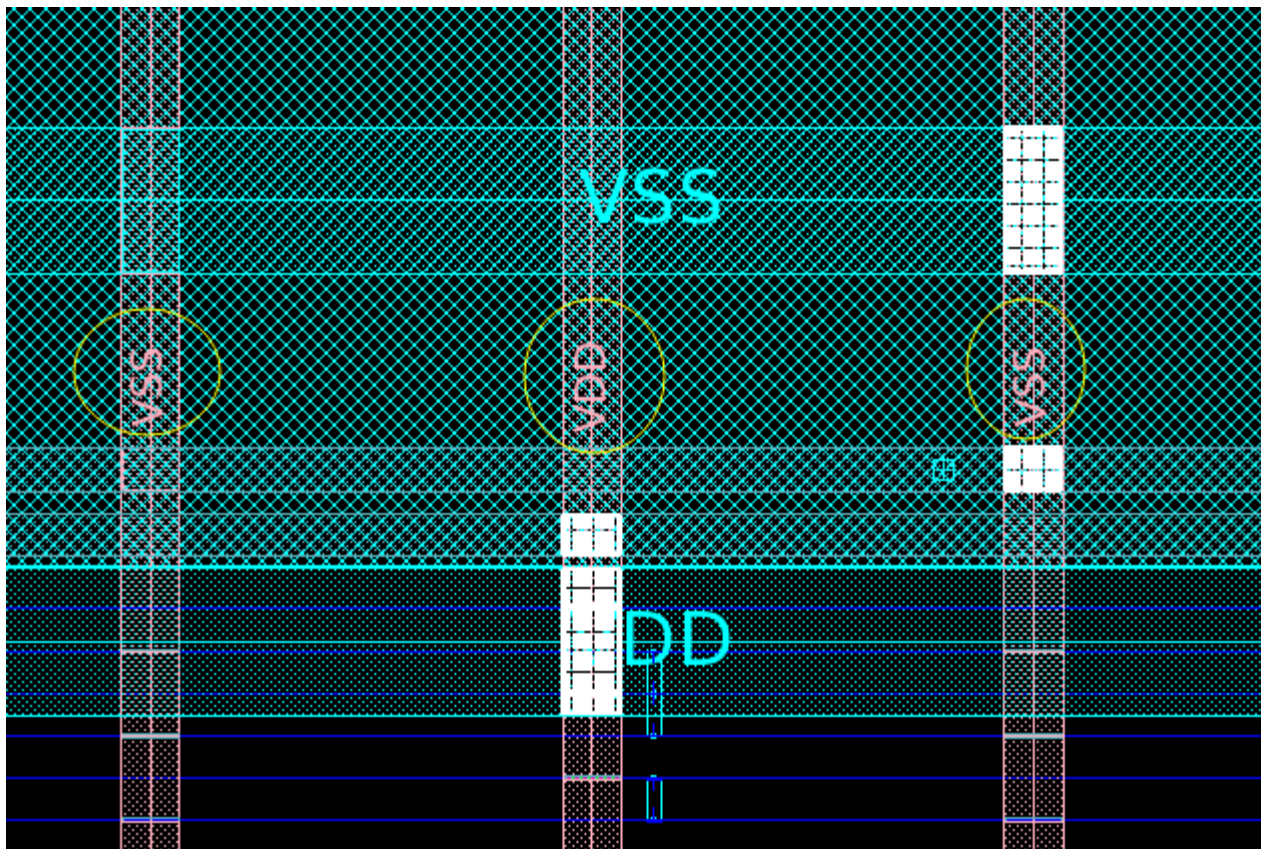
摆放好Macro和port等的位置

3.物理实现:Floorplan

加上Power&Ground的strap

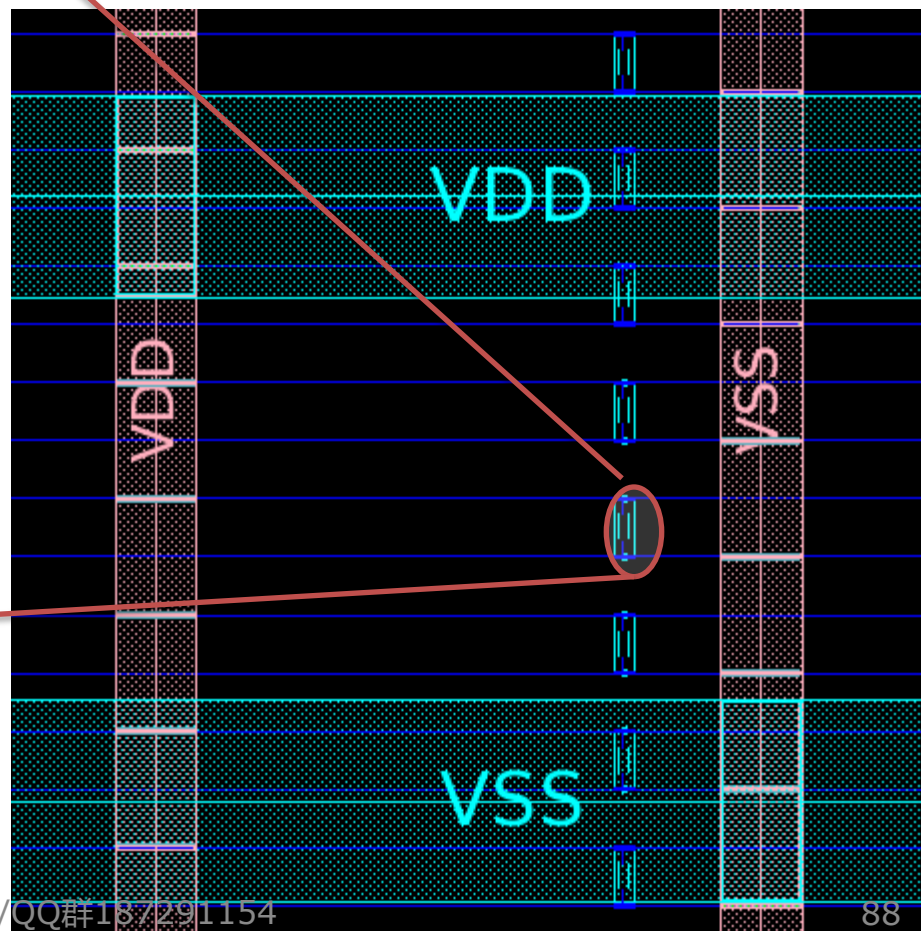
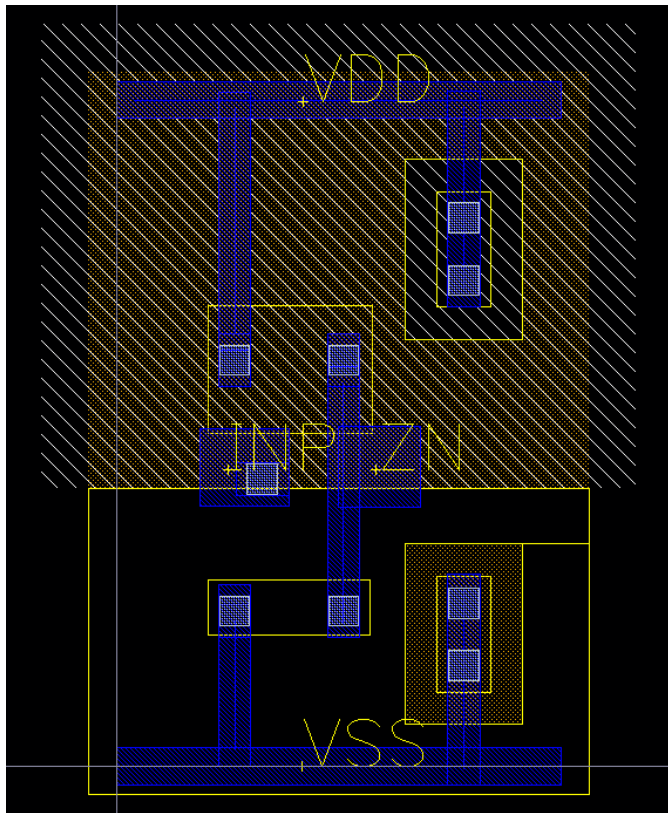


3.物理实现：Floorplan



Macro供电例子

3.物理实现：Floorplan



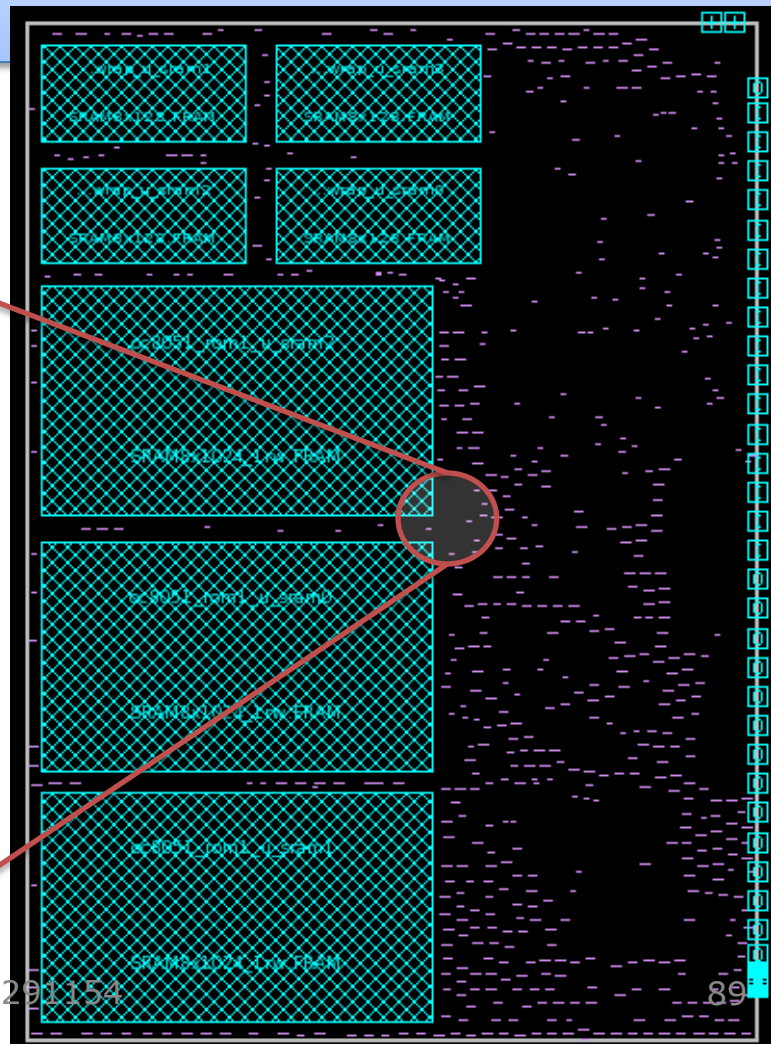
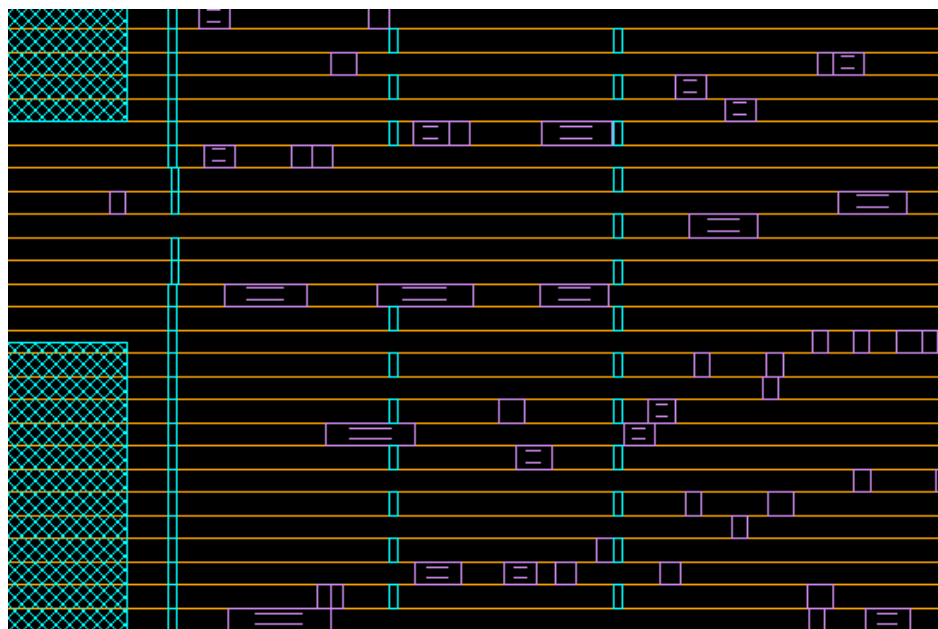
Stdcel供电例子

3.物理实现：Placement

要完成的工作：

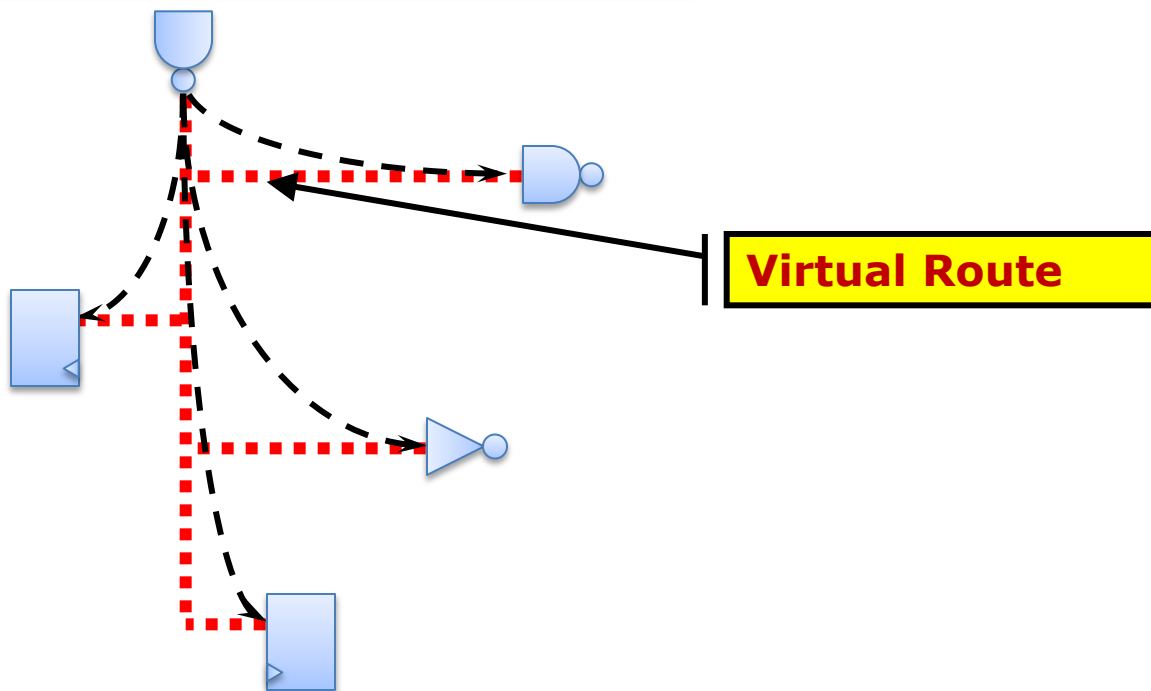
自动摆放所有未摆放好的stdcel；
满足时序（性能）要求的同时，尽量减小面积和功耗；
时序优化,面积优化，功耗优化；

place_opt的结果



3.物理实现：Placement

place阶段尚未进行route，icc如何估算net的r和c呢？



- 1.Route之前，icc做place_opt进行优化时用virtual Route估计连接线的长度和形状，以此根据TLU+模型估计RC参数；；
- 2.事实上，在后面降到的CTS阶段，由于没有route，所以也是通过同样的方法估计的RC参数；

3.物理实现：Placement

需要满足sdc约束下的setup timing，所以完成place后，要分析Timing

```
icc_shell> report_timing
```

Point	Incr	Path	Voltage

clock clk (rise edge)	0.00	0.00	
clock network delay (ideal)	0.00	0.00	
oc8051_rom1_u_sram0/CE (SRAM8x1024_1rw)		0.00	0.00 r 1.08
oc8051_rom1_u_sram0/O[6] (SRAM8x1024_1rw)		7.03	7.03 r 1.08
U1773/Q (AND2X1_HVT)	0.21 *	7.24	r 1.08
... ..			
icc_place9/Z (DELLN2X2_HVT)		1.97 *	18.45 f 1.08
oc8051_ram_top1_oc8051_ram1_u_ram_wrap_u_sram1/A1[3] (SRAM8x128)	0.00 *	18.45	f 1.08
data arrival time		18.45	
clock clk (rise edge)	20.00	20.00	
clock network delay (ideal)	0.00	20.00	
clock uncertainty	-1.00	19.00	
oc8051_ram_top1_oc8051_ram1_u_ram_wrap_u_sram1/CE1 (SRAM8x128)	0.00	19.00	r
library setup time	0.18	19.18	
data required time		19.18	

data required time		19.18	
data arrival time		-18.45	

slack (MET)		0.73	

3.物理实现：Clock Tree Synthesis / CTS

要完成的工作：

1. 在此之前，从synthesis开始到placement，Clock都被假设为理想的，从clock port到reg/CK的延时都为0；我们需要构建一棵Clock Tree，把Clock信号从port送到reg/CK；

2. Clock Tree长好之后还需要对timing进行优化（这个阶段的优化叫做PostCTS Opt），包括setup和hold：

第一，place_opt是基于理想时钟而对设计进行，长完Clock Tree之后的时钟和理想时钟存在区别；

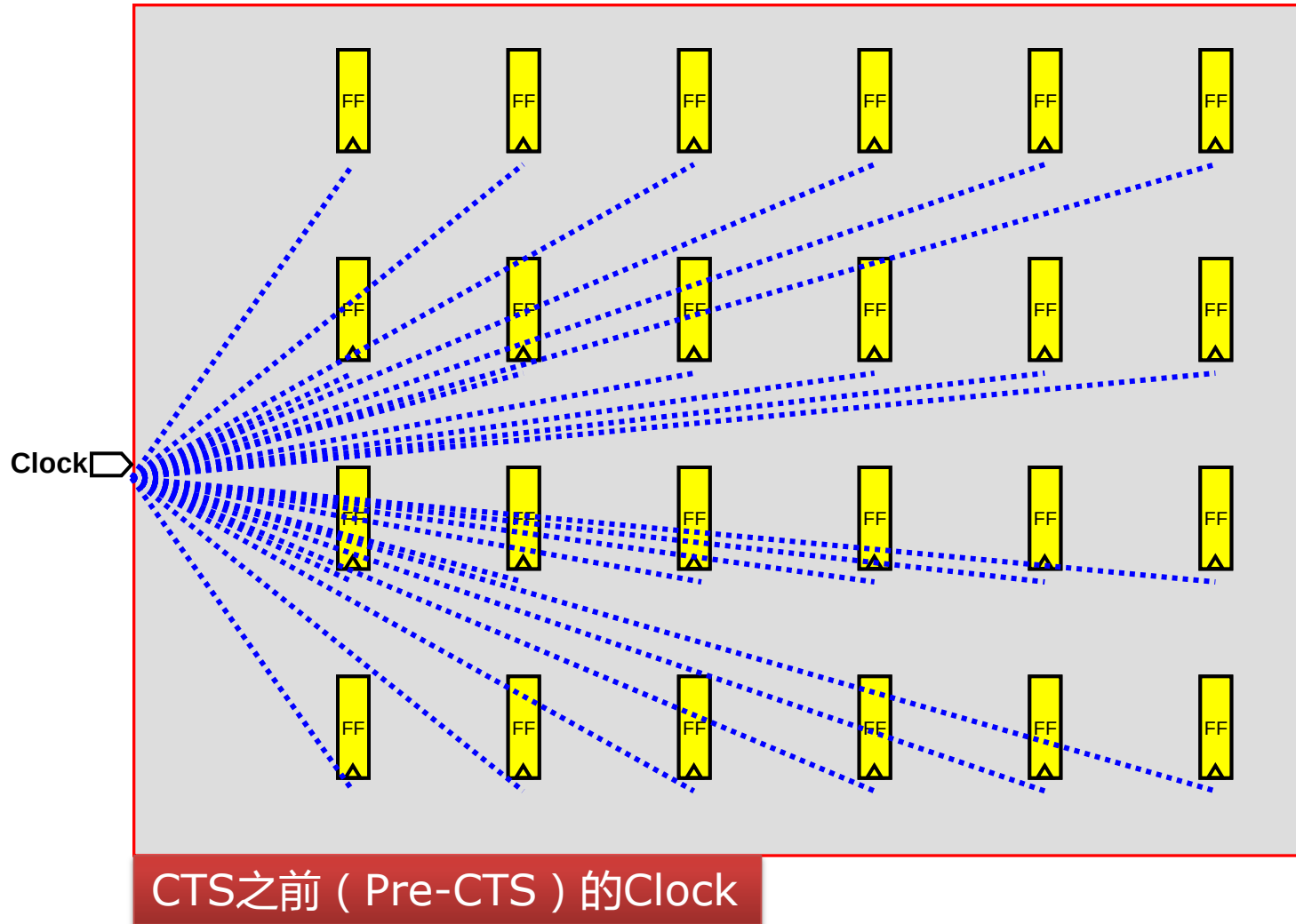
第二，CTS的过程中工具会挪动原来的单元，导致delay有变化；

第三，需要修Hold timing，因为在这之前一直没理会Hold（why？）；

3. 要完成对Clock Net的绕线（route）；

3.物理实现：Clock Tree Synthesis / CTS

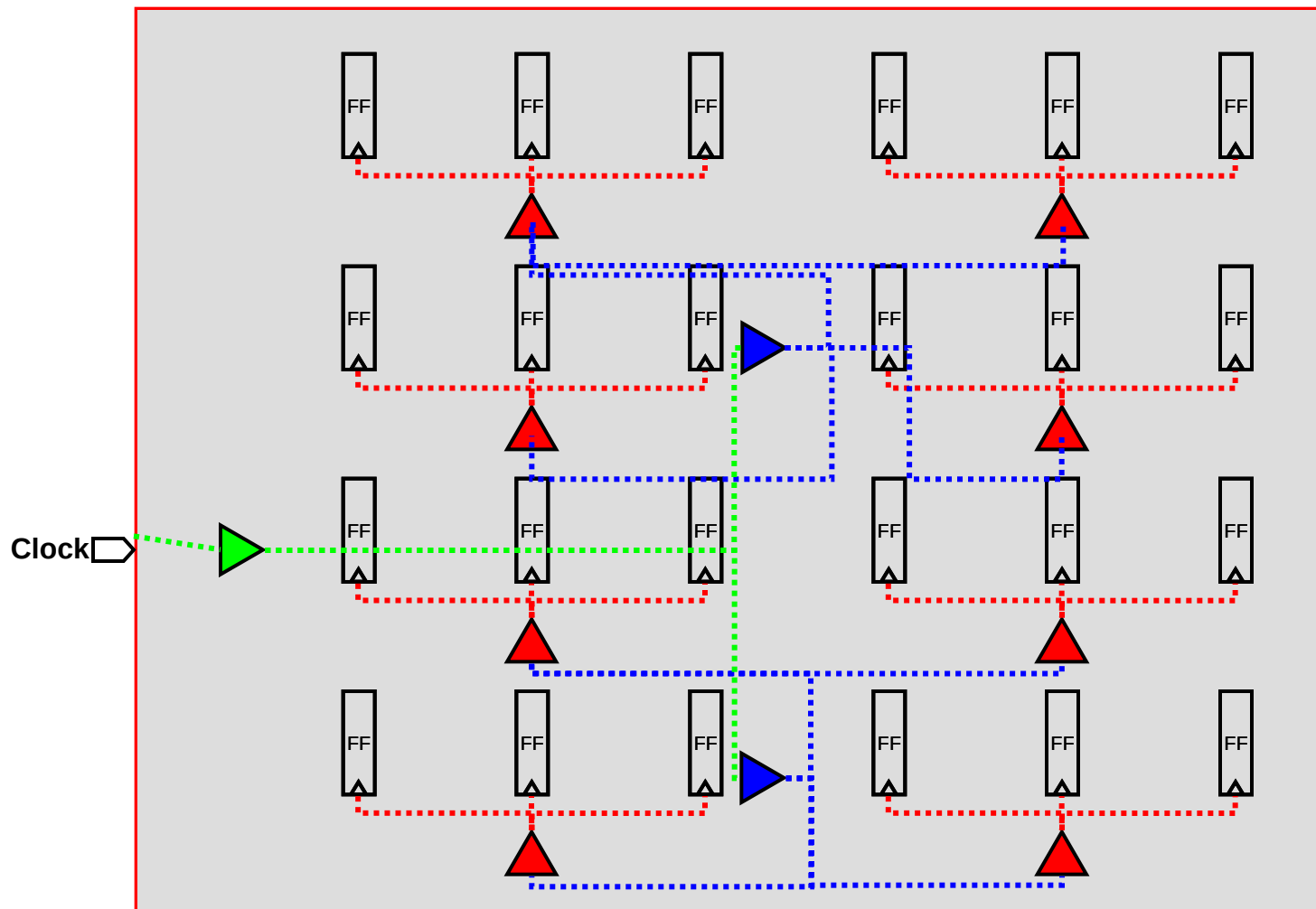
构造Clock Tree：



3.物理实现：Clock Tree Synthesis / CTS

构造Clock Tree：

要分析构建的Tree的质量，这是一个不断迭代的过程；



CTS之后 (Post-CTS) 的Clock

3.物理实现：Clock Tree Synthesis / CTS

优化Setup&Hold Timing：修掉新出现的violation

. Clock Tree长好之后还需要对timing进行优化（这个阶段的优化叫做PostCTS Opt），包括setup和hold：

第一，place_opt是基于理想时钟而对设计进行，长完Clock Tree之后的时钟和理想时钟存在区别；

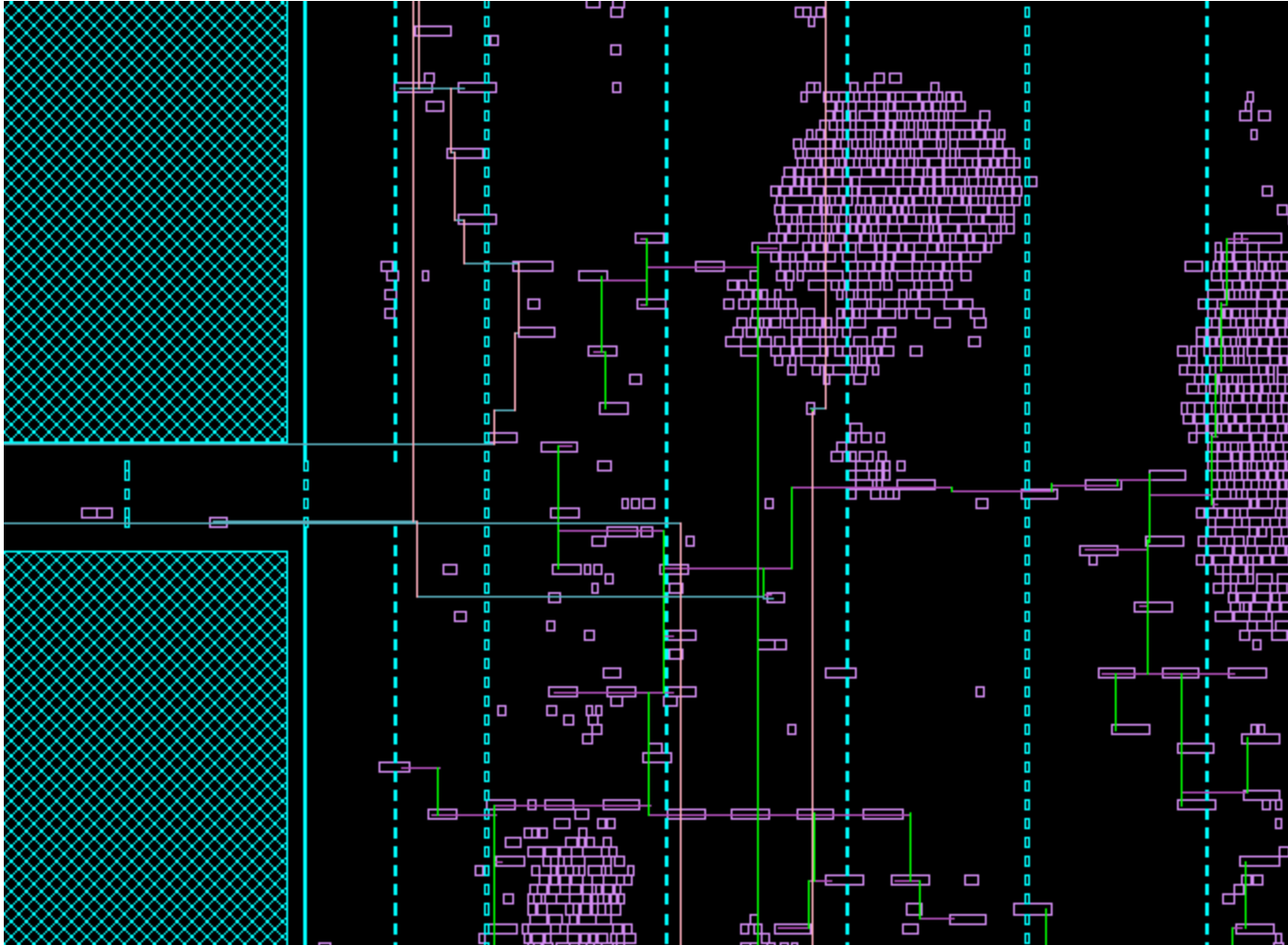
第二，CTS的过程中工具会挪动原来的单元，导致delay有变化；

第三，需要修Hold timing，因为在这之前一直没理会Hold（why？）；

PostCTS的优化之后，也需要分析timing是否满足

3.物理实现：Clock Tree Synthesis / CTS

要完成对Clock Net的绕线 (route)：因为clock net比普通的data path上的net重要



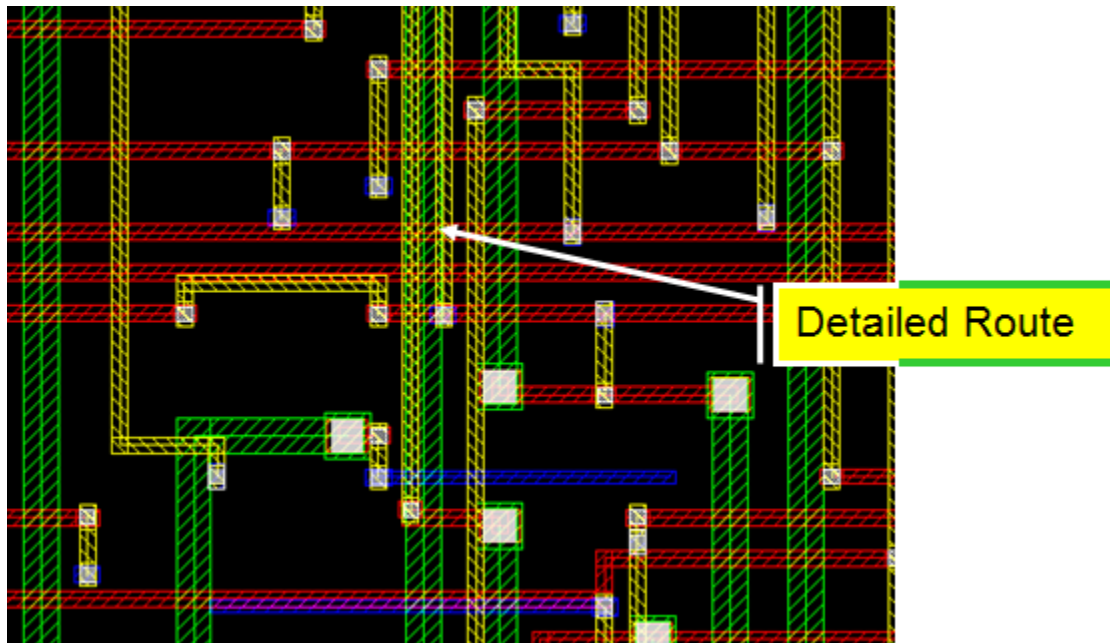
3.物理实现：Routing

Initial Route：对所有未绕线的net进行绕线



3.物理实现：Routing

绕线之后 (PostRoute) 进行Timing优化：包括setup和hold，及power和Area优化



Notes :

- 1.此时所有net都已经完成route，这时叫detailed route（之前的阶段是Virtual Route），所以此时的RC信息和之前估算的会有些区别，delay会有变化；
2. PostRoute优化之后也需要分析是否满足：

3.物理实现：Chip Finish

为提高良率（ yield ）和解决物理规则违规做的工作：

Wire Spreading

Redundant Via Insertion

Filler Cell Insertion

Metal Fill Insertion

Metal Slotting

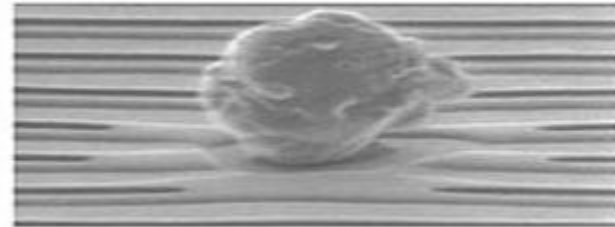
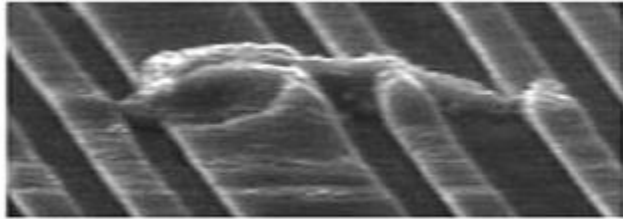
Notes：

1.这一步也通常被称为DFM（Design For Manufacturing）；

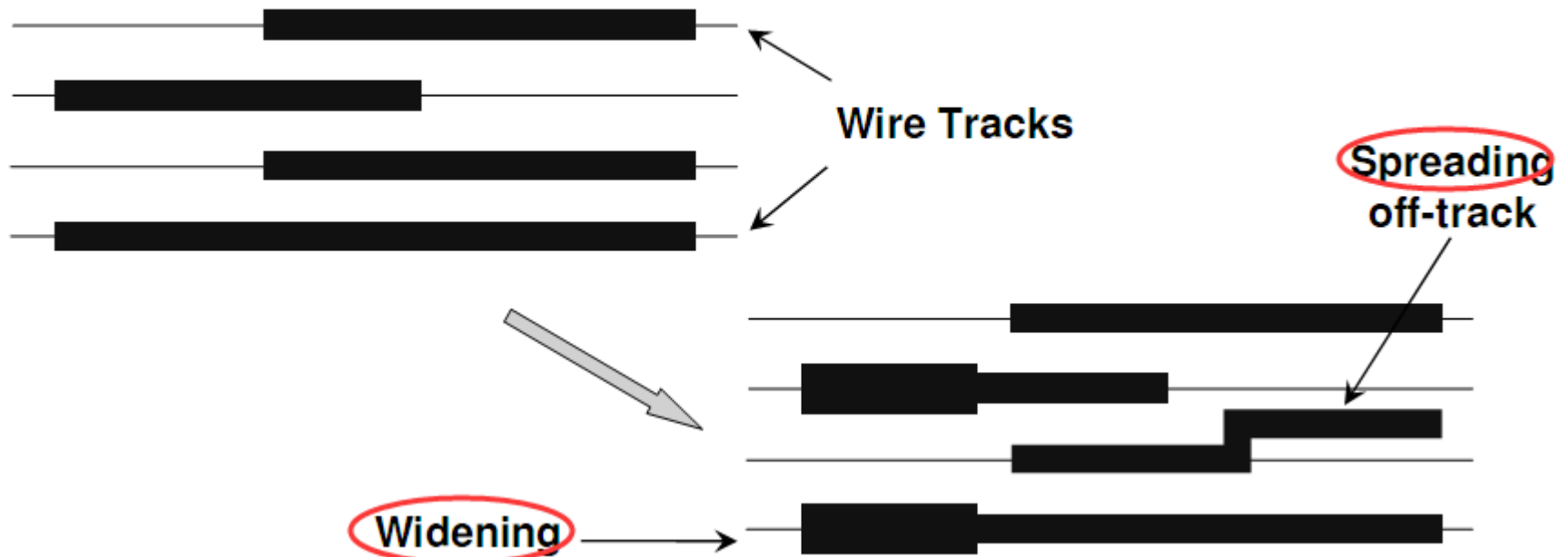
2.良率（Yield）：因为有些芯片在wafer上制造出来有缺陷，有的芯片能正常工作，有些芯片工作不正常，那么正常的芯片占总芯片的比例就是良率；

3.物理实现：Chip Finish

Wire Spreading：提高芯片良率

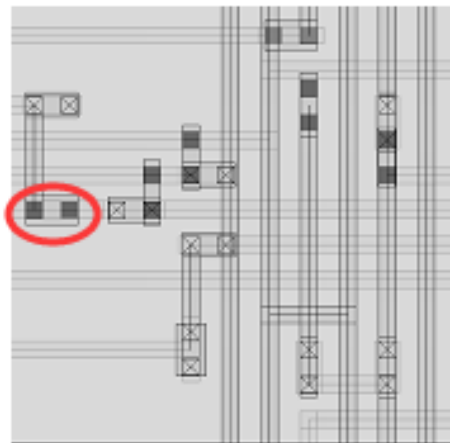
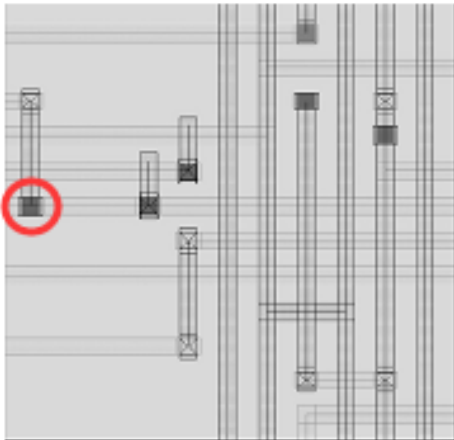
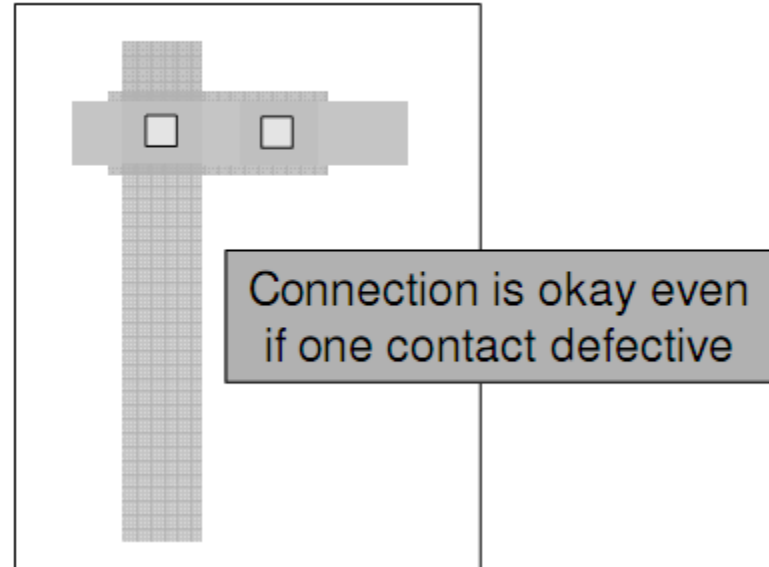
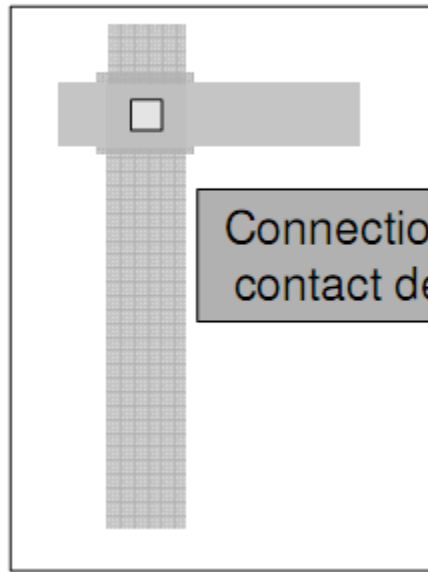


■ Widen wires to reduce *open* critical area



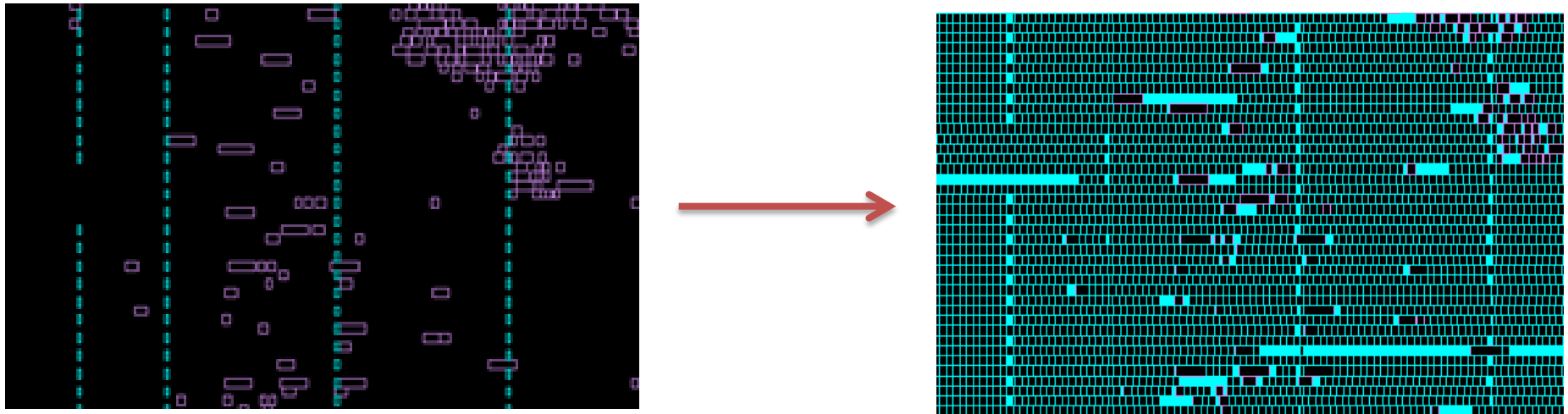
3.物理实现：Chip Finish

Redundant Via Insertion：提高芯片良率



3.物理实现：Chip Finish

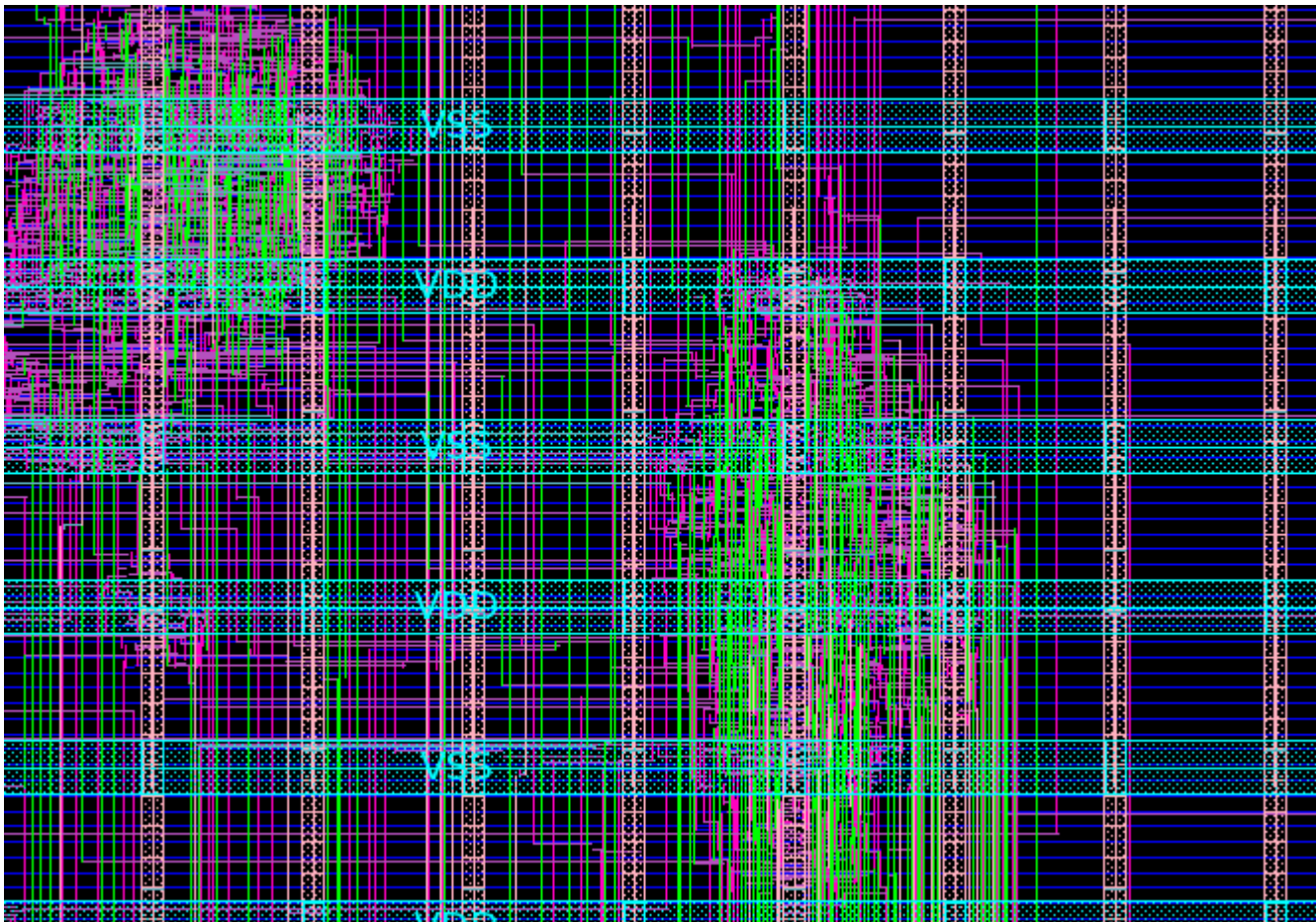
Filler Cell：为了满足物理设计规则



填满未摆放cell的row/site, 构建连续完整的NWell/PWell

3.物理实现：Chip Finish

Metal Fil：为了满足物理设计规则

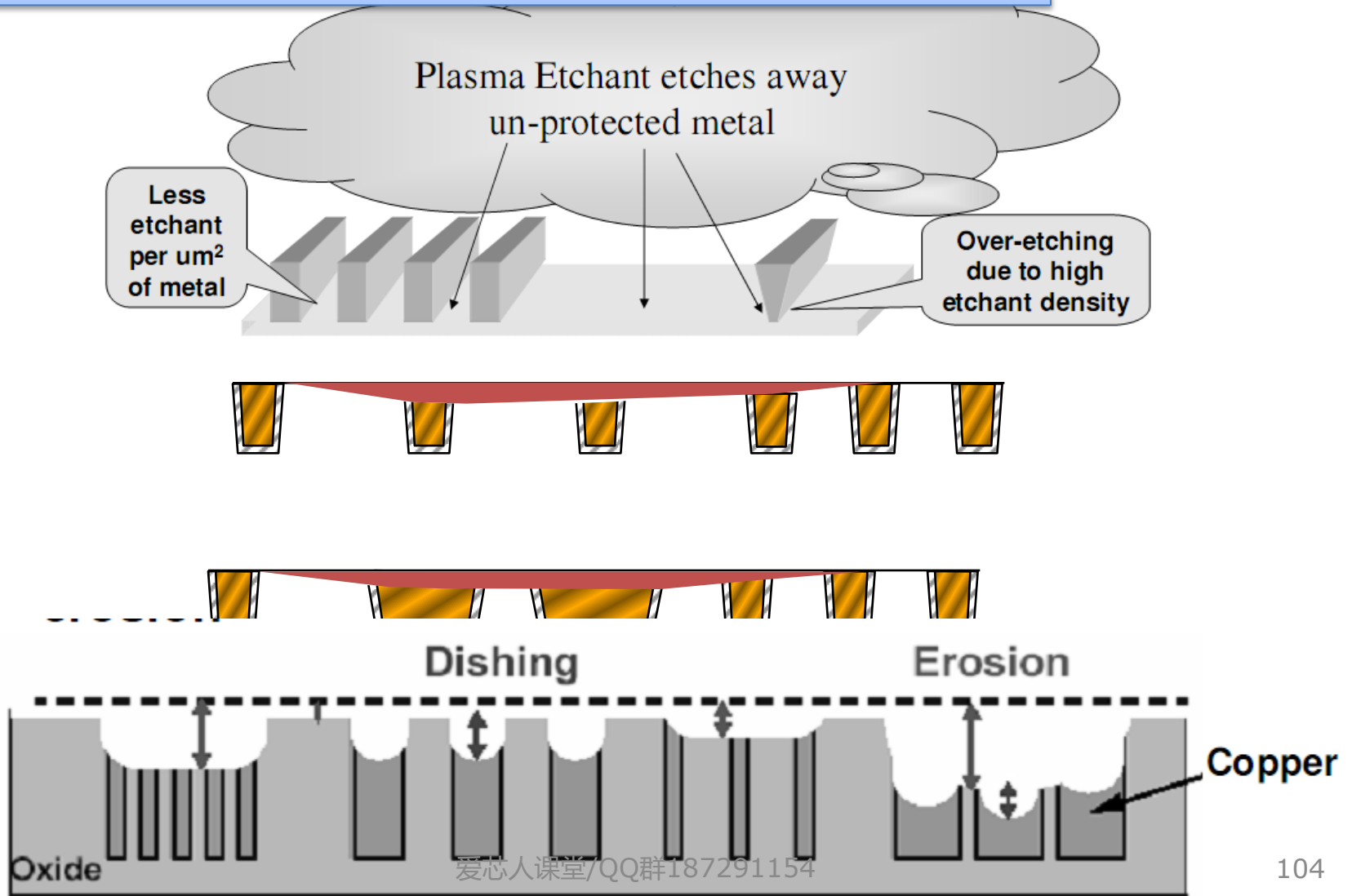


各处Metal的密度不一样，在制造的过程中容易出现下面的问题：

3.物理实现：Chip Finish

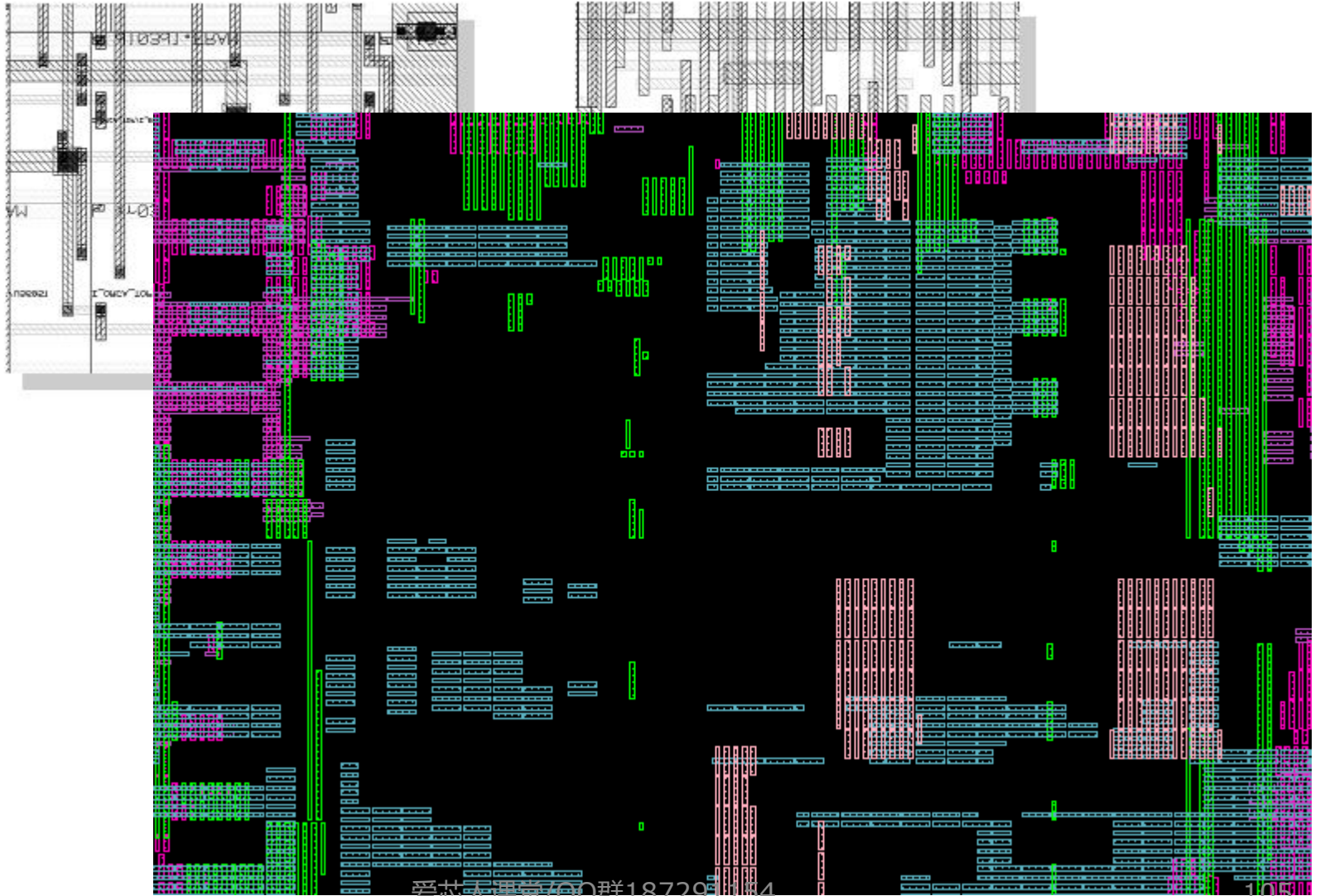
Metal Fil：为了满足物理设计规则

各处Metal的密度不一样，在制造的过程中容易出现下面的问题：



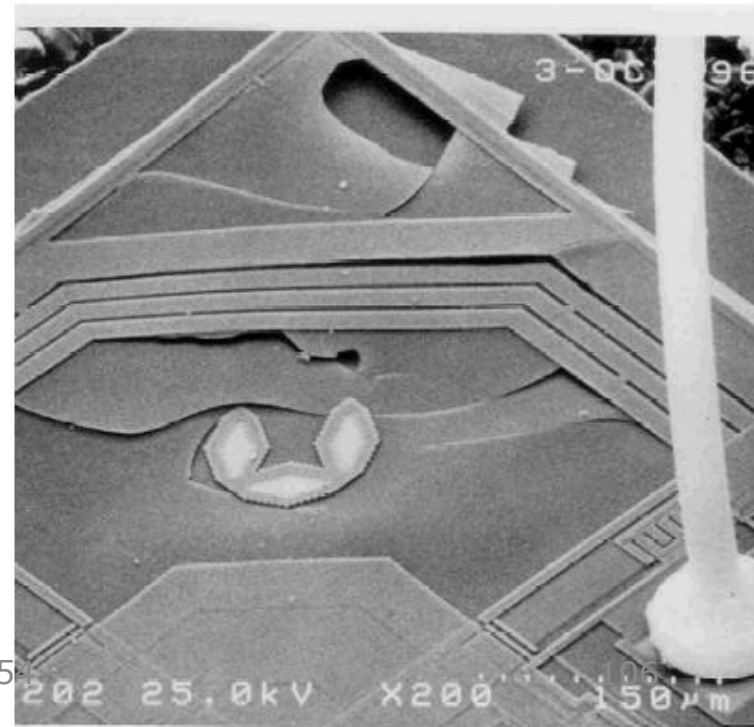
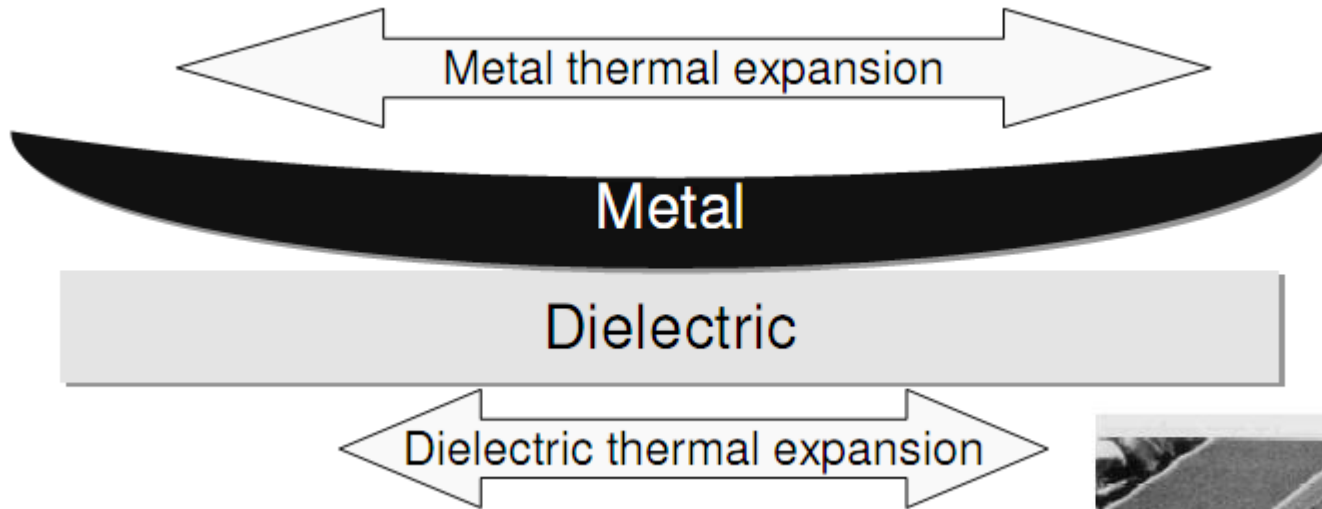
3.物理实现：Chip Finish

Metal Fil：为了满足物理设计规则



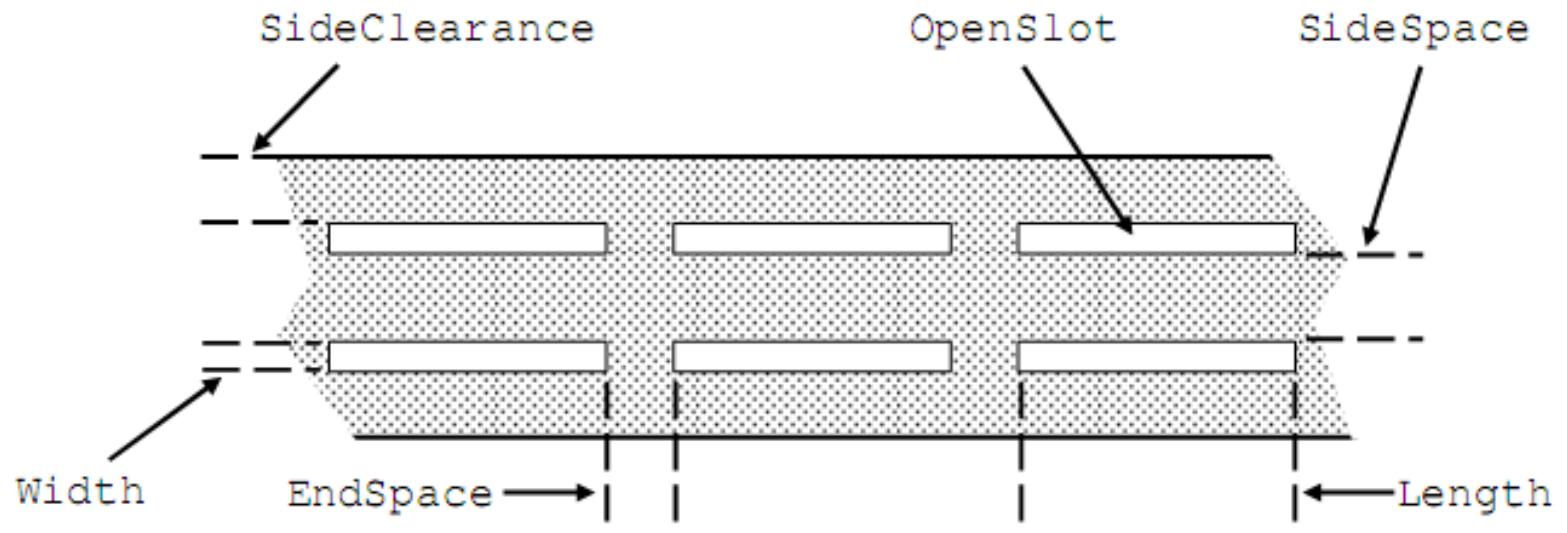
3.物理实现：Chip Finish

Metal Slot：提高良率



3.物理实现：Chip Finish

Metal Slot：提高良率



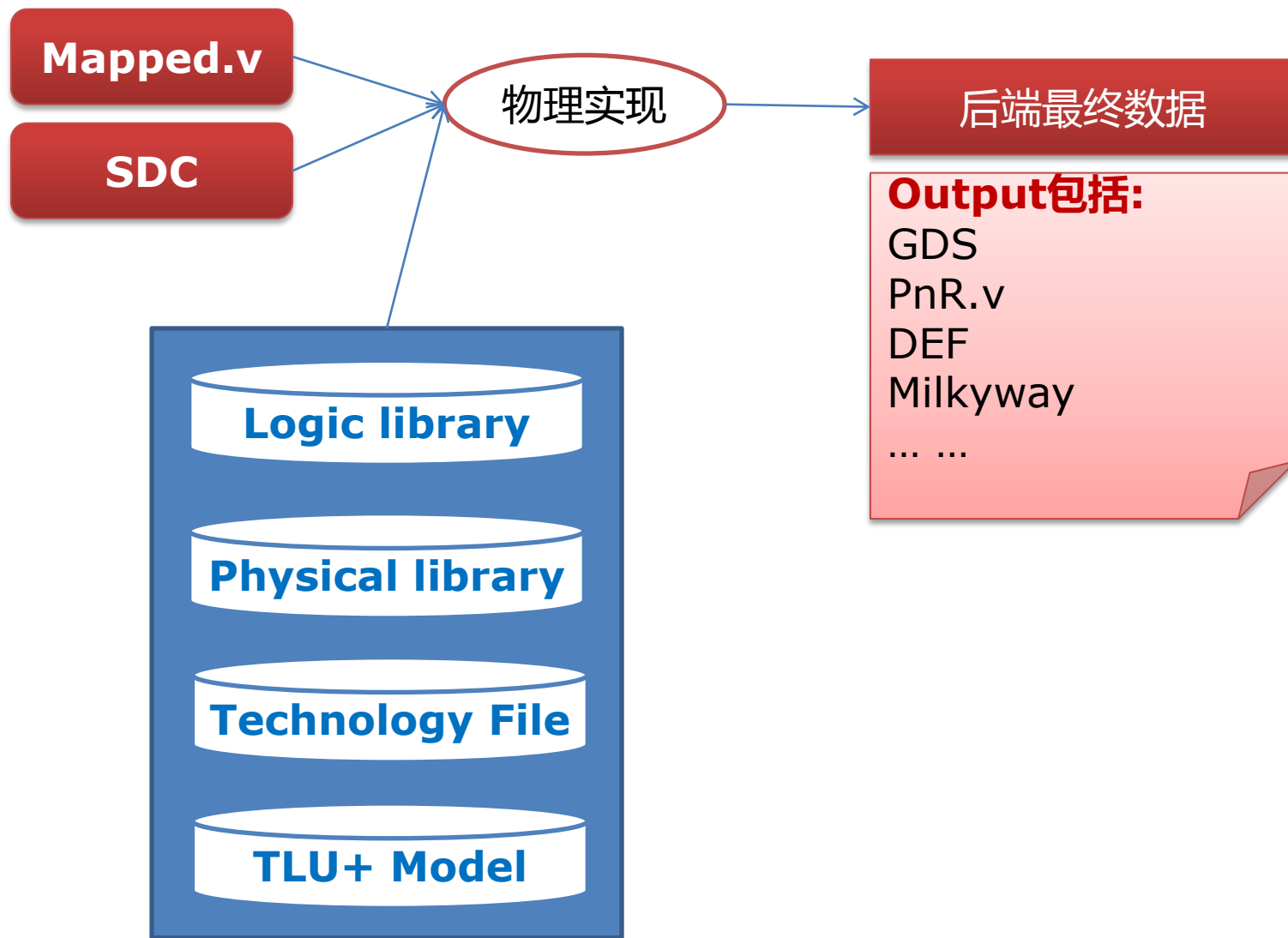
3.物理实现：Write Data Out

```
icc_shell> ls ../output_data/v0/  
.  
..  
oc8051_top.func_lt_cmin.output.sdc  
oc8051_top.func_ml_cmin.output.sdc  
oc8051_top.func_tt_typ.output.sdc  
oc8051_top.func_wc_cmax.output.sdc  
oc8051_top.func_wcl_cmax.output.sdc  
oc8051_top.gds  
icc_shell> █
```

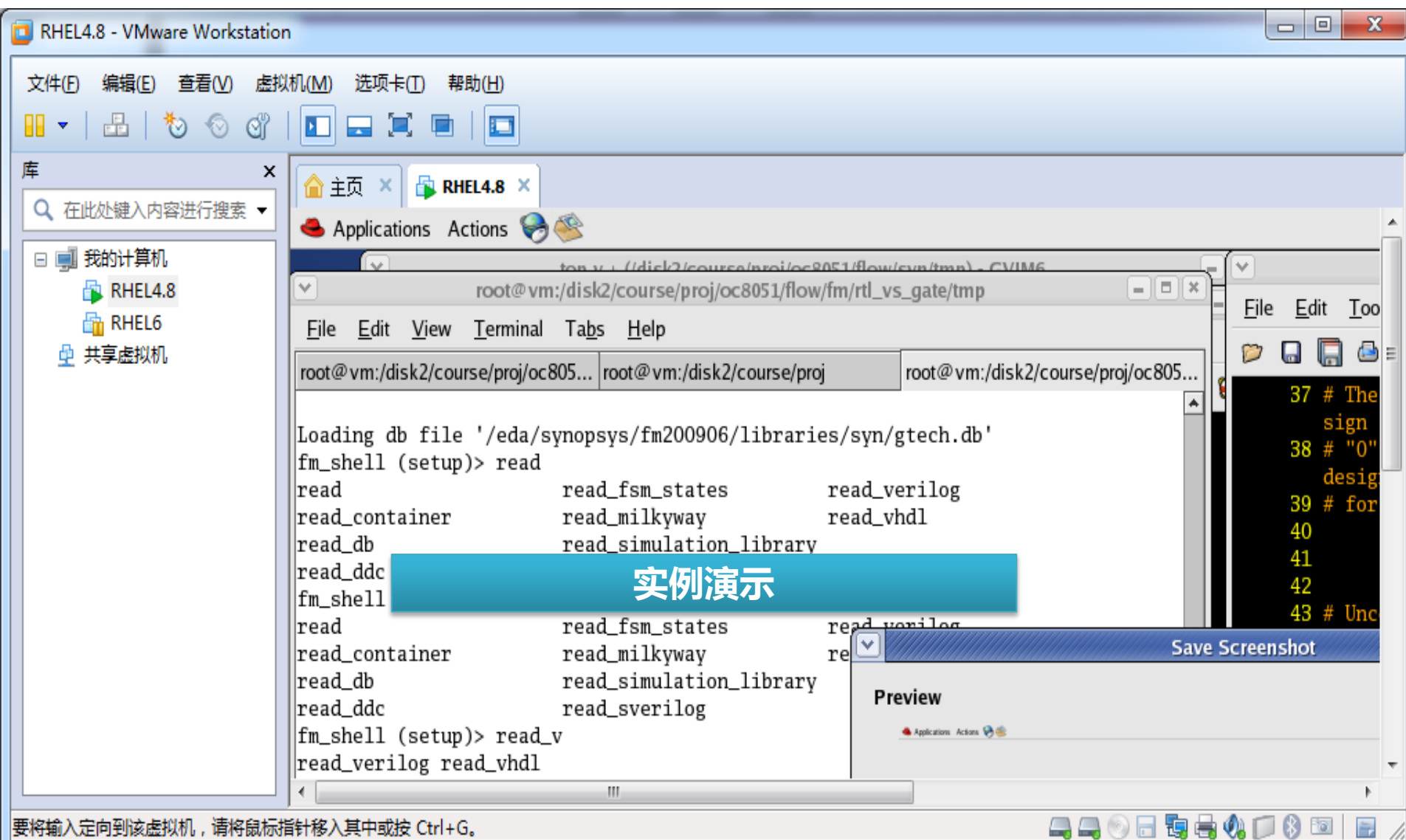
```
oc8051_top.gds.rename.log  
oc8051_top.output.dc.v  
oc8051_top.output.def  
oc8051_top.output.filler.def  
oc8051_top.output.pg.lvs.filler.v  
oc8051_top_LIB  
spef
```

PnR.v

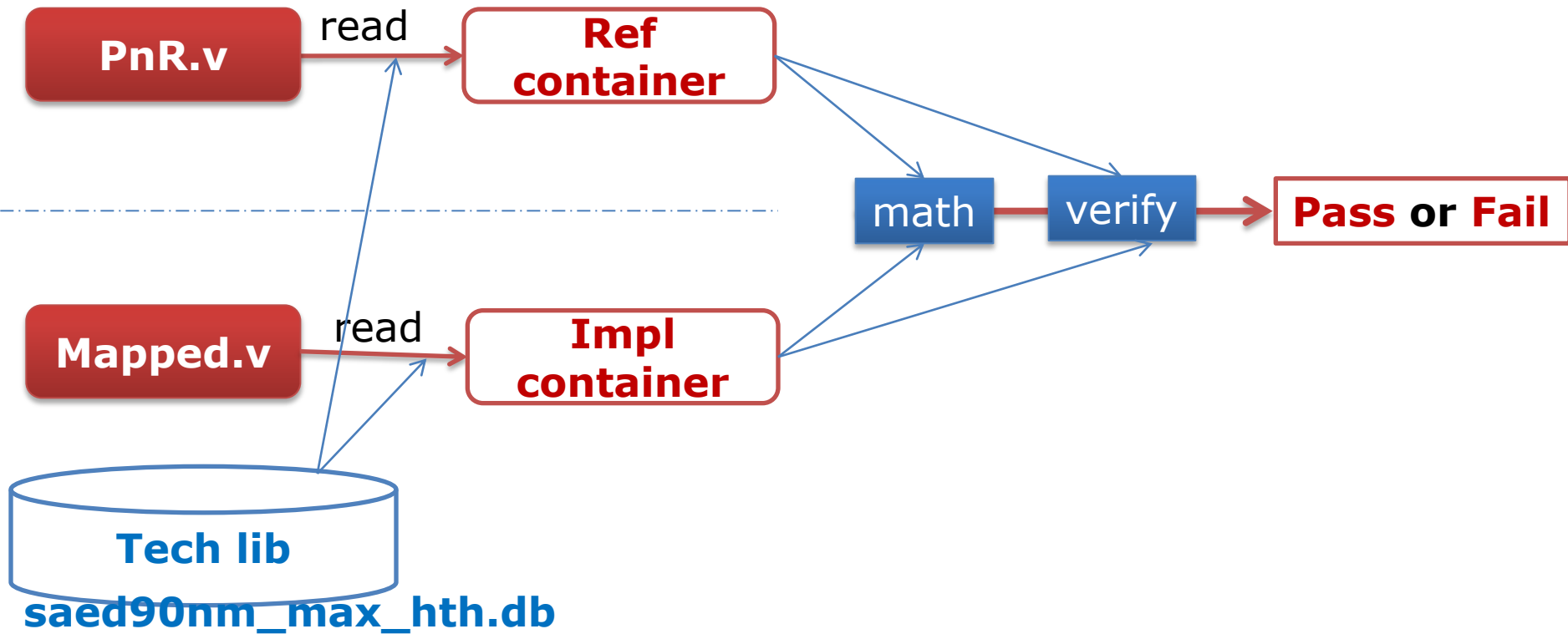
3.物理实现：从Mapped.v 到 GDS



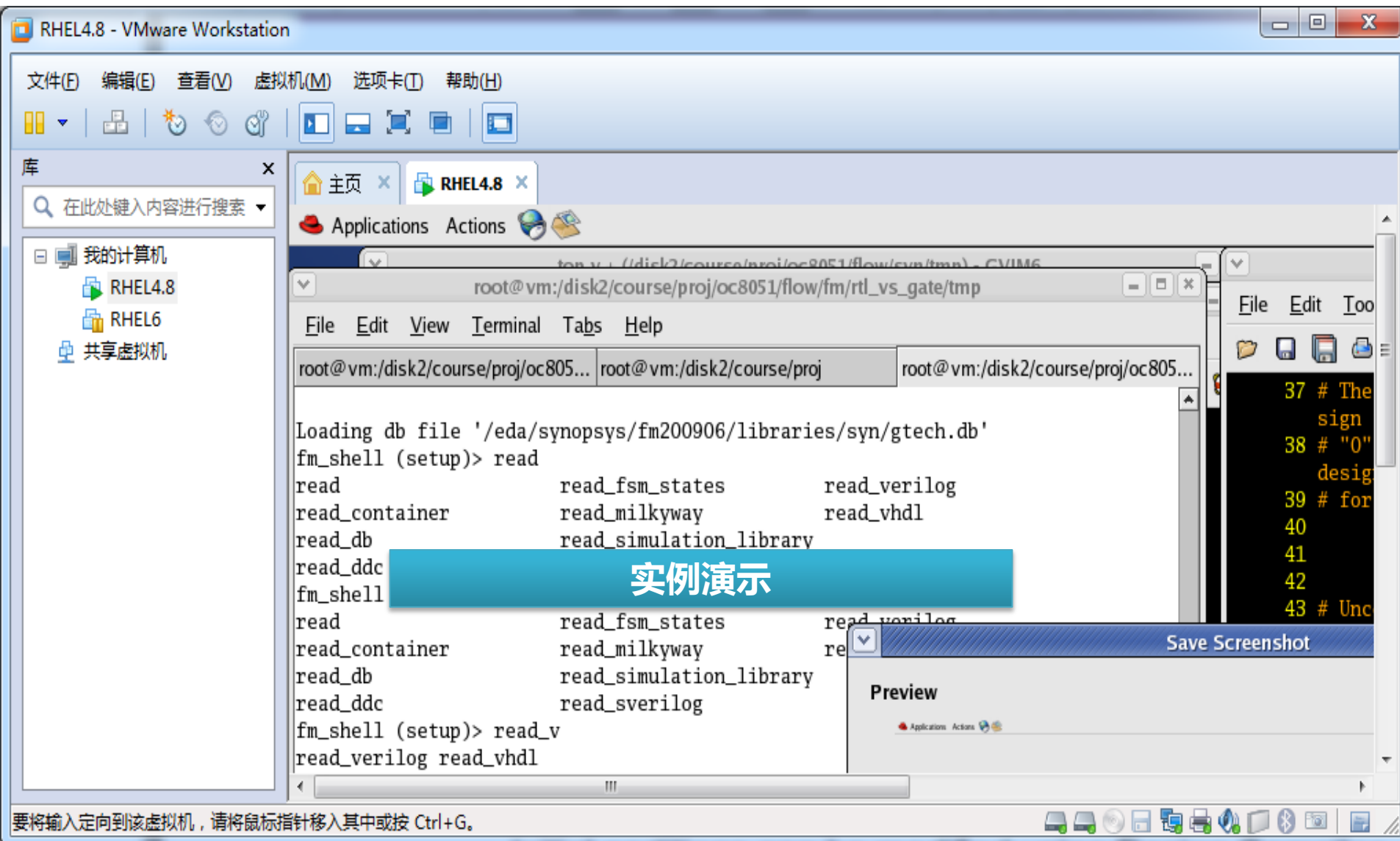
3.物理实现：从Mapped.v 到 GDS



4.形式验证：mapped.v vs. PnR.v



4.形式验证：mapped.v vs. PnR.v



RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片

再看版图

对这个**GDS**有什么要求呢？

1. 功能，要和**RTL**的一样；
2. 性能，要满足预定的性能目标；
3. 物理规则，**Fab**能拿它正常制造；
4. 功耗&面积，**IR Drop**满足要求，
功耗&面积要尽可能小；



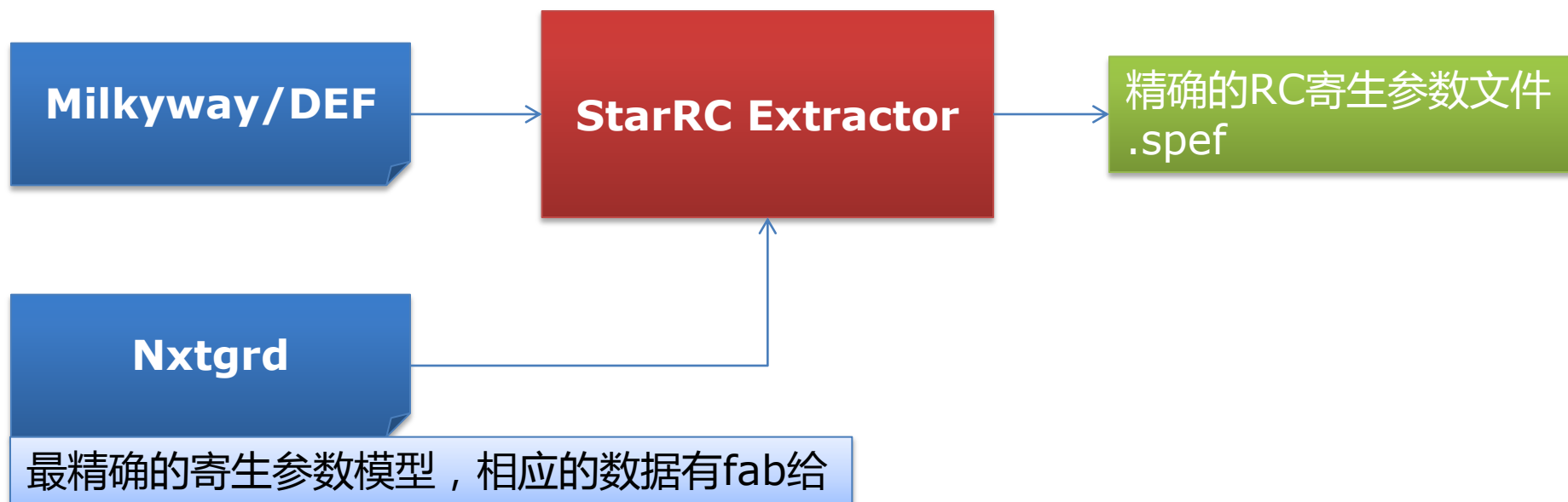
5. Signoff STA: StarRC + PT

Notes :

1. STA : Static Timing Analysis , 简称STA。是相对于使用激励动态仿真而言的 , STA通过静态分析delay并检查时序是否满足 ;
2. Signoff STA : 可以理解为最终的、权威的STA , 此处timing要是过了就人为性能就满足要求了 ;
3. Signoff STA要求使用最精确的RC寄生参数和STA算法 ;
4. 高精度的RC参数用synopsys的StarRC工具提取 , Cadence相应的工具叫QRC ;
5. STA工具用synopsys的PrimeTime (PT) , Cadence相应的工具叫ETS ;

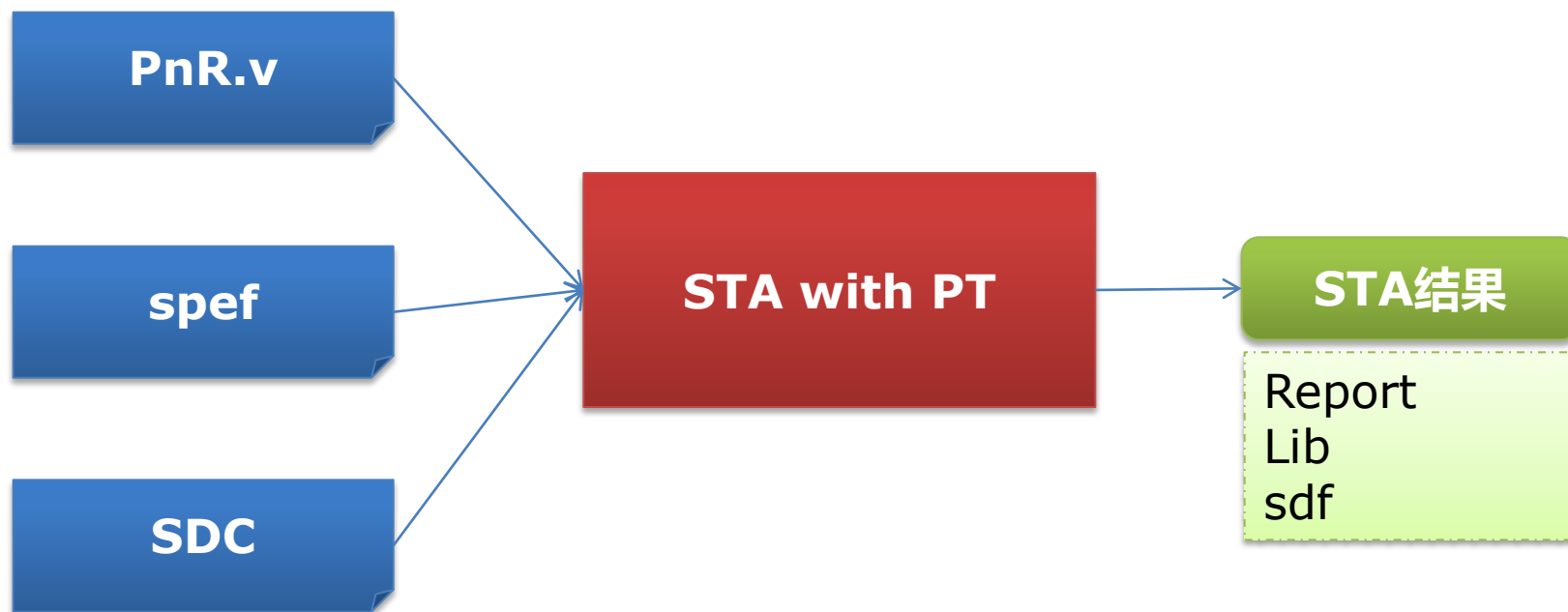
5.Signoff STA: StarRC + PT

StarRC提取寄生参数

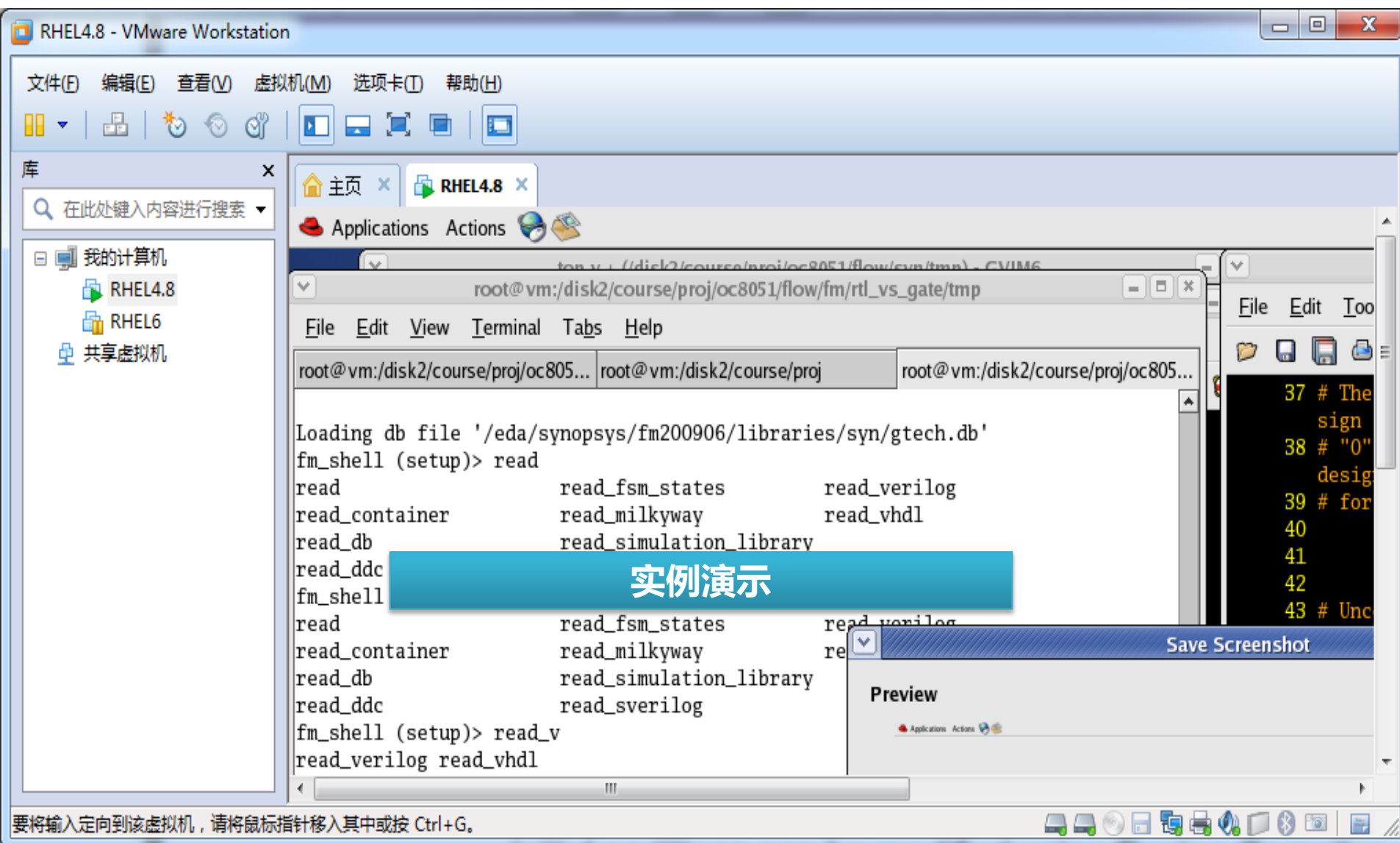


5.Signoff STA: StarRC + PT

STA: 使用PT



5.Signoff STA: StarRC + PT



RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片

再看版图

对这个**GDS**有什么要求呢？

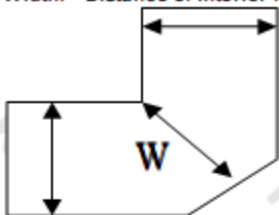
1. 功能，要和**RTL**的一样；
2. 性能，要满足预定的性能目标；
3. 物理规则，**Fab**能拿它正常制造；
4. 功耗&面积，**IR Drop**满足要求，
功耗&面积要尽可能小；

5.Physical Verification : DRC , LVS , Antenna

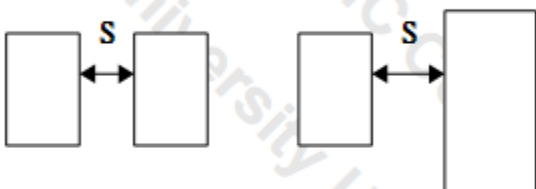
DRC : 物理规则检查 (Design Rule Check)

为什么需要DRC呢？

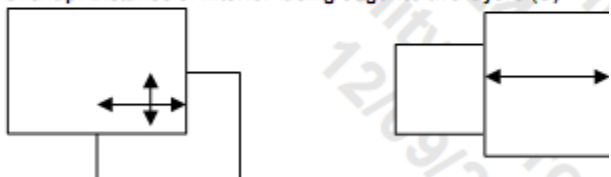
Width: Distance of interior-facing edge for single layer. (W)



Space: Distance of exterior-facing edge for one or two layer (S)



Overlap: Distance of interior-facing edge for two layers (O)



Enclosure: Distance of inside edge to outside edge (Fully inside)

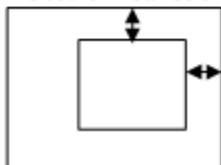


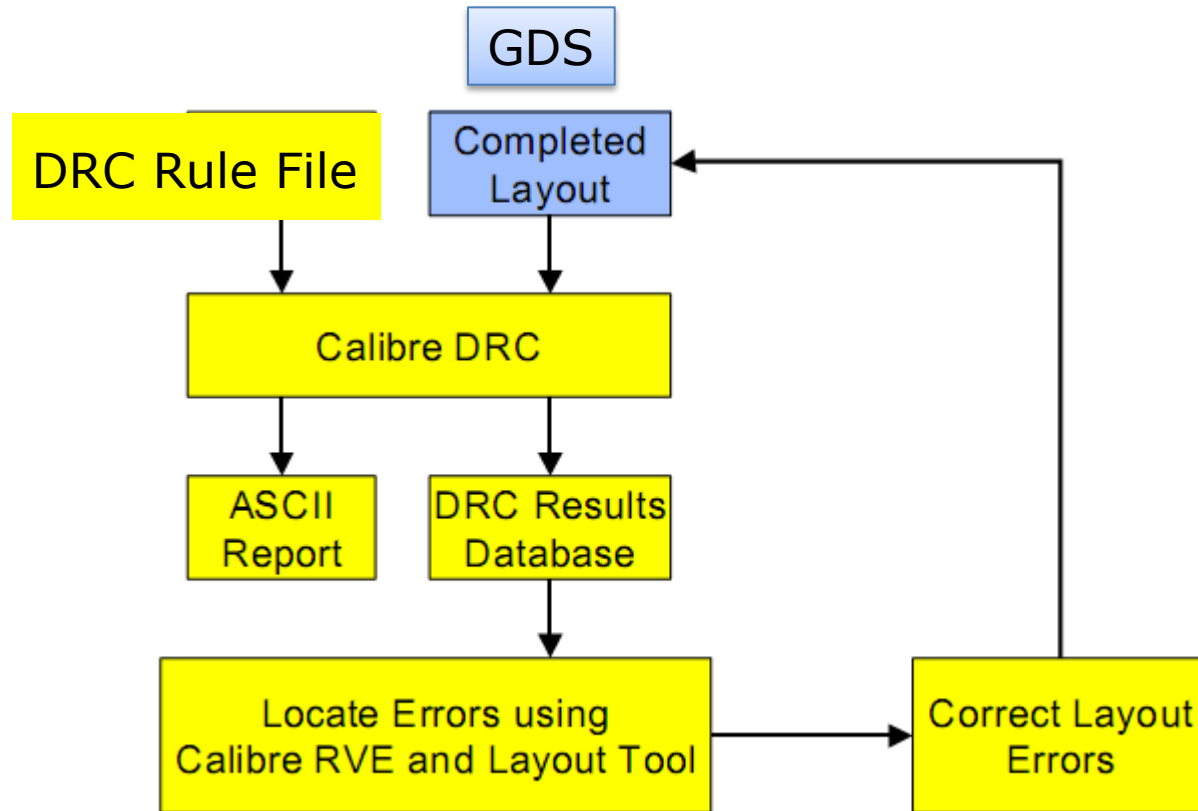
Figure 3.2.3.1: Pad enclosure window (CB)

Rule No.	Description	Label	Rule
Single in-line (Figure 3.2.3.1) / Staggered (Figure 3.2.3.2) / Tri-Tier (Figure 3.2.3.3)			
#CB.W.1	Width of CB	A	≥ Table 3.2.3.1
#CB.W.2	Length of CB (the edge perpendicular to nearby chip edge)	A1	≥ Table 3.2.3.1
CB.W.3	Width of pad metal under Mtop-2 within CB region Staggered: M1, M2~Mtop-3 under inner pad Tri-tier: M1, M2~Mtop-3 under middle pad M2~Mtop-3 under inner pad C015/C018 CUP pad are excluded.	E	≥ Table 3.2.3.1
CB.W.4 ^u	Recommended total width of all metal layers connecting with bond pad. (Refer to the "I/O ESD Protection Guideline" section in the corresponding design rule manual.)	W	≥ 20
#CB.S.1	Space of two CB	B	≥ Table 3.2.3.1
CB.S.2	Space of metal pad geometries, or space of metal pad to metal line DRC check metal pad by the following definitions: (1) {Mx AND [CB sizing 3]}, or (2) {(Mtop/Mtop-1) AND [CB sizing 3]} for C015/C018 CUP pad	F	≥ 2.5
#CB.S.3	Space of the different pad geometries [(Single to Stagger), (Single to Tri-tier) or (Stagger to Tri-tier)]	B1	≥ 15
#CB.P.1 ^u	Pitch of (4 pads at the chip corner) Note: Starting from the corners, the first four pitches must be larger than the pitch in the regularly repeating pad center area.	C1	≥ Table 3.2.3.1
#CB.P.2 ^u	Pitch of x-direction (inner and middle pad of Tri-Tier)	C2	≥ 13 ≤ 40
#CB.P.3 ^u	Pitch of x-direction [(inner and outer pad of Staggered) or (middle and outer pad of Tri-Tier)]	C3	≥ Table 3.2.3.1
#CB.P.4 ^u	Pitch of y-direction (between inner-middle-outer pad) Staggered: inner-outer pad Tri-Tier: middle-outer pad, and inner-middle pad	C4	≥ Table 3.2.3.1
CB.EN.1	CB enclosure by Mx and Mtop in pad regions Except, Staggered: M1, M2~Mtop-3 under inner pad Tri-Tier: M1, M2~Mtop-3 under middle and inner pad C015/C018 CUP pad: M1~Mtop-2	D	≥ 2 (All process)
#CB.R.1	For pad pitch < 55μm, one pair of notches on each pad CB is required, to be the reference point for probing and bonding. Please refer to Figure 3.2.3.1 for the size and location.		

Fab的要求，会给ic后端一个drc的规则文件，包含所有物理要求

5. Physical Verification : DRC , LVS , Antenna

DRC : 物理规则检查 (Design Rule Check)

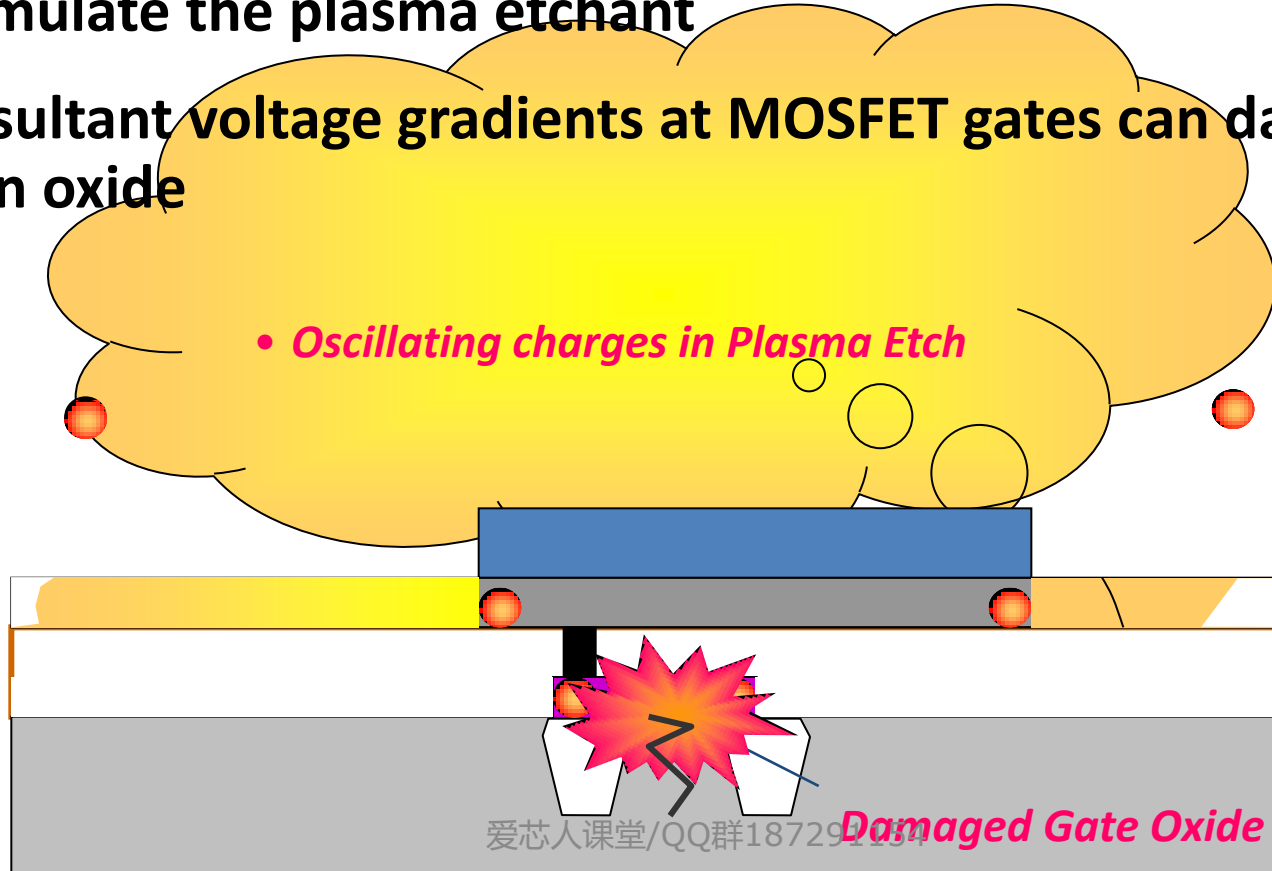


5. Physical Verification : DRC , LVS , Antenna

Antenna Check : 天线规则检查

为什么需要Antenna Check呢？

- Metal wires (antennae) placed in an EM field generate voltage gradients
- During the metal etch stage, strong EM fields are used to stimulate the plasma etchant
- Resultant voltage gradients at MOSFET gates can damage the thin oxide

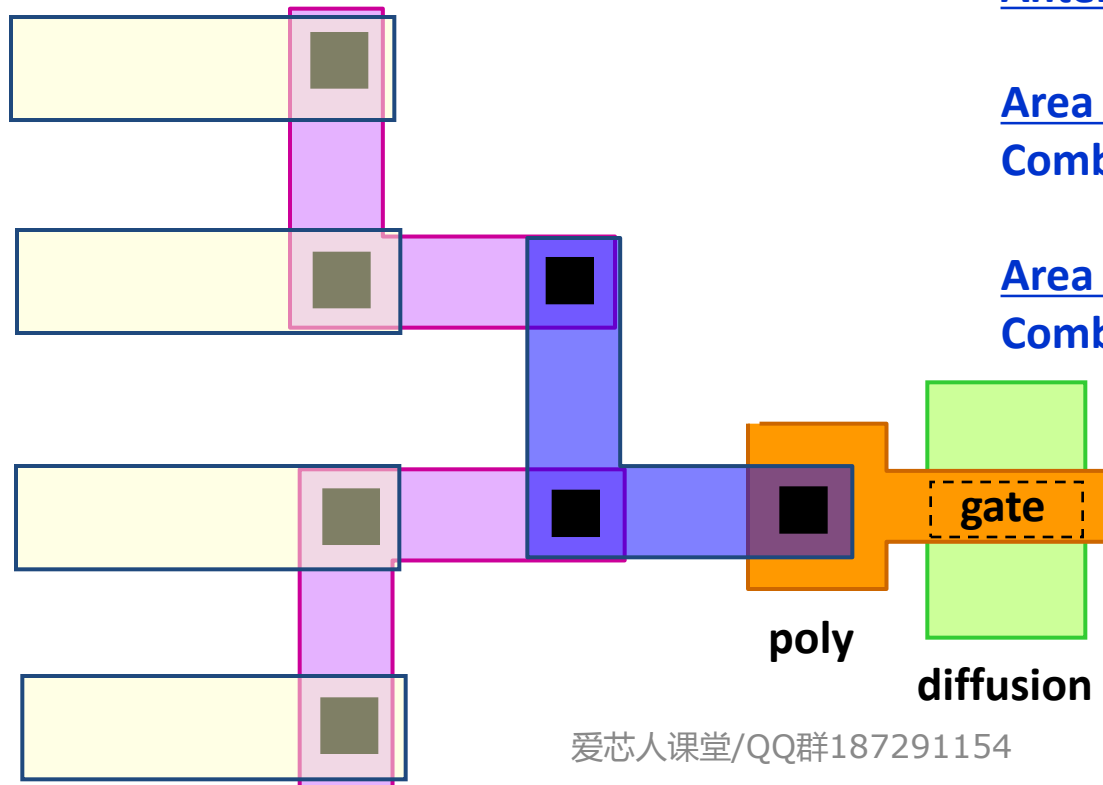


5. Physical Verification : DRC , LVS , Antenna

Antenna Check : 天线规则检查

为什么需要Antenna Check呢？

- As length of wire increases during processing, the voltage stressing the gate oxide increases
- Antenna rules define acceptable length of wires



Antenna Ratios:

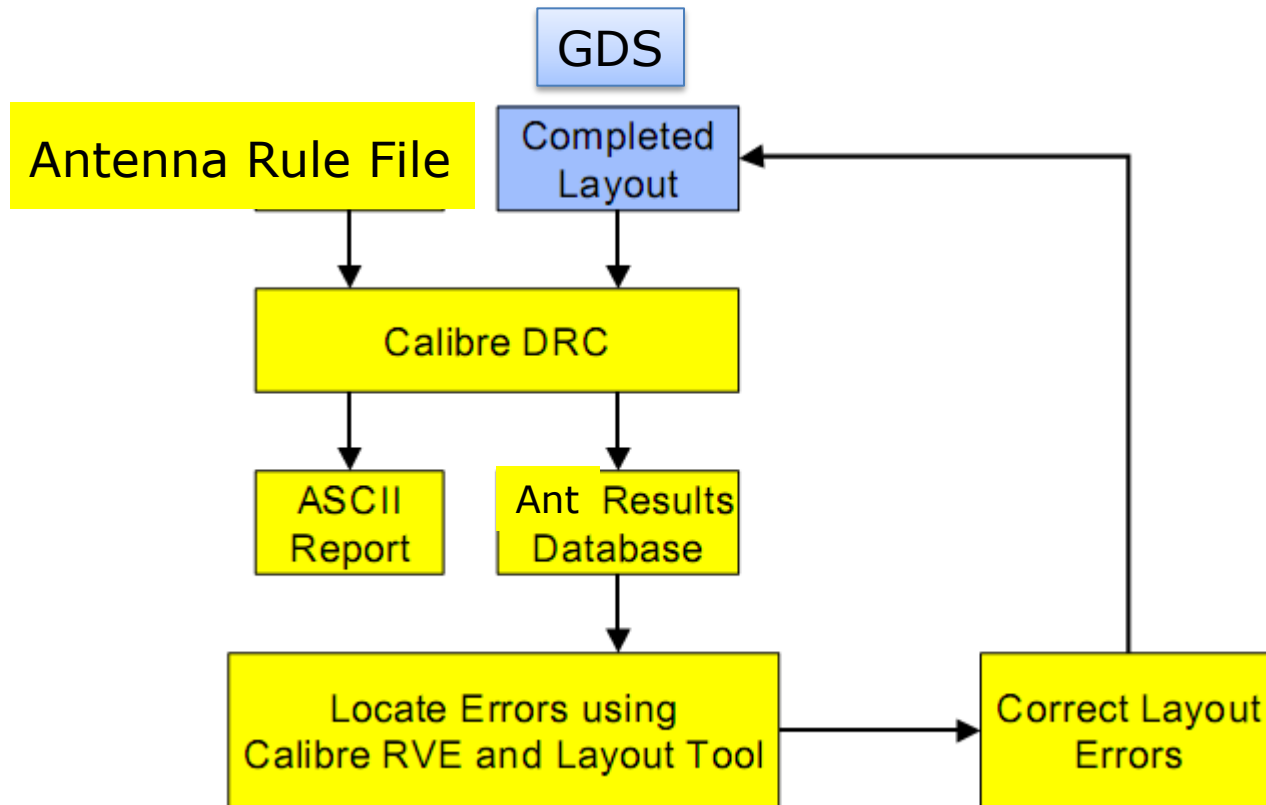
Area of Metal Connected to Gate
Combined Area of Gate

Or

Area of Metal Connected to Gate
Combined Perimeter of Gate

5. Physical Verification : DRC , LVS , Antenna

DRC : 物理规则检查 (Design Rule Check)



RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片

再看版图

对这个**GDS**有什么要求呢？

1. 功能，要和**RTL**的一样；
2. 性能，要满足预定的性能目标；
3. 物理规则，**Fab**能拿它正常制造；
4. 功耗&面积，**IR Drop**满足要求，
功耗&面积要尽可能小；

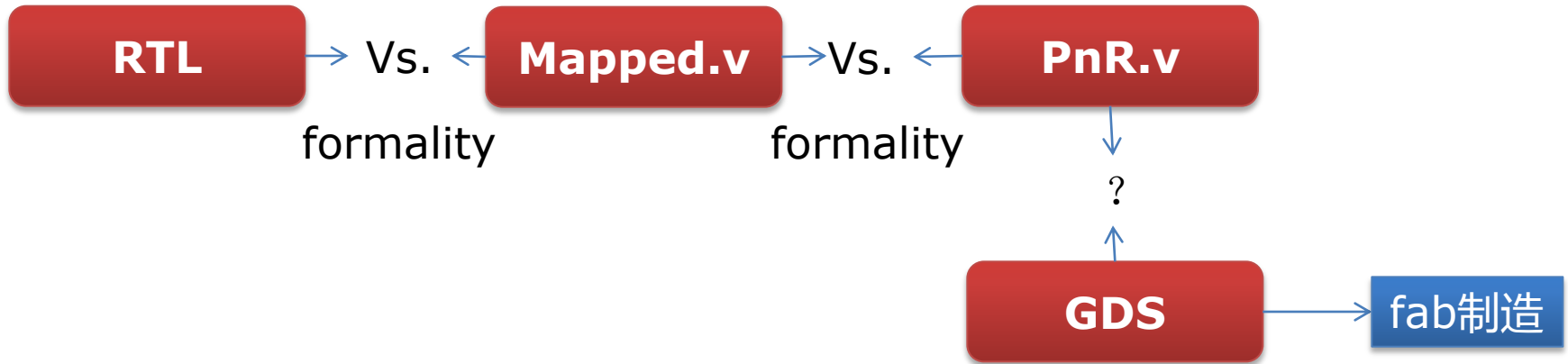
?



5. Physical Verification : DRC , LVS , Antenna

LVS : Layout vs. Schematic

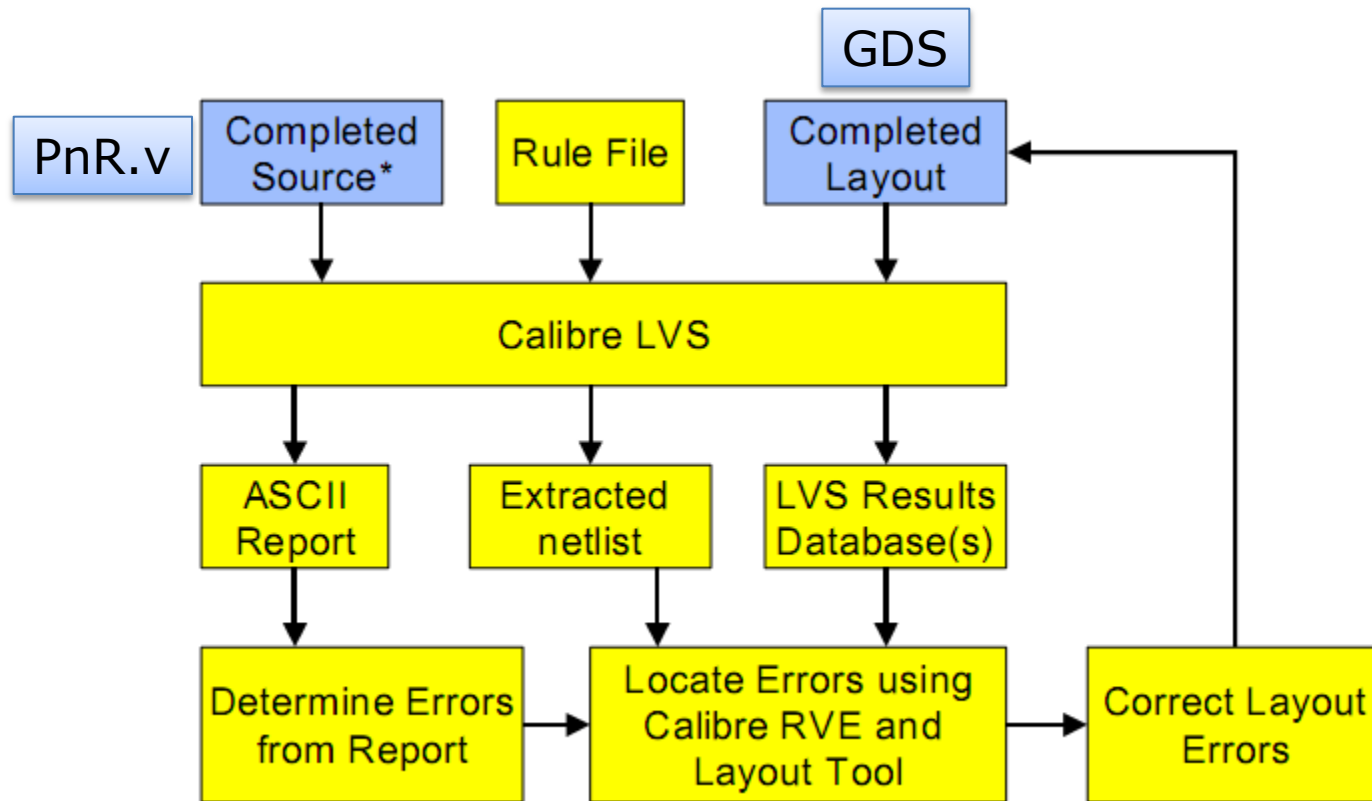
为什么需要LVS呢？



如何保证拿去fab制造的GDS功能和RTL是等价的呢？

5. Physical Verification : DRC , LVS , Antenna

LVS : Layout vs. Schematic



RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

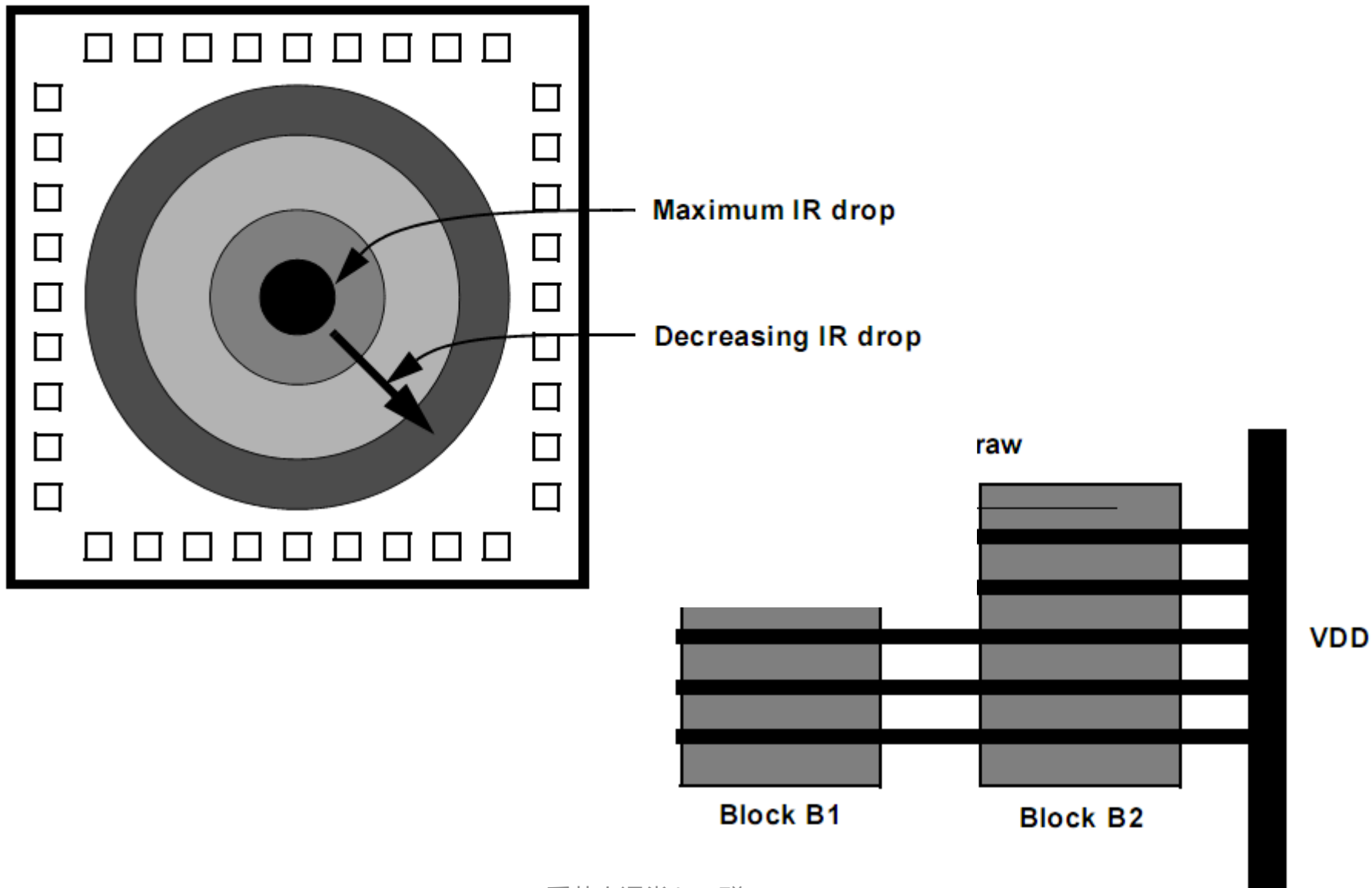
芯片

再看版图

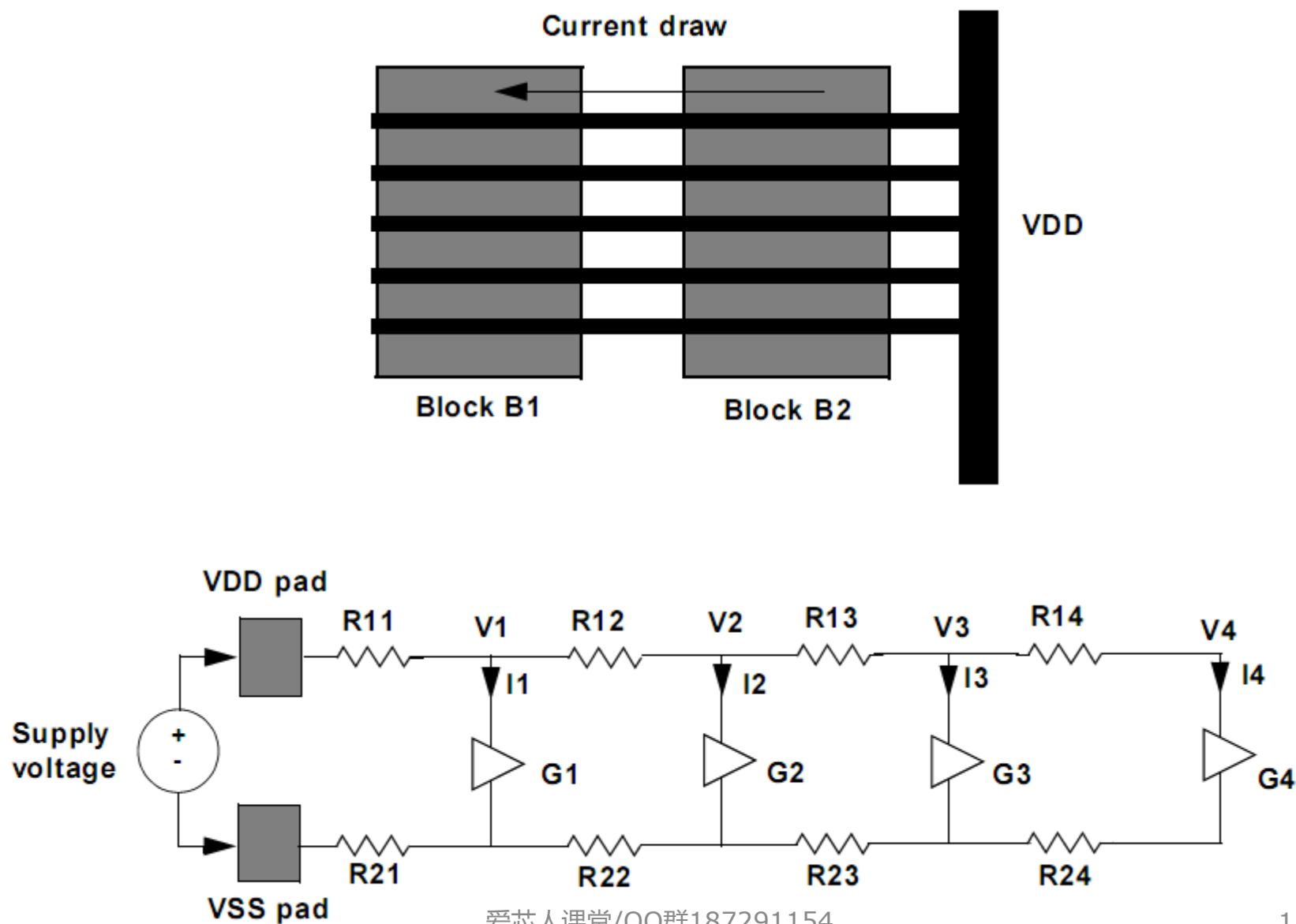
对这个**GDS**有什么要求呢？

1. 功能，要和**RTL**的一样；
2. 性能，要满足预定的性能目标；
3. 物理规则，**Fab**能拿它正常制造；
4. 功耗&面积，**IR Drop**满足要求，
功耗&面积要尽可能小；

6. Power Signoff: IR Drop



6. Power Signoff: IR Drop



6. Power Signoff: IR Drop

- **Static IR drop**

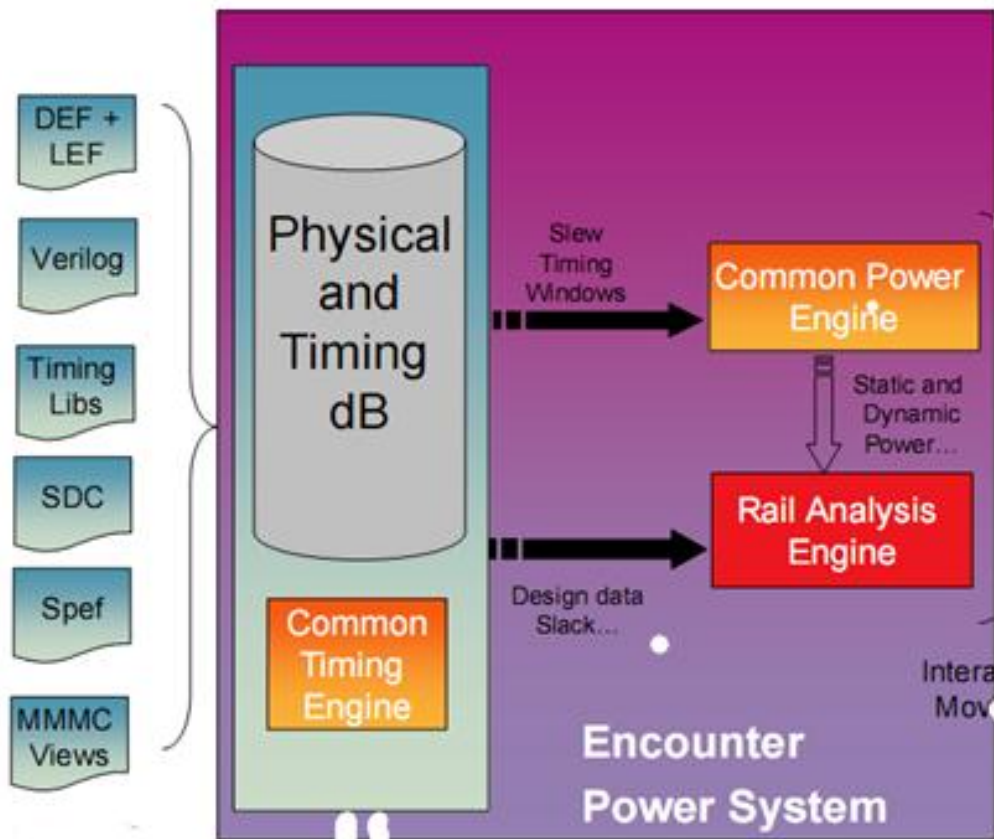
- 3% for VDD + VSS (Flip chip)
- 5% for VDD + VSS (wire bond)

- **Dynamic IR drop**

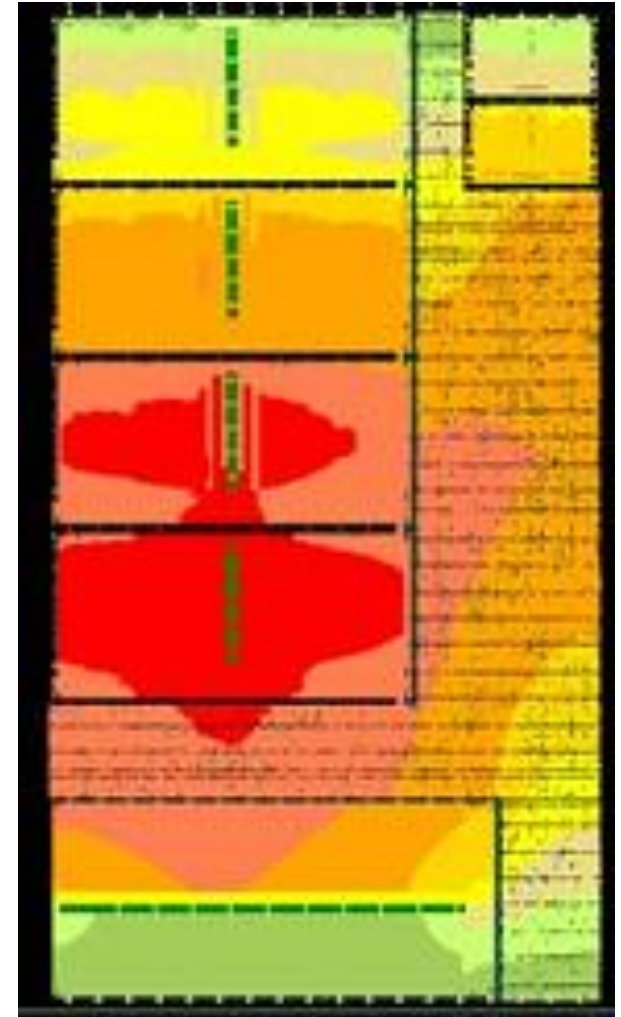
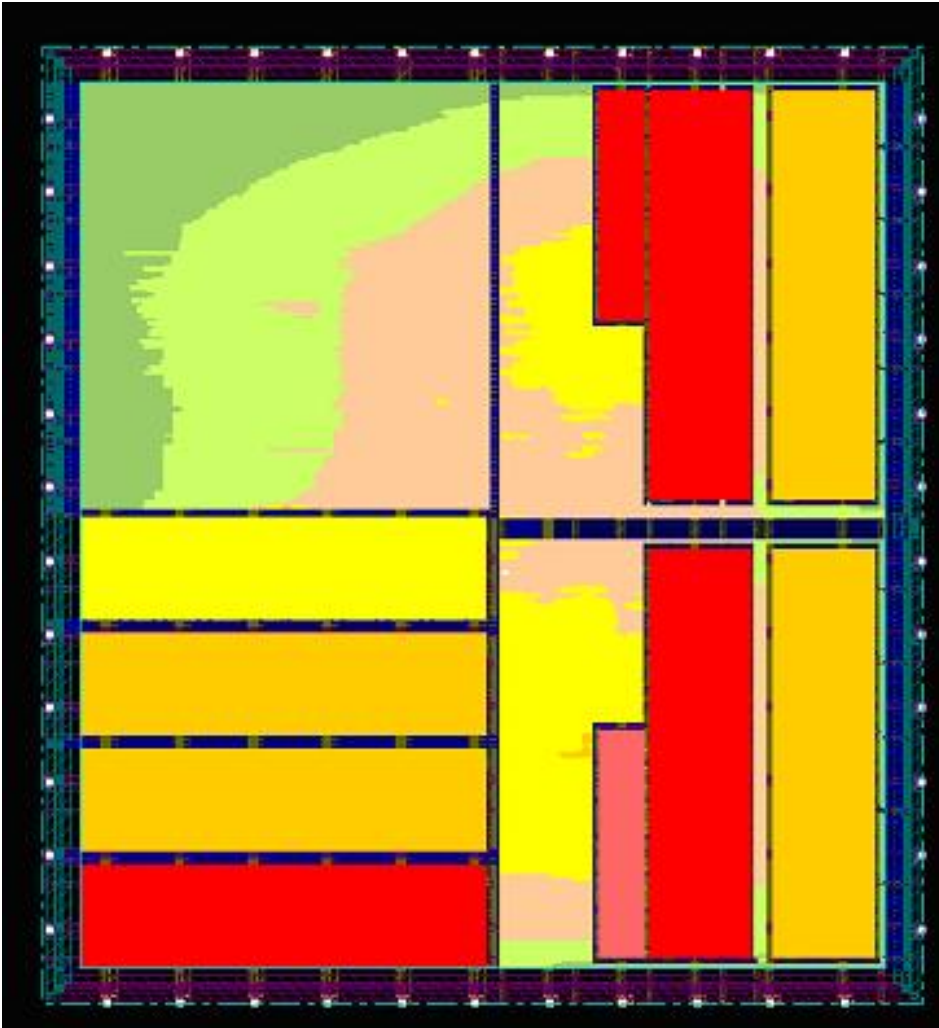
- Around 5X in signoff constraint
- Scan Mode IR drop
 - ◆ Peak power usually around clock-edge
 - ◆ Analyzing IR drop during small timing window when flops are switching at the same time

6. Power Signoff

工具：常用的有Cadence的EPS和Synopsys的PrimeRail



6. Power Signoff: IR Drop



RTL文件

后端
设计

GDS 版图

制造/
封装/
测试

芯片

再看版图

对这个**GDS**有什么要求呢？

1. 功能，要和**RTL**的一样；
2. 性能，要满足预定的性能目标；
3. 物理规则，**Fab**能拿它正常制造；
4. 功耗&面积，**IR Drop**满足要求，
功耗&面积要尽可能小；

One more thing ...

DFT is not include.

内容

数字IC后端流程

关于后端学习

关于爱芯人课堂

从事数字后端设计，你需要：

- 1.理解后端设计流程中的诸多概念，理解流程中每一步的工作；
- 2.学会使用流程相关的工具，遇到问题知道如何分析，能找到候选解决方案并尝试；
熟练使用工具常用命令和部分GUI操作，提高分析和解决问题的效率；
- 3.需要了解Verilog语法，至少要能看懂；
- 4.熟练使用TCL脚本；
- 5.熟练使用Linux环境，熟练使用grep/sed/awk/perl/shell脚本等基本用法，最好精通一种linux脚本语言；
- 6.熟练使用vim/emacs中一种文本编辑器；

关于后端学习

理论

实践

1. 工具的Student Guide, User Guide, help, man, printvar等；
2. 后面的专题课；
3. 概念性的东西要理解透，要知道是什么，为什么；
4. **Solvnet**，百度

1. 做一个项目，从RTL到GDS，按照流片的标准来；
2. 遇到问题一定要分析，找到解决办法；
3. 一定要养成使用脚本的习惯，包括tcl和linux下的工具或脚本。这将决定你今后在技术上能否达到一个较高的层次；
4. 关于跑lab或者今后按照一个给你的脚本跑项目，学习阶段一定要看看每一步都干了什么，不要用make等；
5. 学习阶段不妨去学习给你的脚本中各变量和命令是干什么的，这将引出很多需要你去理解的概念；

后端设计本身不难，但是学习初期会遇到很多不知道的概念容易受挫，但是后端概念不多，学习一条就少一条不懂的！

内容

数字IC后端流程

关于后端学习

关于爱芯人课堂

关于爱芯人

爱芯人

爱新人

爱心人

爱芯人/QQ群187291154

在爱芯人... 你只需付出一点点，就能获得超值回报！！！！

后续课程讲flow上的细节

课堂会安排项目，并按照流片标准来实践~

要求大家认真对待每次课程和项目~

每次课后认真评价，认真反馈意见~



数字后端设计流程

完

谢谢大家，幸苦了！

爱芯人，只为芯人思考，帮助芯人高效学习！

爱芯人课堂/QQ群**187291154**

在爱芯人课堂，你只需付出一点点，就能找个好工作！